Computer Architecture

Lecture 25: Accelerating Biological Sequence Analysis via New Architectures and Algorithms

Nika Mansouri Ghiasi
Prof. Onur Mutlu
ETH Zürich

Fall 2025

19 December 2025

Brief Self Introduction

- **Nika Mansouri Ghiasi**
  - PhD candidate, advised by Prof. Onur Mutlu
  - SAFARI Research Group @ ETH Zurich
  - Department of Information Technology and Electrical Engineering (D-ITET)
  - Worked as a visiting researcher at RSG @ Stanford

- **Research Interests**
  - Computer architecture
  - Data-intensive health-related applications
  - Bioinformatics
  - Storage systems

- **Contact and Links**
  - n.mansorighiasi@gmail.com
  - https://sites.google.com/view/nikamansourighiasi
  - https://safari.ethz.ch/

Agenda for Today

- Brief Overview of (Meta)Genomics
- The Need for Accelerating (Meta)Genomics
- Algorithm-Architecture Co-Design for Accelerating (Meta)Genomics
  - Some Examples
- Diving Deeper with the Examples of Storage-Centric System Design for (Meta)Genomics
- Introducing the Architecture for Health (Arch4Health) Initiative

Brief Overview of (Meta)Genomics

Applications of Genomics and Metagenomics



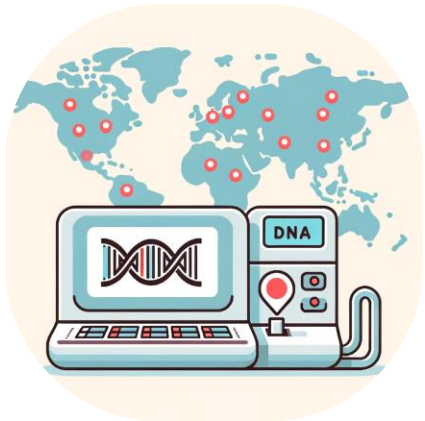
Uncovering and treating diseases linked to genomic variations



Developing **personalized medicine**

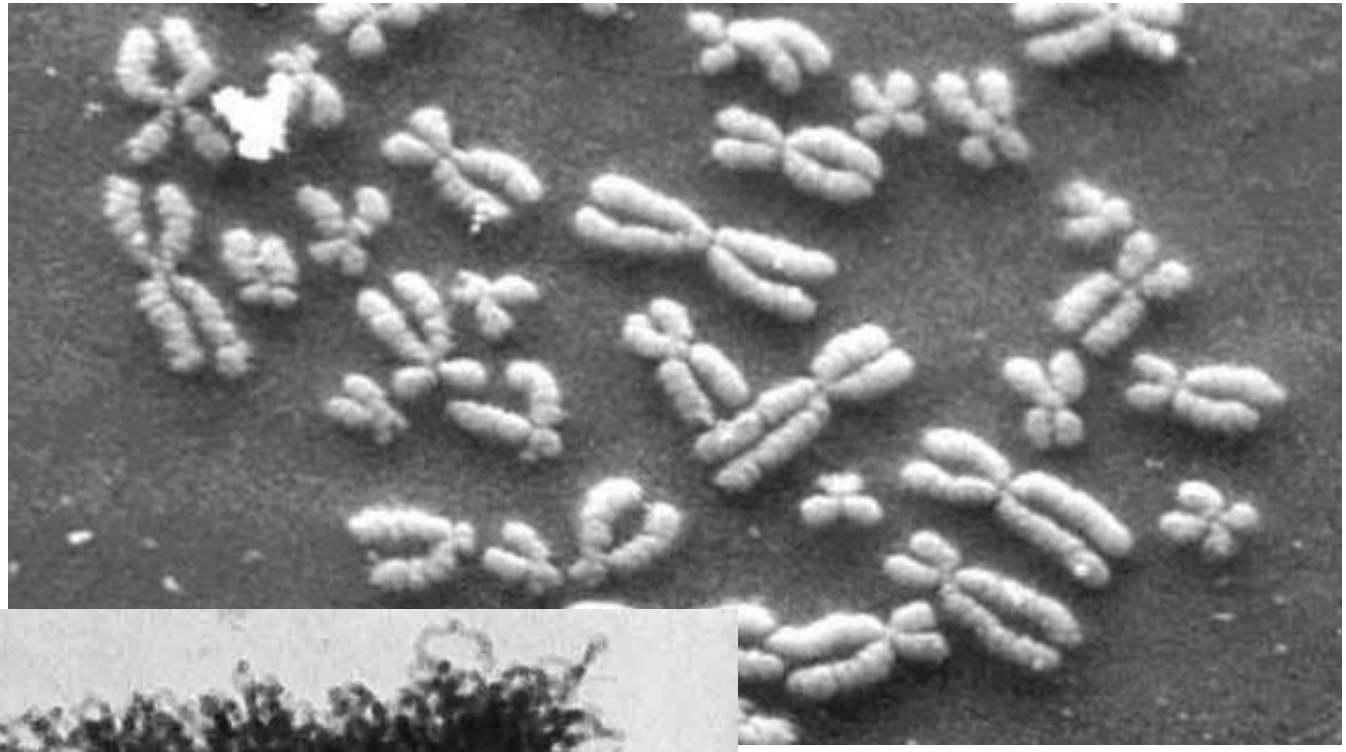


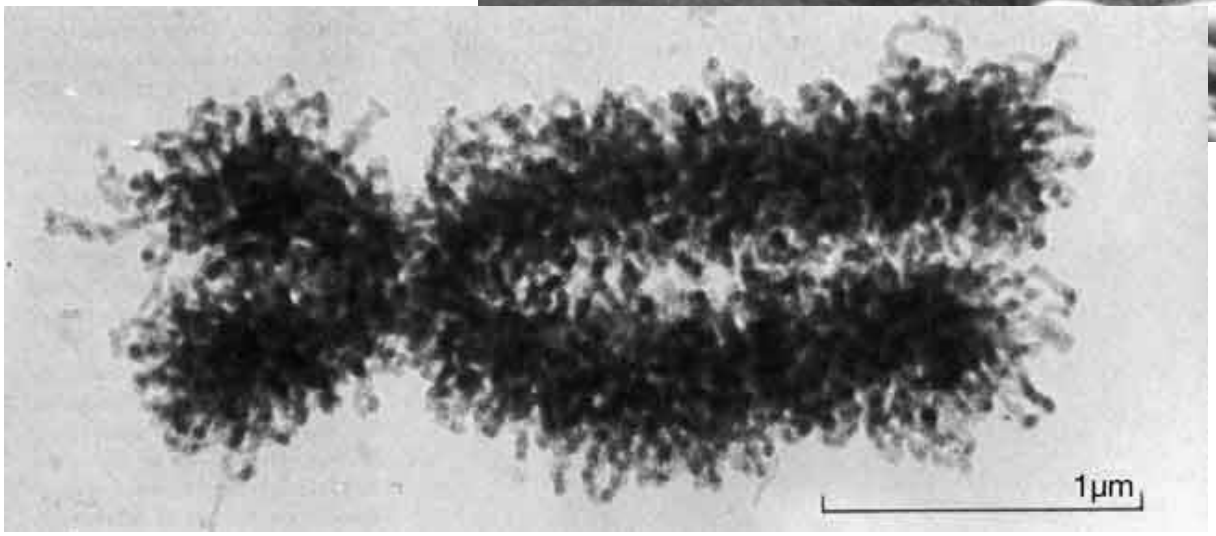
Detecting **pathogens** in the environment



Rapid surveillance of **disease outbreaks**

DNA Under Electron Microscope





CCTCCTCAGTGCCACCCAGCCCACTGGCAGCTCCCAAACA
GGCTCTTATTAAAACACCCTGTTCCCTGCCCCTTGGAGTG
AGGTGTCAAGGACCTAAACTAAAAAAAAAAAAAGAAAA
AGAAAAGAAAAAGAATTTAAAATTTAAGTAATTCTTTGAA
AAAAACTAATTTCTAAGCTTCTTCATGTCAAGGACCTAATG
TGCTAAACAGCACTTTT**TTGACCATTAT**TTTGGATCTGAAA
GAAATCAAGAATAAATGAAGGACTTGATACATTGGAAGA
GGAGAGTCAAGGACCTACAGAAAAAAAAAAAAAAGAAA
AAGAAAAGAAAAAGA**A**TTTAAAATTTAAGTAATTCTTTGA
AAAAAACTAATTTCTAAGCTTCTT**C**ATGTCAAGGACCTAAT
GTCTGTGTTGCAGGTCTTCTTGCATTTCCCTGTCAAAAGA
AAAAGAATTTAAAATTTAAGTAATTCTTTGAAAAAAACTA
ATTTCTAAGCTTCTTCATGTCAAGGACCTAATGTCAGGCC
GGCTCTTATTAAAACACCCTGTTCCCTGCCCCTTGGAGTG

High-Throughput Sequencers



Illumina MiSeq



Pacific Biosciences Sequel II

Oxford Nanopore PromethION





Oxford Nanopore MinION



Illumina NovaSeq 6000



Pacific Biosciences RS II



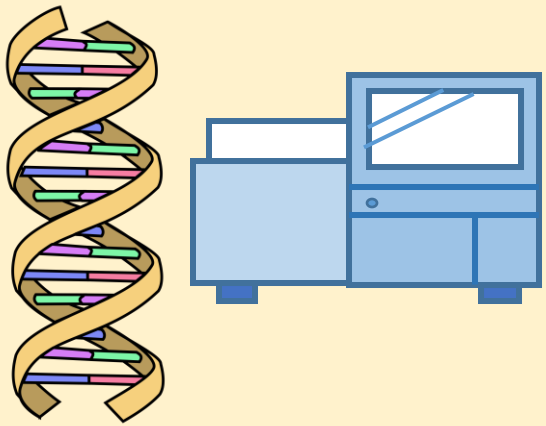
Oxford Nanopore SmidgION

… and more! All produce data with different properties.

High-Throughput Sequencers

Current sequencing technologies cannot sequence very long DNA molecules end-to-end

Instead, they extract smaller fragments of the original DNA sequence, known as reads



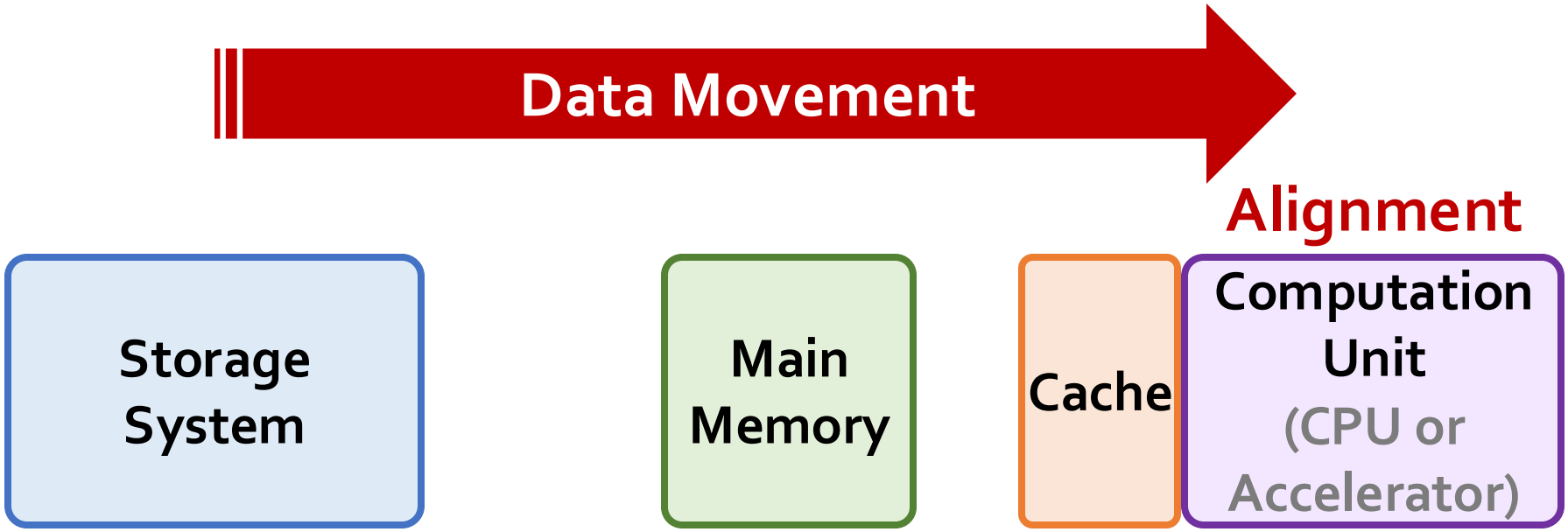
Genome Sequence Analysis

- **Read mapping:** first key step in genome sequence analysis
  - Aligns reads to potential matching locations in the reference genome
  - For each matching location, the alignment step finds the degree of similarity (alignment score)



- Calculating the alignment score requires computationally-expensive approximate string matching (ASM) to account for differences between reads and the reference genome due to:
  - Sequencing errors
  - Genetic variation

Genome Sequence Analysis

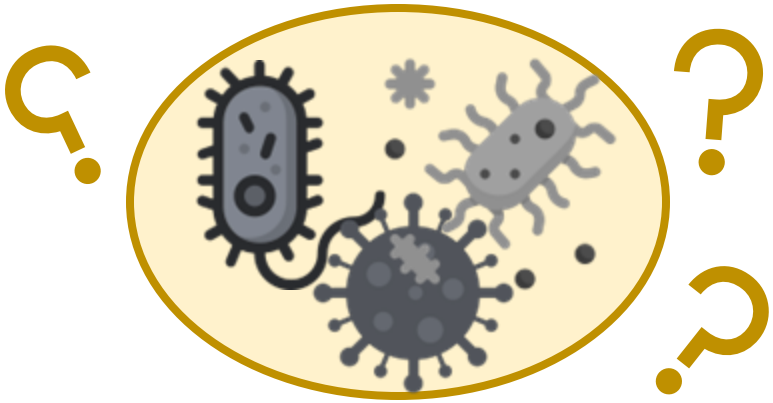


✗ Computation overhead

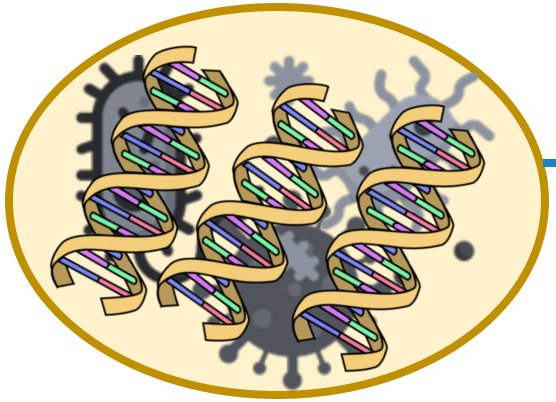
✗ Data movement overhead

Metagenomics

- ***Metagenomics:*** Study of genome sequences of **diverse organisms** within a **shared environment** (e.g., blood, ocean, soil)



Metagenomic Analysis



Metagenomic sample with species that are **not known** in advance



A large database containing information on **many species**

Preparation of Input Queries

Query K-mers

GCTCA

CTCAT

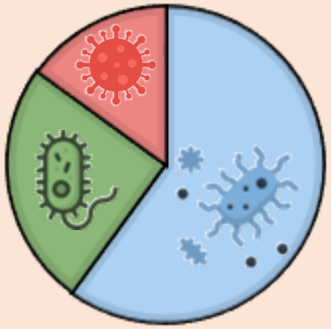
TCATG

...

Presence/Absence Identification

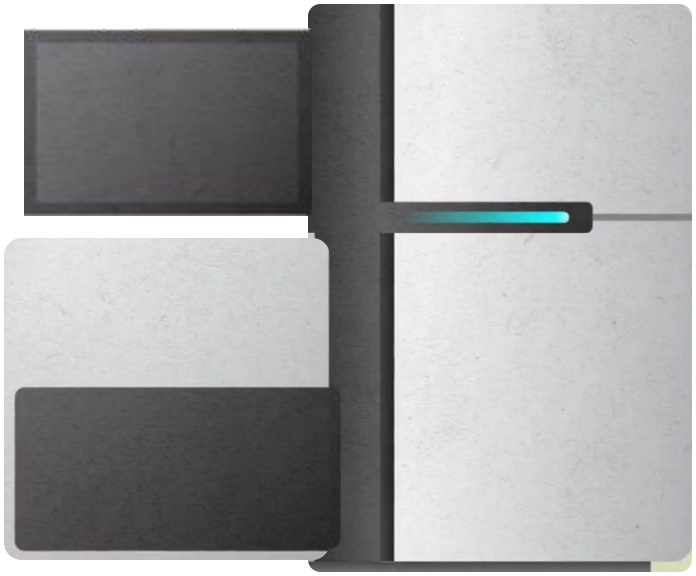


Abundance Estimation

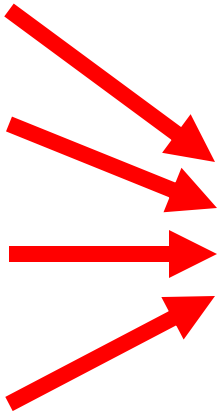


The Need for Accelerating (Meta)Genomics

Problems with (Meta) Genome Analysis Today



Special-Purpose Machine
for **Data Generation**





General-Purpose Machine
for **Data Analysis**

Challenges

Efficiently analyzing large-scale (meta)genomics data poses significant challenges for conventional computing systems



Performance & Scalability



Cost Effectiveness



Energy Efficiency

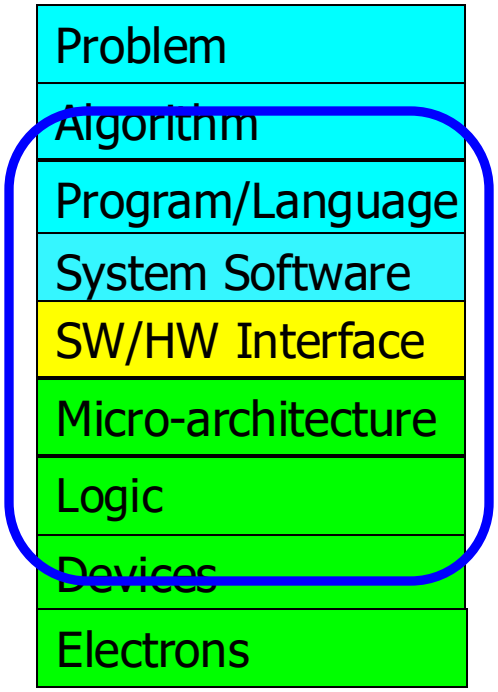


Privacy & Security

Algorithm-Architecture Co-Design for Accelerating (Meta)Genomics

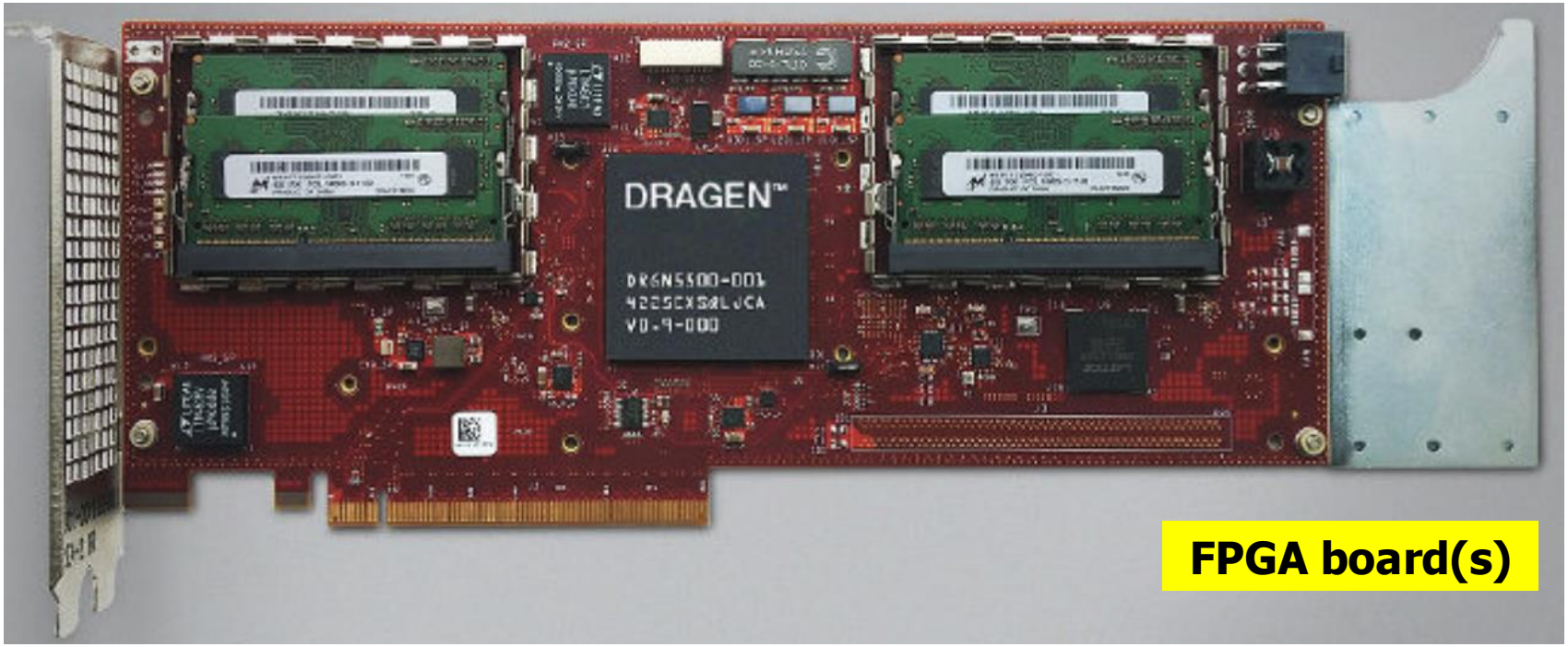
Algorithm-Arch-Device Co-Design is Critical

Computer Architecture (expanded view)



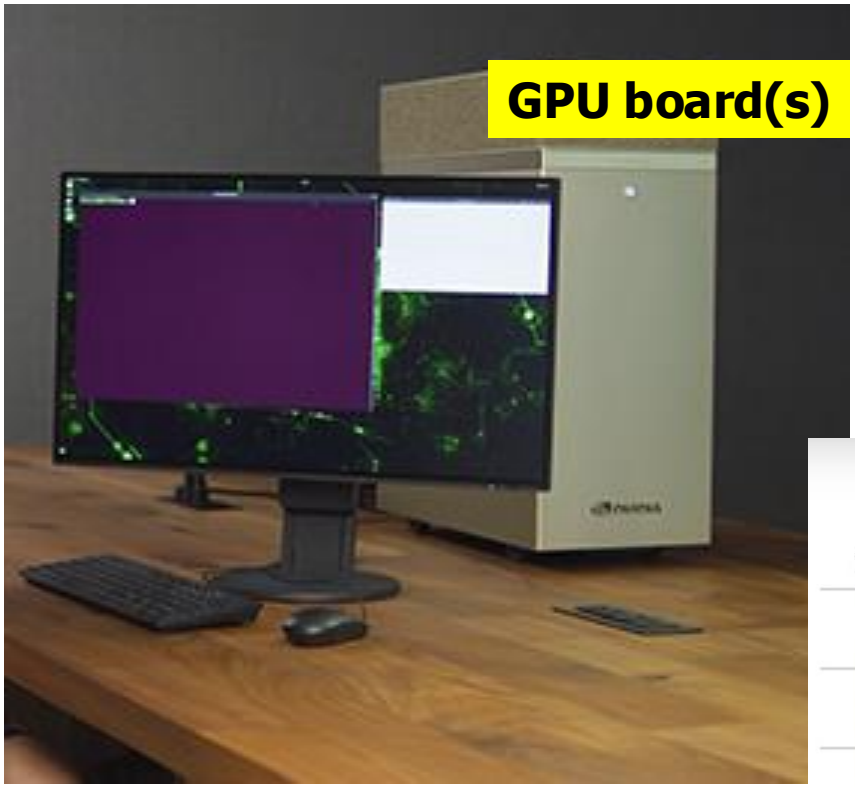
Illumina DRAGEN Bio-IT Platform (2018)

- Processes whole genome at 30x coverage in ~25 minutes with hardware support for data compression

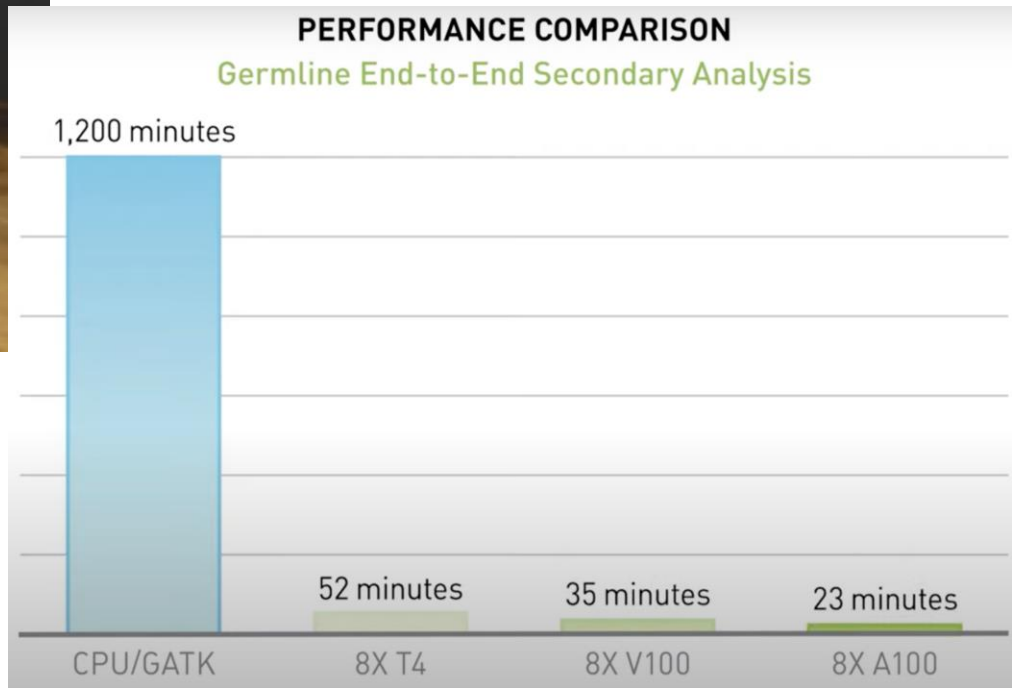


emea.illumina.com/products/by-type/informatics-products/dragen-bio-it-platform.html
emea.illumina.com/company/news-center/press-releases/2018/2349147.html

NVIDIA Clara Parabricks (2020)



A University of Michigan startup in 2018 joined NVIDIA in 2020



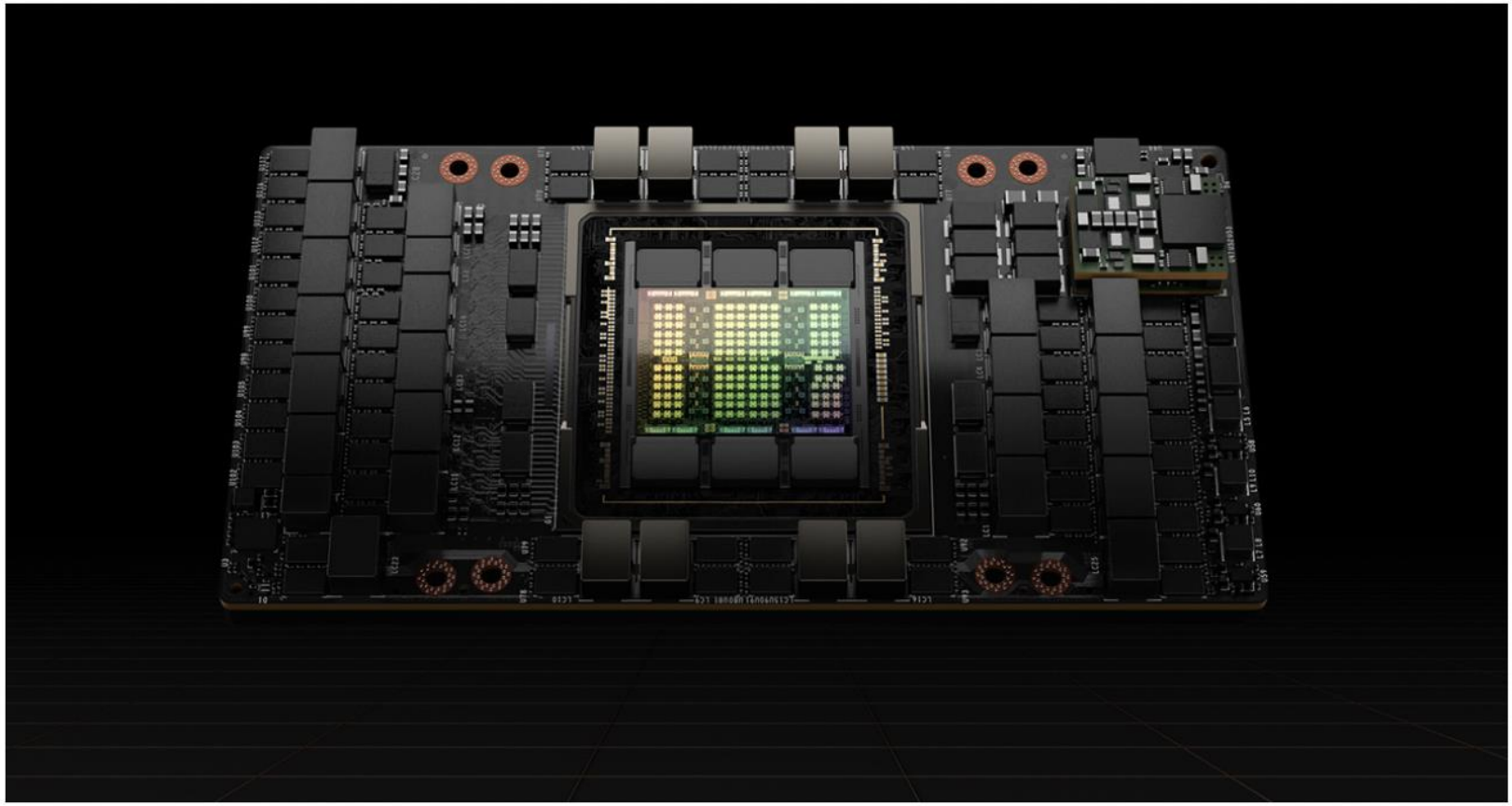
https://developer.nvidia.com/clara-parabricks

NVIDIA Hopper DPX Instructions (2022)

NVIDIA Hopper GPU Architecture Accelerates Dynamic Programming Up to 40x Using New DPX Instructions

Dynamic programming algorithms are used in healthcare, robotics, quantum computing, data science and more.

March 22, 2022 by DION HARRIS



https://blogs.nvidia.com/blog/2022/03/22/nvidia-hopper-accelerates-dynamic-programming-using-dpx-instructions/

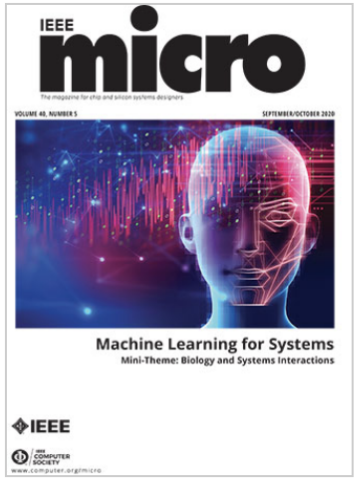
Our Dream (circa 2007)

- An embedded device that can perform comprehensive (meta)genomics analysis in real time
  - Which of these DNAs does this DNA segment match with?
  - What is the likely genetic disposition of this patient to this drug?
  - What disease/condition might this particular DNA/RNA piece associated with?
  - . . .

22

A Few Overview Readings (I)

Mohammed Alser, Zülal Bingöl, Damla Senol Cali, Jeremie Kim, Saugata Ghose, Can Alkan, Onur Mutlu
"Accelerating Genome Analysis: A Primer on an Ongoing Journey"
IEEE Micro, August 2020.



Previous | Next

Table of Contents

Past Issues

Home / Magazines / IEEE Micro / 2020.05

IEEE Micro

Accelerating Genome Analysis: A Primer on an Ongoing Journey

Sept.-Oct. 2020, pp. 65-75, vol. 40
DOI Bookmark: 10.1109/MM.2020.3013728

Authors

Mohammed Alser, ETH Zürich
Zulal Bingol, Bilkent University
Damla Senol Cali, Carnegie Mellon University
Jeremie Kim, ETH Zurich and Carnegie Mellon University
Saugata Ghose, University of Illinois at Urbana–Champaign and Carnegie Mellon University
Can Alkan, Bilkent University
Onur Mutlu, ETH Zurich, Carnegie Mellon University, and Bilkent University

https://arxiv.org/pdf/2008.00961.pdf

A Few Overview Readings (II)

Gagandeep Singh, Mohammed Alser, Damla Senol Cali, Dionysios Diamantopoulos, Juan Gomez-Luna, Henk Corporaal, Onur Mutlu,
"**FPGA-Based Near-Memory Acceleration of Modern Data-Intensive Applications**"
IEEE Micro, 2021.
[Source Code]



Previous | Next

Table of Contents

Past Issues

Home / Magazines / IEEE Micro / 2021.04

IEEE Micro

FPGA-Based Near-Memory Acceleration of Modern Data-Intensive Applications

July-Aug. 2021, pp. 39-48, vol. 41
DOI Bookmark: 10.1109/MM.2021.3088396

Authors

Gagandeep Singh, ETH Zürich, Zürich, Switzerland
Mohammed Alser, ETH Zürich, Zürich, Switzerland
Damla Senol Cali, Carnegie Mellon University, Pittsburgh, PA, USA
Dionysios Diamantopoulos, Zürich Lab, IBM Research Europe, Rüschlikon, Switzerland
Juan Gomez-Luna, ETH Zürich, Zürich, Switzerland
Henk Corporaal, Eindhoven University of Technology, Eindhoven, The Netherlands
Onur Mutlu, ETH Zürich, Zürich, Switzerland

https://arxiv.org/pdf/2106.06433.pdf

A Few Overview Readings (III)

Mohammed Alser, Joel Lindegger, Can Firtina, Nour Almadhoun, Haiyu Mao, Gagandeep Singh, Juan Gomez-Luna, Onur Mutlu
"From Molecules to Genomic Variations: Intelligent Algorithms and Architectures for Intelligent Genome Analysis"
Computational and Structural Biotechnology Journal, 2022
[Source code]





journal homepage: www.elsevier.com/locate/csbj



Review

From molecules to genomic variations: Accelerating genome analysis via intelligent algorithms and architectures



Mohammed Alser *, Joel Lindegger, Can Firtina, Nour Almadhoun, Haiyu Mao, Gagandeep Singh, Juan Gomez-Luna, Onur Mutlu *

ETH Zurich, Gloriastrasse 35, 8092 Zürich, Switzerland

https://arxiv.org/pdf/2205.07957.pdf

25

A Few Overview Readings (IV)

- Onur Mutlu and Can Firtina,
  "Accelerating Genome Analysis via Algorithm-Architecture Co-Design"
  *Invited Special Session Paper in Proceedings of the 60th Design Automation Conference (**DAC**)*, San Francisco, CA, USA, July 2023.
  [arXiv version]



https://arxiv.org/pdf/2305.00492.pdf

Example: GenASM Framework

- Damla Senol Cali, Gurpreet S. Kalsi, Zulal Bingol, Can Firtina, Lavanya Subramanian, Jeremie S. Kim, Rachata Ausavarungnirun, Mohammed Alser, Juan Gomez-Luna, Amirali Boroumand, Anant Nori, Allison Scibisz, Sreenivas Subramoney, Can Alkan, Saugata Ghose, and Onur Mutlu,
  "GenASM: A High-Performance, Low-Power Approximate String Matching Acceleration Framework for Genome Sequence Analysis"
  *Proceedings of the 53rd International Symposium on Microarchitecture (**MICRO**)*, Virtual, October 2020.
  [Lightning Talk Video (1.5 minutes)]
  [Lightning Talk Slides (pptx) (pdf)]
  [Talk Video (18 minutes)]
  [Slides (pptx) (pdf)]

GenASM: A High-Performance, Low-Power Approximate String Matching Acceleration Framework for Genome Sequence Analysis

Damla Senol Cali[†⋈] Gurpreet S. Kalsi[⋈] Zülal Bingöl[▽] Can Firtina[◇] Lavanya Subramanian[‡] Jeremie S. Kim[◇†]
Rachata Ausavarungnirun[⊙] Mohammed Alser[◇] Juan Gomez-Luna[◇] Amirali Boroumand[†] Anant Nori[⋈]
Allison Scibisz[†] Sreenivas Subramoney[⋈] Can Alkan[▽] Saugata Ghose[⋆†] Onur Mutlu[◇†▽]

[†]*Carnegie Mellon University* [⋈]*Processor Architecture Research Lab, Intel Labs* [▽]*Bilkent University* [◇]*ETH Zürich*
[‡]*Facebook* [⊙]*King Mongkut's University of Technology North Bangkok* [⋆]*University of Illinois at Urbana–Champaign*

GenASM: ASM Framework for GSA

Our Goal:

Accelerate approximate string matching
by designing a fast and flexible framework,
which can accelerate *multiple steps* of genome sequence analysis

- **GenASM:** *First* ASM acceleration framework for GSA
  - Based on the *Bitap* algorithm
    - Uses fast and simple bitwise operations to perform ASM
  - Modified and extended ASM algorithm
    - Highly-parallel Bitap with long read support
    - Bitvector-based novel algorithm to perform *traceback*
  - Co-design of our modified scalable and memory-efficient algorithms with low-power and area-efficient hardware accelerators

GenASM: Key Results

(1) Read Alignment

- **116×** speedup, **37×** less power than **Minimap2** (state-of-the-art **SW**)
- **111×** speedup, **33×** less power than **BWA-MEM** (state-of-the-art **SW**)
- **3.9×** better throughput, **2.7×** less power than **Darwin** (state-of-the-art **HW**)
- **1.9×** better throughput, **82%** less logic power than **GenAx** (state-of-the-art **HW**)

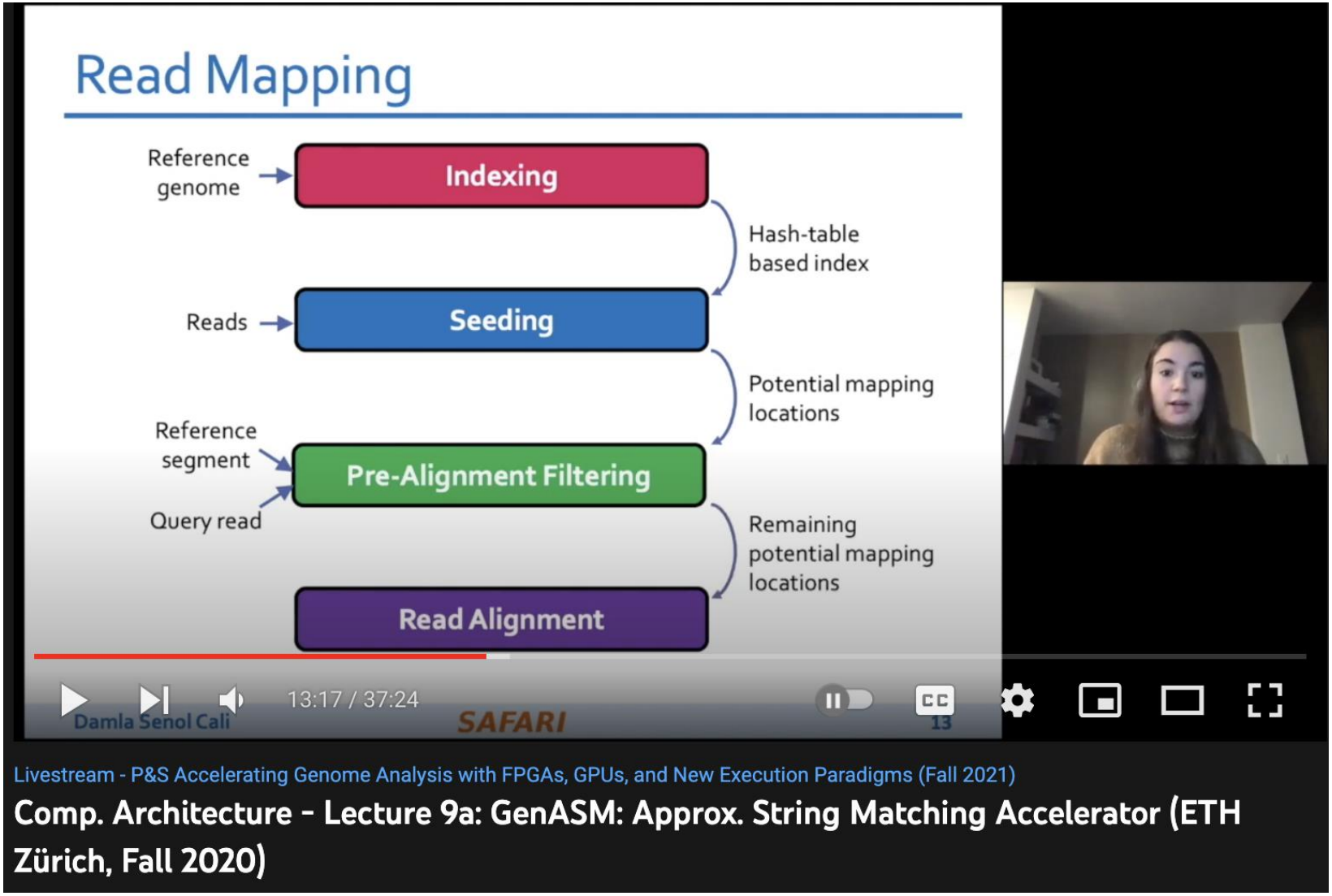
(2) Pre-Alignment Filtering

- **3.7×** speedup, **1.7×** less power than **Shouji** (state-of-the-art **HW**)

(3) Edit Distance Calculation

- **22–12501×** speedup, **548–582×** less power than **Edlib** (state-of-the-art **SW**)
- **9.3–400×** speedup, **67×** less power than **ASAP** (state-of-the-art **HW**)

More on GenASM



Example: Scrooge for Faster ASM

- Joël Lindegger, Damla Senol Cali, Mohammed Alser, Juan Gómez-Luna, Nika Mansouri Ghiasi, and Onur Mutlu,
  "Scrooge: A Fast and Memory-Frugal Genomic Sequence Aligner for CPUs, GPUs, and ASICs"
  Bioinformatics, [published online on] 24 March 2023.
  [Online link at Bioinformatics Journal]
  [arXiv preprint]
  [Scrooge Source Code]

Scrooge: A Fast and Memory-Frugal Genomic Sequence Aligner for CPUs, GPUs, and ASICs

Joël Lindegger[§] Damla Senol Cali[†] Mohammed Alser[§]
Juan Gómez-Luna[§] Nika Mansouri Ghiasi[§] Onur Mutlu[§]

[§]*ETH Zürich* [†]*Bionano Genomics*

Scrooge: Goals

Build a practical and efficient implementation of the GenASM algorithm for multiple computing platforms

Compete with state-of-the-art pairwise sequence aligners like Edlib, KSW2, and BiWFA

Scrooge: Overview

Three novel algorithmic improvements which address inefficiencies in the GenASM algorithm

Efficient open-source implementations for CPUs and GPUs

Key Results

Scrooge consistently outperforms GenASM

- **2.1x speedup over GenASM on CPU**
- **5.9x speedup over GenASM on GPU**
- **3.6x better area efficiency and 3.6x less power than GenASM as an ASIC**

Scrooge consistently outperforms state-of-the-art CPU and GPU baselines, including KSW2, Edlib, and BiWFA

Scrooge on Github

README.md

Scrooge: A fast and memory-frugal genomic sequence aligner for CPUs, GPUs and ASICs

Scrooge is a fast pairwise genomic sequence aligner. It efficiently aligns short and long genomic sequence pairs on multiple computing platforms. It is based on the GenASM algorithm (Senol Cali+, 2020), and adds multiple algorithmic improvements that significantly improve the throughput and resource efficiency for CPUs, GPUs and ASICs. For long reads, the CPU version of Scrooge achieves a 20.1x, 1.7x, and 2.1x speedup over KSW2, Edlib, and a CPU implementation of GenASM, respectively. The GPU version of Scrooge achieves a 4.0x 80.4x, 6.8x, 12.6x and 5.9x speedup over the CPU version of Scrooge, KSW2, Edlib, Darwin-GPU, and a GPU implementation of GenASM, respectively. We estimate an ASIC implementation of Scrooge to use 3.6x less chip area and 2.1x less power than a GenASM ASIC while maintaining the same throughput.

This repository contains Scrooge's CPU and GPU implementations, and several evaluation scripts. We describe Scrooge in our paper on arXiv and in Bioinformatics.

3 forks

Report repository

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Languages

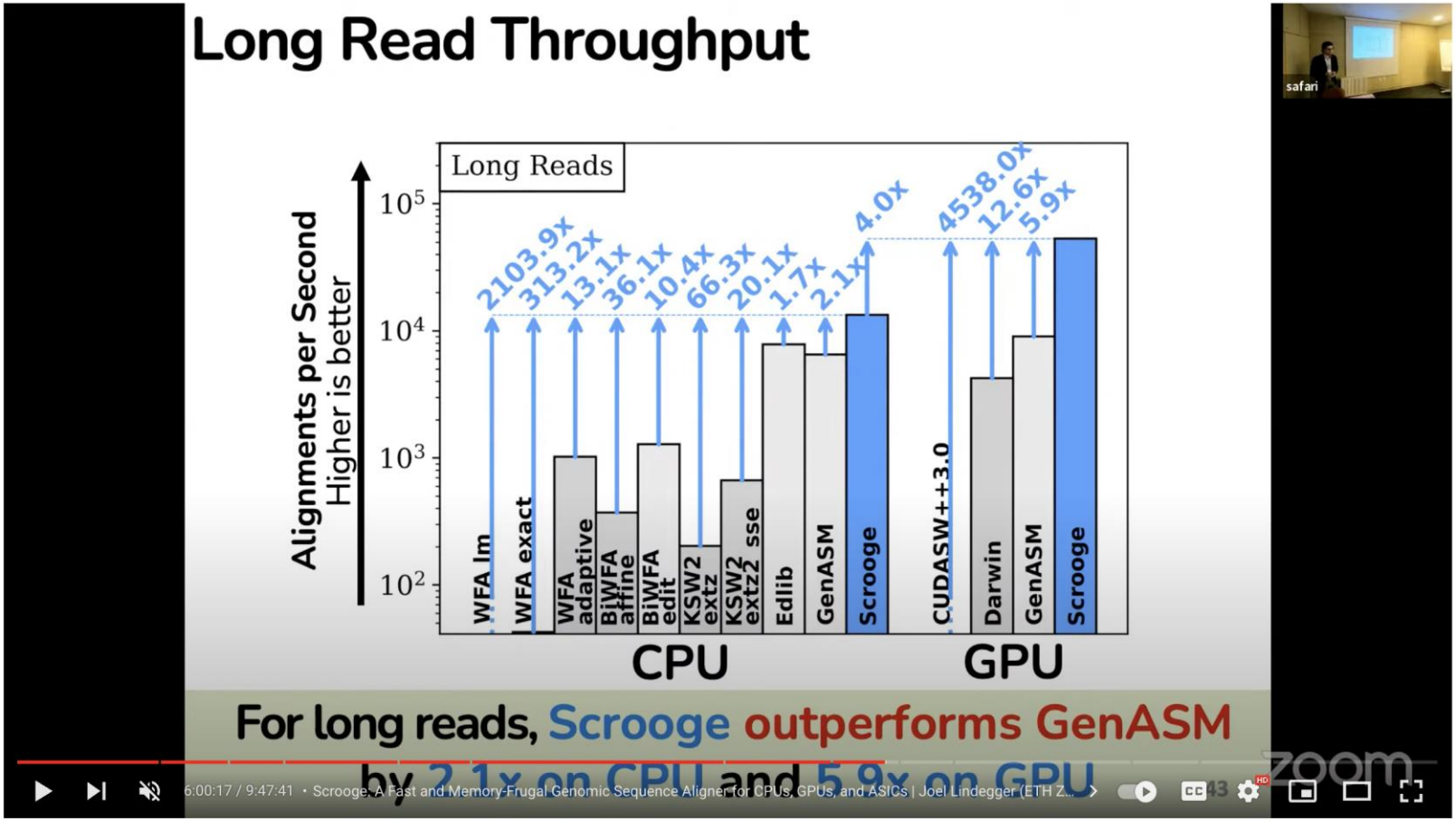
C 33.5% C++ 32.4%

Python 18.1% Cuda 14.5%

Makefile 0.9% Shell 0.6%

Scrooge on GitHub

Scrooge Talk Video



BIO-Arch: Workshop on Hardware Acceleration of Bioinformatics Workloads

Onur Mutlu Lectures
32.9K subscribers

Analytics | Edit video | 68 | Share | Download | Clip | Save

1,447 views Streamed live on Apr 14, 2023
BIO-Arch: Workshop on Hardware Acceleration of Bioinformatics Workloads
https://safari.ethz.ch/recomb23-arch-...

https://www.youtube.com/watch?v=2rCsb4-nLmg&t=20313s

Example: SeGraM

- Damla Senol Cali, Konstantinos Kanellopoulos, Joel Lindegger, Zulal Bingol, Gurpreet S. Kalsi, Ziyi Zuo, Can Firtina, Meryem Banu Cavlak, Jeremie Kim, Nika MansouriGhiasi, Gagandeep Singh, Juan Gomez-Luna, Nour Almadhoun Alserr, Mohammed Alser, Sreenivas Subramoney, Can Alkan, Saugata Ghose, and Onur Mutlu,
  "SeGraM: A Universal Hardware Accelerator for Genomic Sequence-to-Graph and Sequence-to-Sequence Mapping"
  *Proceedings of the 49th International Symposium on Computer Architecture (**ISCA**)*, New York, June 2022.
  [arXiv version]

SeGraM: A Universal Hardware Accelerator for Genomic Sequence-to-Graph and Sequence-to-Sequence Mapping

Damla Senol Cali[1] Konstantinos Kanellopoulos[2] Joël Lindegger[2] Zülal Bingöl[3]
Gurpreet S. Kalsi[4] Ziyi Zuo[5] Can Firtina[2] Meryem Banu Cavlak[2] Jeremie Kim[2]
Nika Mansouri Ghiasi[2] Gagandeep Singh[2] Juan Gómez-Luna[2] Nour Almadhoun Alserr[2]
Mohammed Alser[2] Sreenivas Subramoney[4] Can Alkan[3] Saugata Ghose[6] Onur Mutlu[2]

[1]Bionano Genomics [2]ETH Zürich [3]Bilkent University [4]Intel Labs
[5]Carnegie Mellon University [6]University of Illinois Urbana-Champaign

Genome Sequence Analysis

- Mapping the reads to a reference genome (i.e., ***read mapping***) is a c*ritical step* in genome sequence analysis

Linear Reference: ACGTACGT

Read: ACGG

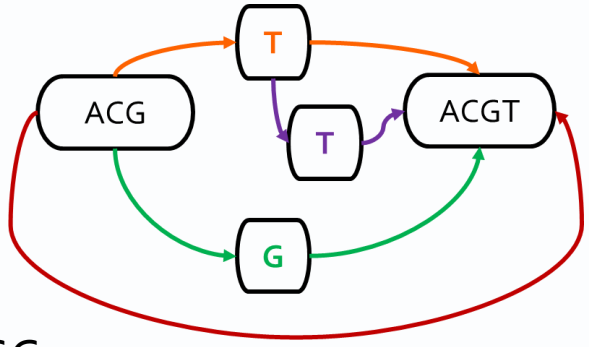
Alternative Sequence: ACGGACGT

Alternative Sequence: ACGTTACGT

Alternative Sequence: ACG–ACGT

Sequence-to-Sequence (S2S) Mapping

Graph-based Reference:



Read: ACGG

Sequence-to-Graph (S2G) Mapping

Sequence-to-graph mapping results in **notable quality improvements.**

However, it is a **more difficult** computational problem,

with **no prior hardware design.**

SeGraM: Use Cases and Key Results

(1) Sequence-to-Graph (S2G) Mapping

- **5.9×/106×** speedup, **4.1×/3.0×** less power than **GraphAligner** for long and short reads, respectively (state-of-the-art **SW**)
- **3.9×/742×** speedup, **4.4×/3.2×** less power than **vg** for long and short reads, respectively (state-of-the-art **SW**)

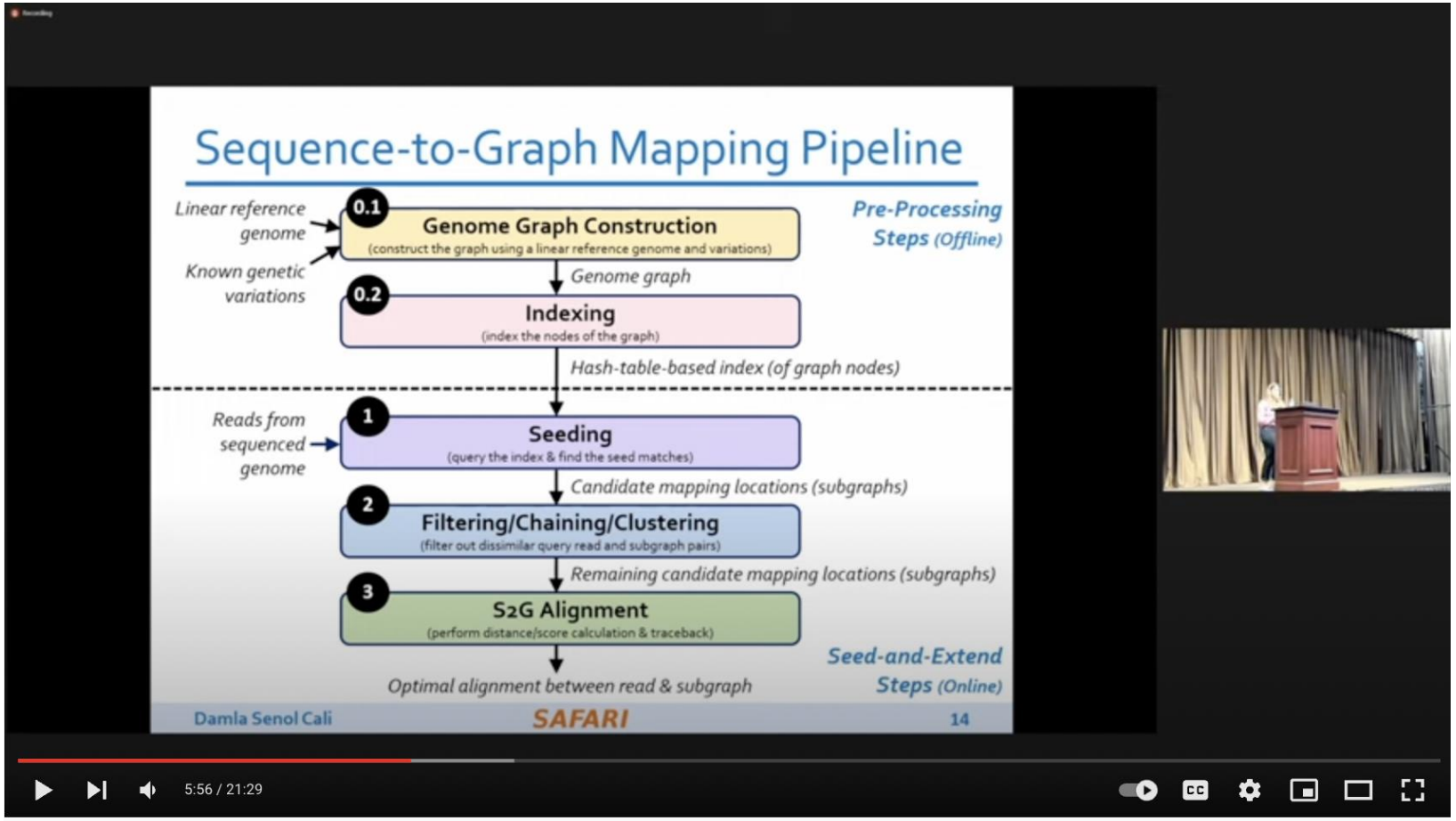
(2) Sequence-to-Graph (S2G) Alignment

- **41×–539×** speedup over **PaSGAL** with AVX-512 support (state-of-the-art sw)

(3) Sequence-to-Sequence (S2S) Alignment

- **1.2×/4.8×** higher throughput than **GenASM** and **GACT of Darwin** for long reads (state-of-the-art **HW**)
- **1.3×/2.4×** higher throughput than **GenASM** and **SillaX of GenAX** for short reads (state-of-the-art **HW**)

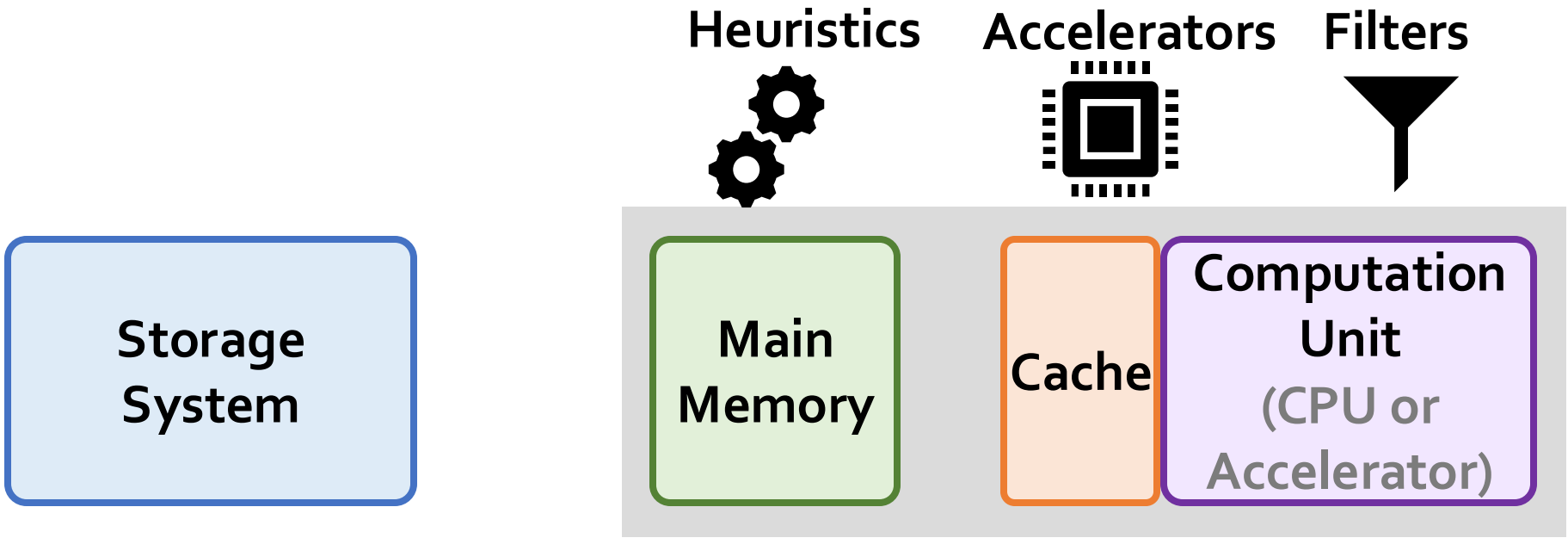
SeGraM Talk Video



https://www.youtube.com/watch?v=gyjqYoyDP9s

Diving Deeper with Examples of Storage-Centric System Design for (Meta)Genomics

Accelerating Genome Sequence Analysis



✓ Computation overhead

✗ Data movement overhead

Storage-Centric Systems for (Meta)Genomics

Genomics

GenStore *(ASPLOS'22)*

Metagenomics

MegIS *(ISCA'24)*

Emerging Areas in (Meta)Genomics

GRAINS
Genome Graph Analysis

MARS *(ICS'25)*
Raw Genomics Signal Analysis

Efficient Analysis on Compressed Genomic Data

SAGe *(to appear in HPCA'26)*

Storage-Centric Systems for (Meta)Genomics

Genomics

GenStore *(ASPLOS'22)*

Metagenomics

MegIS *(ISCA'24)*

Emerging Areas in (Meta)Genomics

GRAINS
Genome Graph Analysis

MARS *(ICS'25)*
Raw Genomics Signal Analysis

Efficient Analysis on Compressed Genomic Data

SAGe *(to appear in HPCA'26)*

Outline

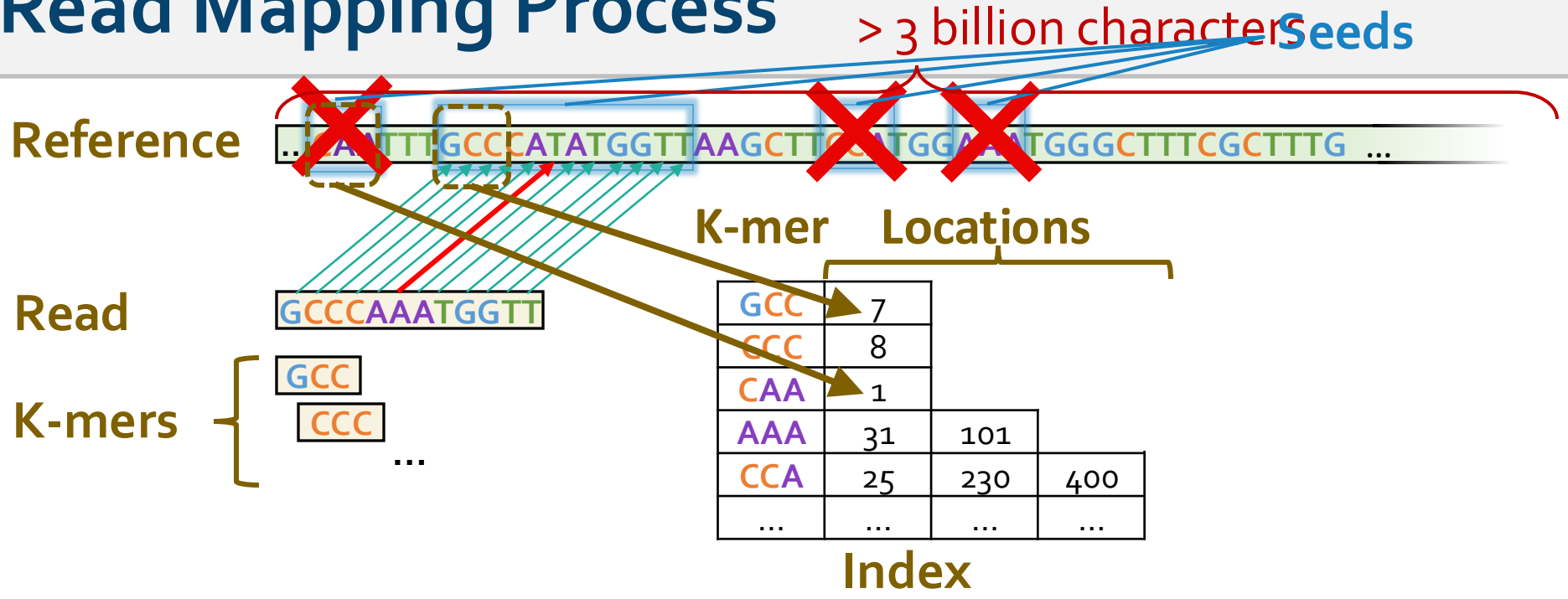
Background

Motivation and Goal

GenStore

Evaluation

Read Mapping Process



Seeding	Determine potential matching locations (seeds) in the reference genome
Seed Filtering (e.g., Chaining)	Prune some seeds in the reference genome
Alignment	Determine the exact differences between the read and the reference genome

Outline

Background

Motivation and Goal

GenStore

Evaluation

Motivation

- Case study on a real-world genomic read dataset
  - Various read mapping systems
  - Various state-of-the-art SSD configurations

The ideal in-storage filter significantly improves performance by

1) reducing the computation overhead

2) reducing the data movement overhead

Motivation

- Case study on a real-world genomic read dataset
  - Various read mapping systems
  - Various state-of-the-art SSD configurations

Filtering outside SSD provides lower performance benefit since it

1) does not reduce the data movement overhead

2) must compete with read mapping for system resources

A HW accelerator reduces the computation bottleneck,

which makes I/O a larger bottleneck in the system

Our Goal

Design an in-storage filter for genome sequence analysis in a cost-effective manner

Design Objectives:

Performance

Provide high in-storage filtering performance to overlap the filtering with the read mapping of unfiltered data

Applicability

Support reads with 1) different properties and 2) different degrees of genetic variation in the compared genomes

Low-cost

Do not require significant hardware overhead

Outline

Background

Motivation and Goal

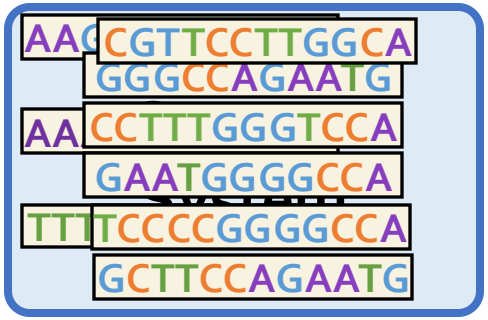
GenStore

Evaluation

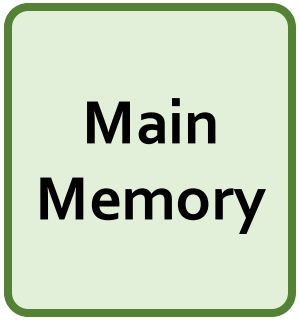
GenStore

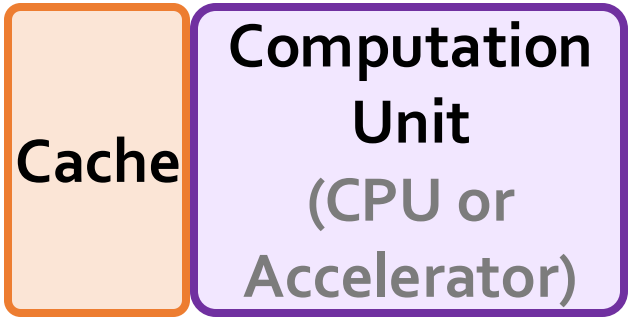


Filter* reads that do *not* require alignment *inside the storage system



Filtered Reads





Challenges

- Different behavior across read mapping workloads
- Limited hardware resources in the SSD

Filtering Opportunities

- Sequencing machines produce one of two kinds of reads
  - **Short reads:** highly accurate and short
  - **Long reads:** less accurate and long

Reads that do not require the expensive alignment step:

Exactly-matching reads

Do not need expensive approximate string matching during alignment

- Low sequencing error rates (short reads) combined with
- Low genetic variation

Non-matching reads

Do not have potential matching locations, so they skip alignment

- High sequencing error rates (long reads) or
- High genetic variation (short or long reads)

GenStore

GenStore-EM for Exactly-Matching Reads

GenStore-NM for Non-Matching Reads

GenStore

GenStore-EM for Exactly-Matching Reads

GenStore-NM for Non-Matching Reads

GenStore-EM

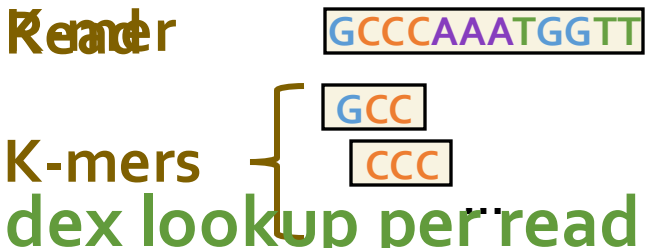
- Efficient in-storage filter for reads with at least one exact match in the reference genome
- Uses simple operations, without requiring alignment
- **Challenge:** large number of random accesses per read to the reference genome and its index

Expensive random accesses to flash chips

Limited DRAM capacity inside the SSD

GenStore-EM: Data Structures

- **Read-sized k-mers:** to reduce the number of accesses per each read



✓ Only one index lookup per read

- **Sorted read-sized k-mers:** to avoid random accesses to the index

✓ Sequential scan of the read set and the index

GenStore-EM: Data Structures

Sorted Read Table

	Read
	AAAAAAAAAA
	AAAAAAAAAG
	AAAAAAAACT
	…

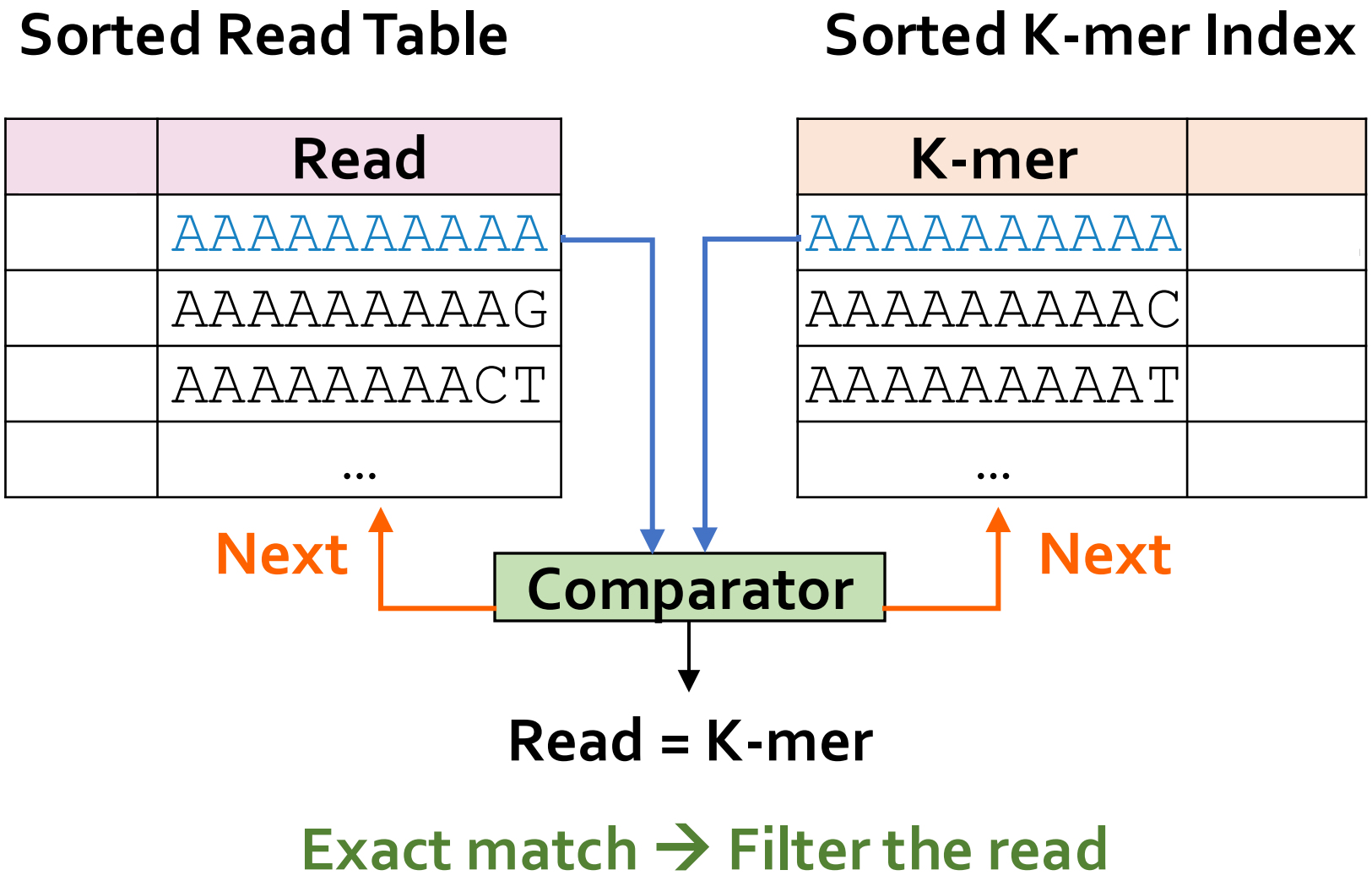
Sorted

Sorted K-mer Index

K-mer	
AAAAAAAAAA	
AAAAAAAAAC	
AAAAAAAAAT	
…	

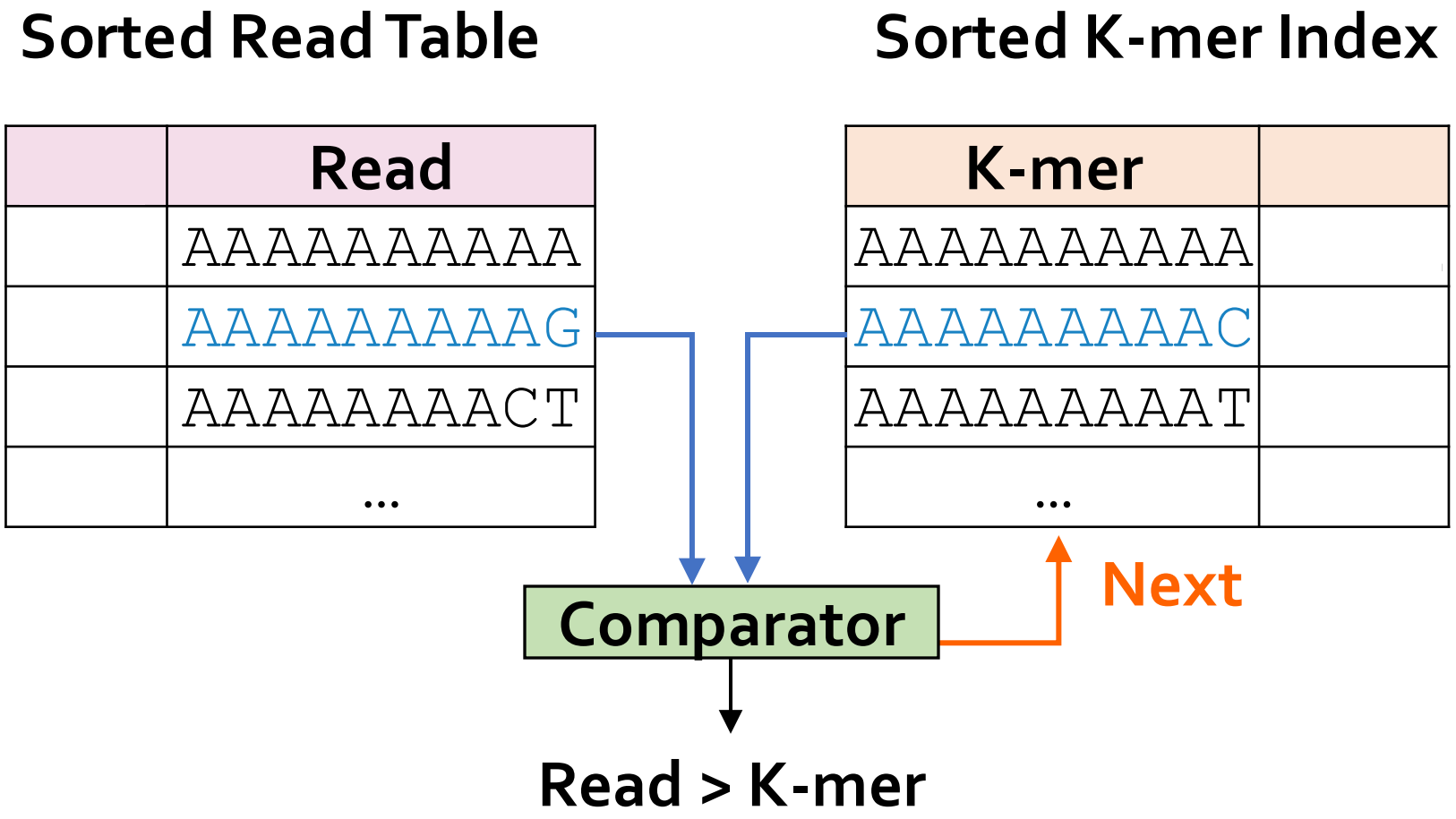
Read-sized K-mers

GenStore-EM: Finding a Match

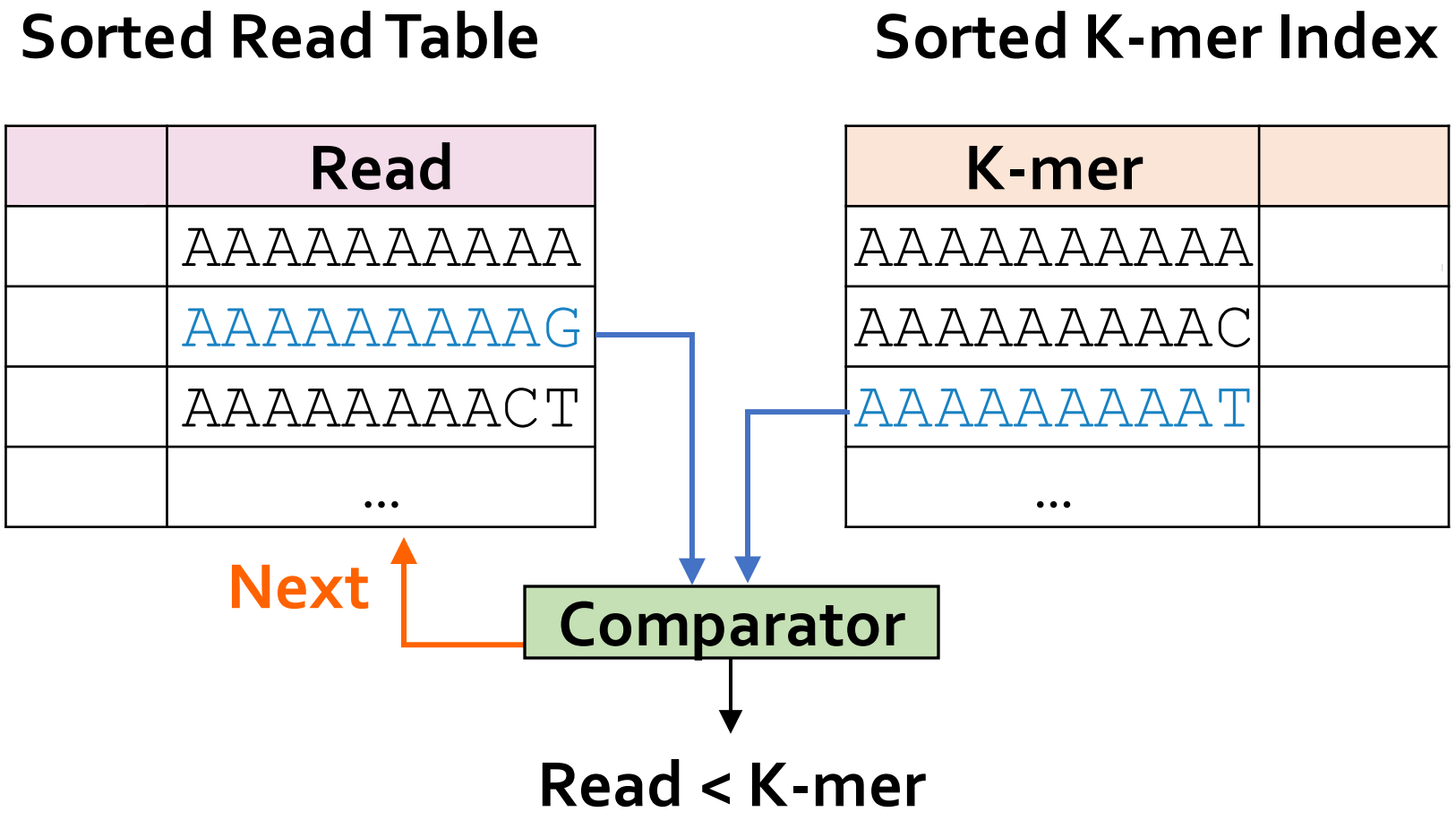


Exact match → Filter the read

GenStore-EM: Not Finding a Match

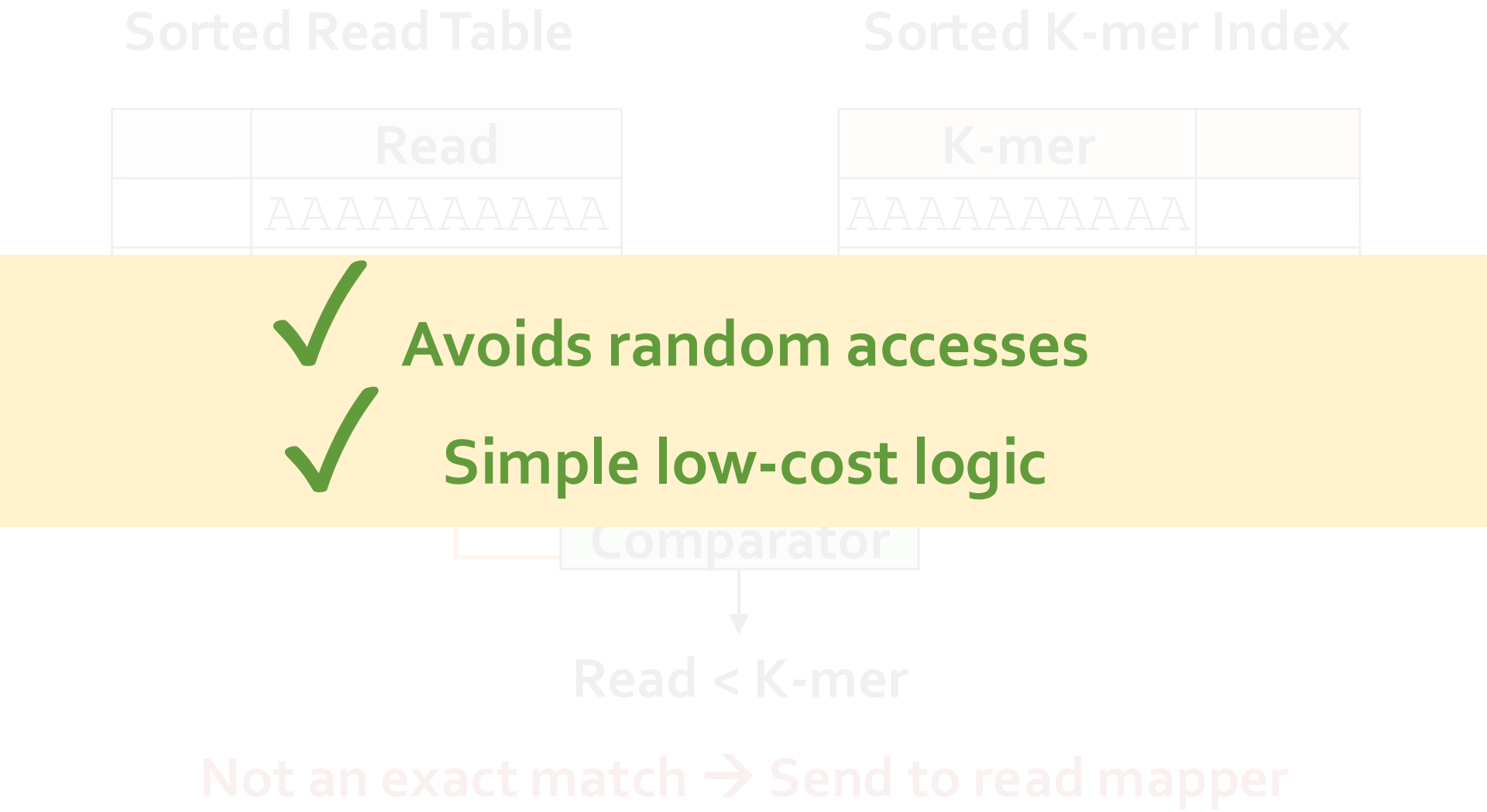


GenStore-EM: Not Finding a Match



Not an exact match → Send to read mapper

GenStore-EM: Not Finding a Match



✓ Avoids random accesses

✓ Simple low-cost logic

GenStore-EM: Optimization

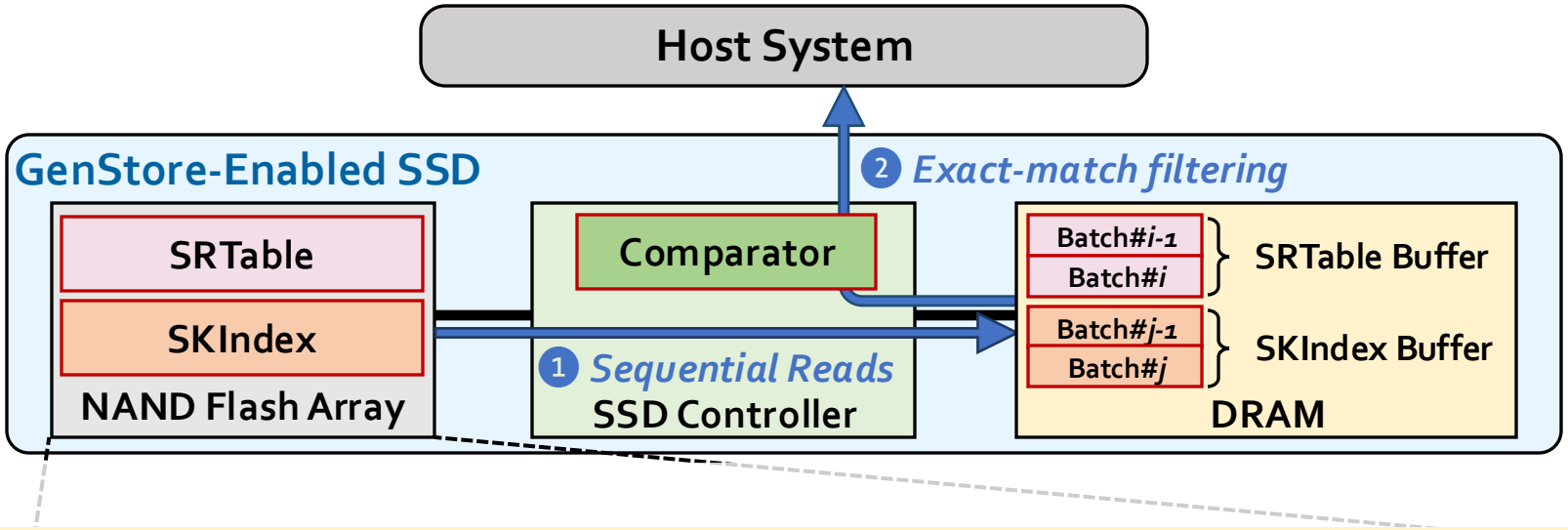
- Read-sized k-mer index takes up a large amount of space (126 GB for human index) due to the larger number of unique k-mers

Sorted K-mer Index

Strong Hash Value	Loc.
1	1, 8, …
4	51
7	23, 37
16	…

Using strong hash values instead of read-sized k-mers reduces the size of the index by 3.9x

GenStore-EM: Design



Steps 1 and 2 are pipelined.
During filtering, GenStore-EM sends the unfiltered reads to the host system.

Data is evenly distributed between channels, dies, and planes to leverage the full internal bandwidth of the SSD

GenStore

GenStore-EM for Exactly-Matching Reads

GenStore-NM for Non-Matching Reads

GenStore-NM

- Efficient chaining-based in-storage filter to prune most of the non-matching reads

Seeding	Determine potential matching locations (seeds) in the reference genome
Seed Filtering (e.g., Chaining)	Prune some seeds in the reference genome
Alignment	Determine the exact differences between the read and the reference genome

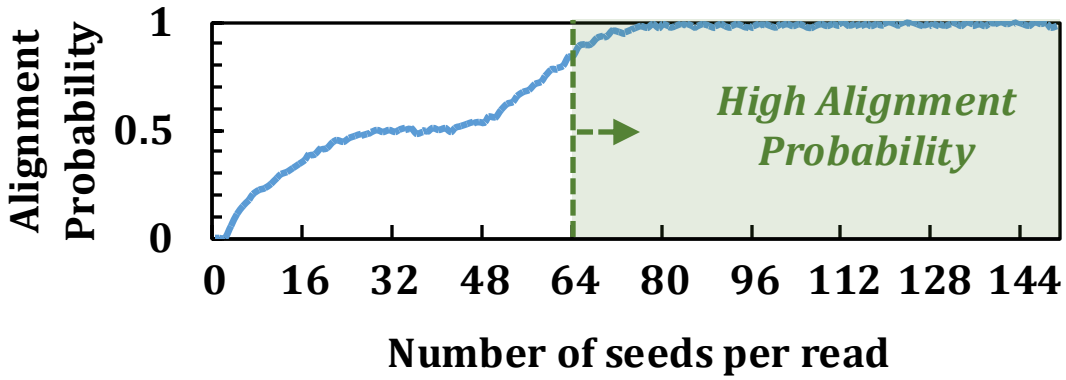
- **Challenge:** how to perform chaining inside the SSD

Costly dynamic programming on many seeds in each read

Particularly challenging for long reads with many seeds

GenStore-NM: Mechanism

- GenStore-NM uses a light-weight chaining filter
  - Selectively performs chaining only on reads with a small number of seeds
  - Directly sends reads that require more complex chaining to the host system

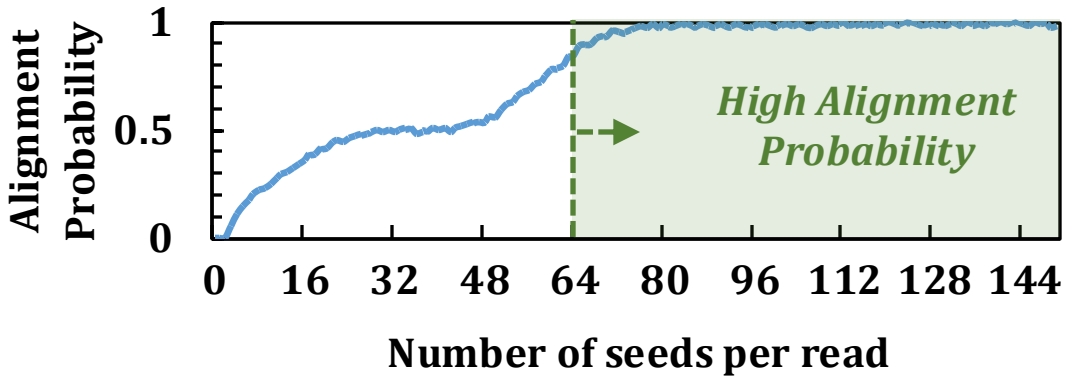


Reads with a sufficiently large number of seeds are very likely to align to the reference genome

✓ **Filters many non-aligning reads without costly hardware resources in the SSD**

GenStore-NM: Mechanism

- GenStore-NM uses a light-weight chaining filter
  - Selectively performs chaining only on reads with a small number of seeds
  - Directly sends reads that require more complex chaining to the host system



Reads with a sufficiently large number of seeds are very likely to align to the reference genome

Details on GenStore-NM's design are in the paper

Outline

Background

Motivation and Goal

GenStore

Evaluation

Evaluation Methodology

Read Mappers

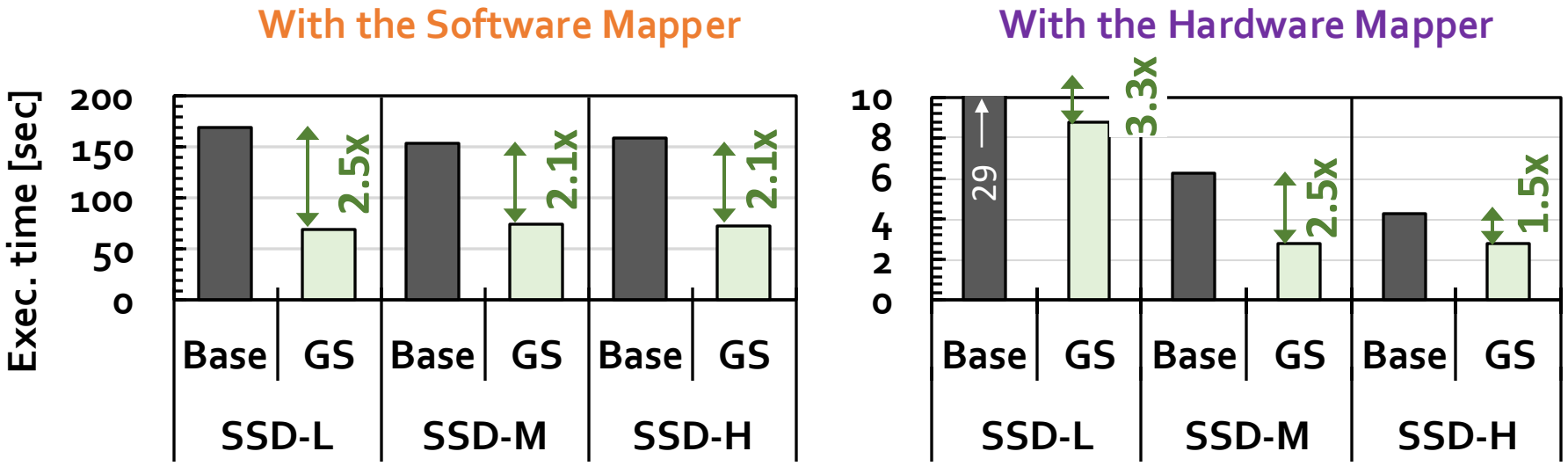
- **Base:** state-of-the-art software or hardware read mappers
  - Minimap2 [Bioinformatics'18]: software mapper for short and long reads
  - GenCache [MICRO'19]: hardware mapper for short reads
  - Darwin [ASPLOS'18]: hardware mapper for long reads
- **GS:** Base integrated with GenStore

SSD Configurations

- **SSD-L:** with SATA3 interface (0.5 GB/s sequential read bandwidth)
- **SSD-M:** with PCIe Gen3 interface (3.5 GB/s sequential read bandwidth)
- **SSD-H:** with PCIe Gen4 interface (7 GB/s sequential read bandwidth)

Performance – GenStore-EM

For a read set with 80% exactly-matching reads



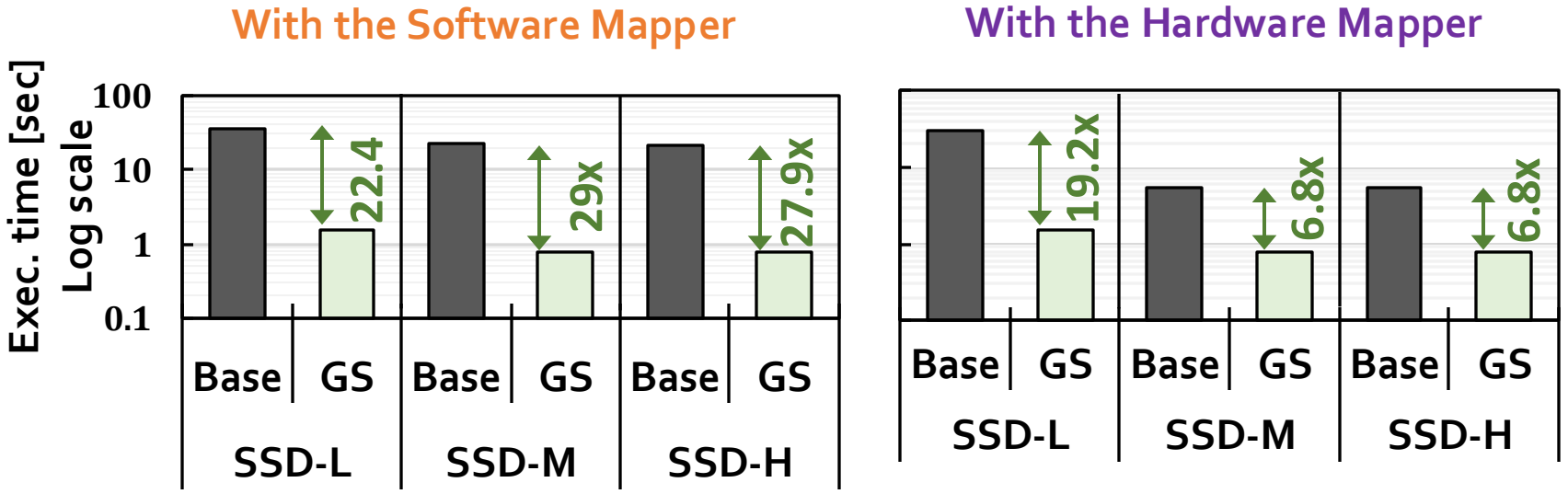
2.1× - 2.5× speedup compared to the software Base

1.5× – 3.3× speedup compared to the hardware Base

On average 3.92× energy reduction

Performance – GenStore-NM

For a read set with 99.7% non-matching reads



22.4× – 27.9× speedup compared to the software Base

6.8× – 19.2× speedup compared to the hardware Base

On average 27.2× energy reduction

Area and Power

- Based on **Synthesis** of **GenStore** accelerators using the Synopsys Design Compiler @ 65nm technology node

Logic unit	# of instances	Area [mm²]	Power [mW]
Comparator	1 per SSD	0.0007	0.14
K -mer Window	2 per channel	0.0018	0.27
Hash Accelerator	2 per SSD	0.008	1.8
Location Buffer	1 per channel	0.00725	0.37375
Chaining Buffer	1 per channel	0.008	0.95
Chaining PE	1 per channel	0.004	0.98
Control	1 per SSD	0.0002	0.11
Total for an 8-channel SSD	-	0.2	26.6

Only 0.006% of a 14nm Intel Processor, less than 9.5% of the three ARM processors in a SATA SSD controller

Other Results in the Paper

- Effect of read set features on performance
  - Data size (up to 440 GB)
  - Filter ratio
- Performance benefit of an implementation of GenStore outside the SSD
  - In some cases, it provides performance benefits due more efficient streaming accesses
  - Provides significantly lower benefit compared to GenStore
- More detailed characterization of non-matching reads across different read mapping use cases and species

More in the Paper

GenStore: A High-Performance and Energy-Efficient In-Storage Computing System for Genome Sequence Analysis

Nika Mansouri Ghiasi[1] Jisung Park[1] Harun Mustafa[1] Jeremie Kim[1] Ataberk Olgun[1]
Arvid Gollwitzer[1] Damla Senol Cali[2] Can Firtina[1] Haiyu Mao[1] Nour Almadhoun Alserr[1]
Rachata Ausavarungnirun[3] Nandita Vijaykumar[4] Mohammed Alser[1] Onur Mutlu[1]

[1]ETH Zürich [2]Bionano Genomics [3]KMUTNB [4]University of Toronto

https://arxiv.org/abs/2202.10400

https://github.com/CMU-SAFARI/GenStore

Storage-Centric Systems for (Meta)Genomics

Genomics

GenStore *(ASPLOS'22)*

Metagenomics

MegIS *(ISCA'24)*

Emerging Areas in (Meta)Genomics

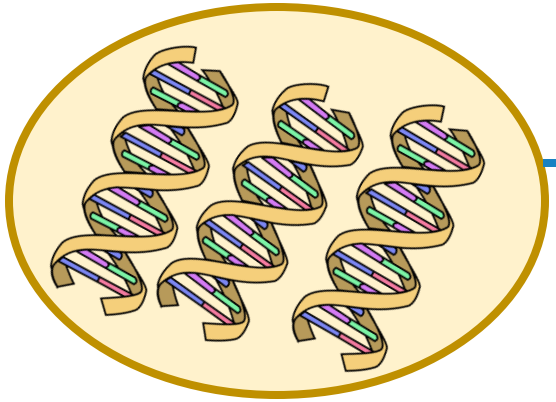
GRAINS
Genome Graph Analysis

MARS *(ICS'25)*
Raw Genomics Signal Analysis

Efficient Analysis on Compressed Genomic Data

SAGe *(to appear in HPCA'26)*

Metagenomic Analysis



Metagenomic sample with species that are **not known** in advance



A large database containing information on **many species**

Preparation of Input Queries

Query K-mers: GCTCA, CTCAT, TCATG, ...

Presence/Absence Identification



V. cholerae

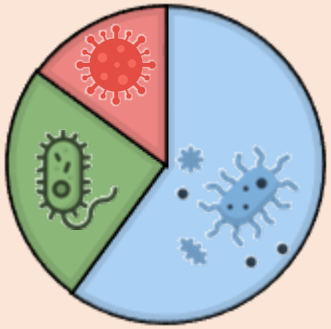


SARS-CoV-2



E. coli

Abundance Estimation



(e.g., > 100 TBs in emerging databases)

Outline

Background

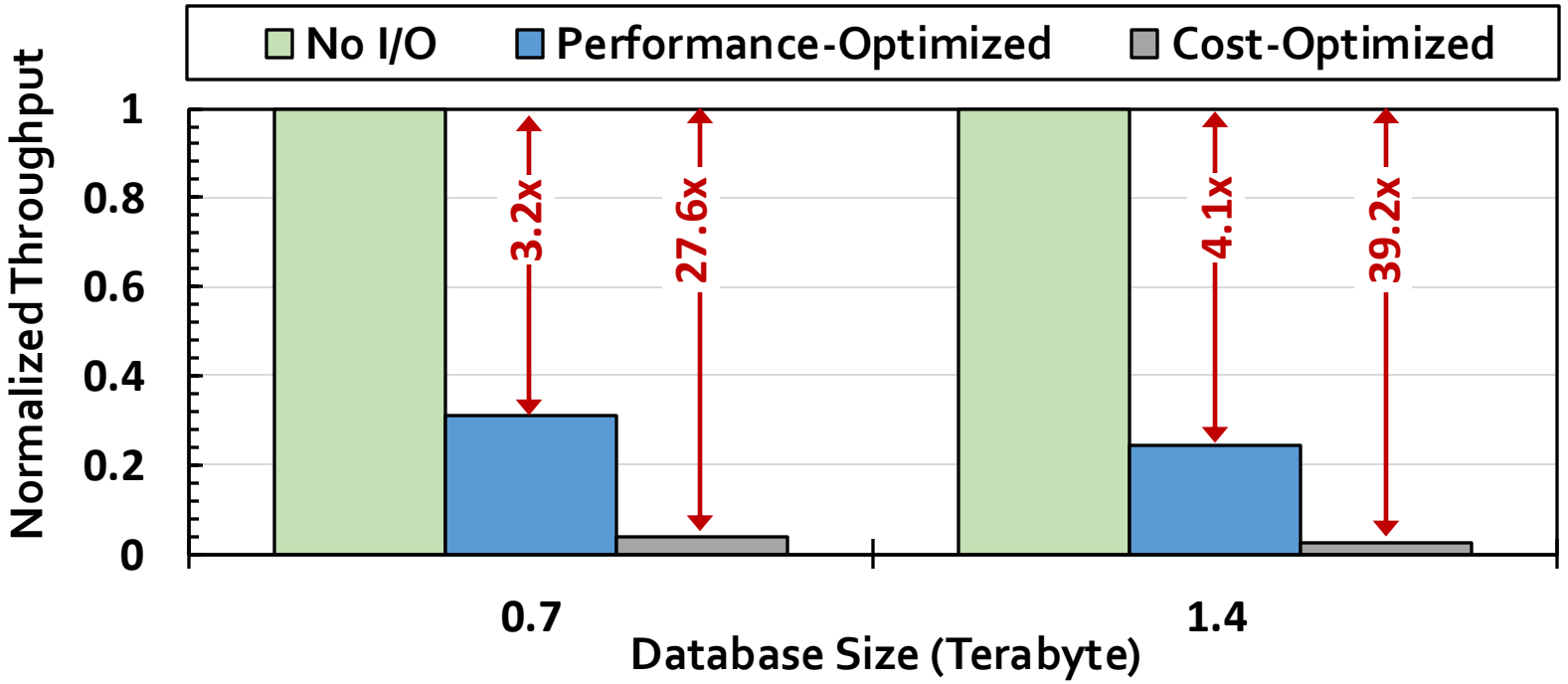
Motivation and Goal

MegIS

Evaluation

Motivation

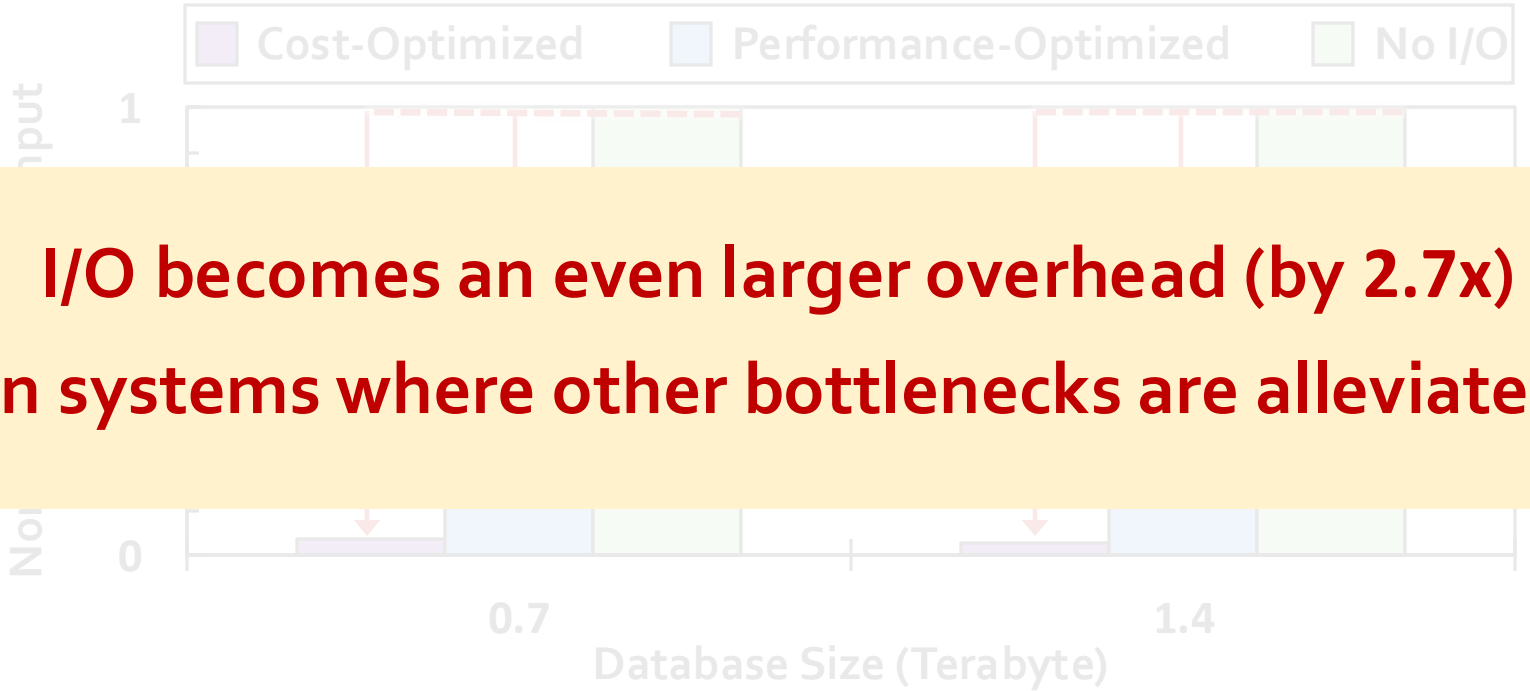
- Case study of the performance of metagenomic analysis tools
- With various state-of-the-art SSD configurations



I/O data movement causes significant performance overhead

Motivation

- Case study on the throughput of metagenomic analysis tools
- With Various state-of-the-art SSD configurations



I/O becomes an even larger overhead (by 2.7x) in systems where other bottlenecks are alleviated

I/O data movement causes significant performance overhead

I/O Overhead is Hard to Avoid

I/O overhead due to accessing **large**, **low-reuse** data is hard to avoid

Sampling techniques to shrink database sizes

[Wood+, Genome Biology'19], [Ounit+, BMC Genomics'15], [Kim+, Genome Research'16], …

✗ *Reduce accuracy to levels unacceptable for many use cases*

Keeping all data required by metagenomic analysis completely and always resident in main memory

✗ *Energy inefficient, costly, unscalable, and unsustainable*

- Database sizes **increase rapidly** (doubling every few months)
- Different analyses need **different databases**

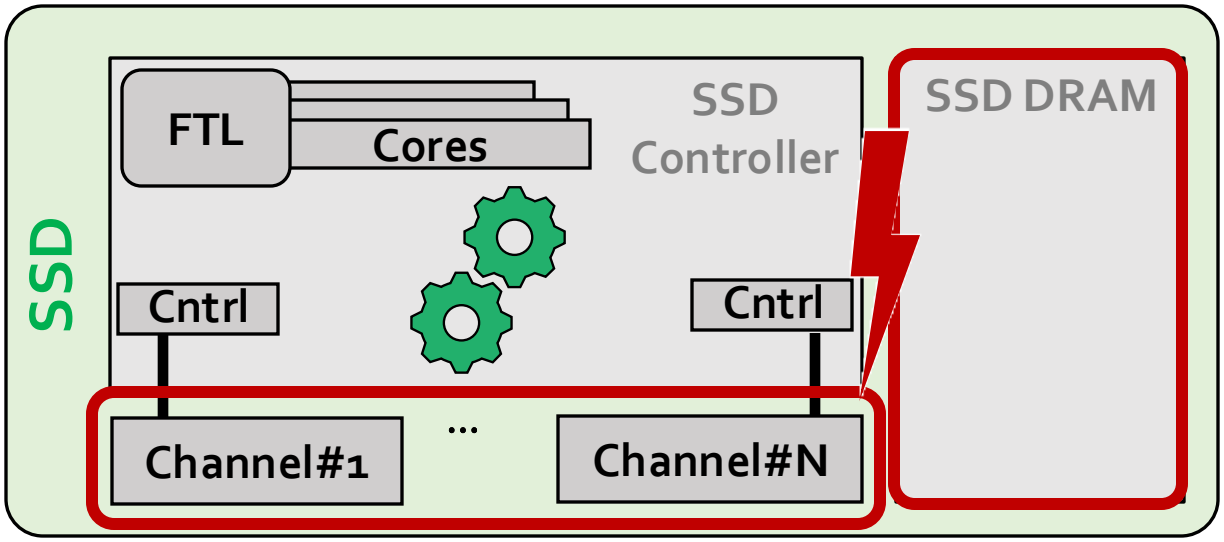
Our Goal

Improve metagenomic analysis ***performance***
by reducing large ***data movement overhead***
from the storage system
in a ***cost-effective*** *manner and with* ***high accuracy***

Challenges of In-Storage Processing

No metagenomic analysis tools can run in-storage due to SSD limits

- Long **latency of NAND flash** chips
- Limited **DRAM capacity** inside the SSD
- Limited **DRAM bandwidth** inside the SSD



Outline

Background

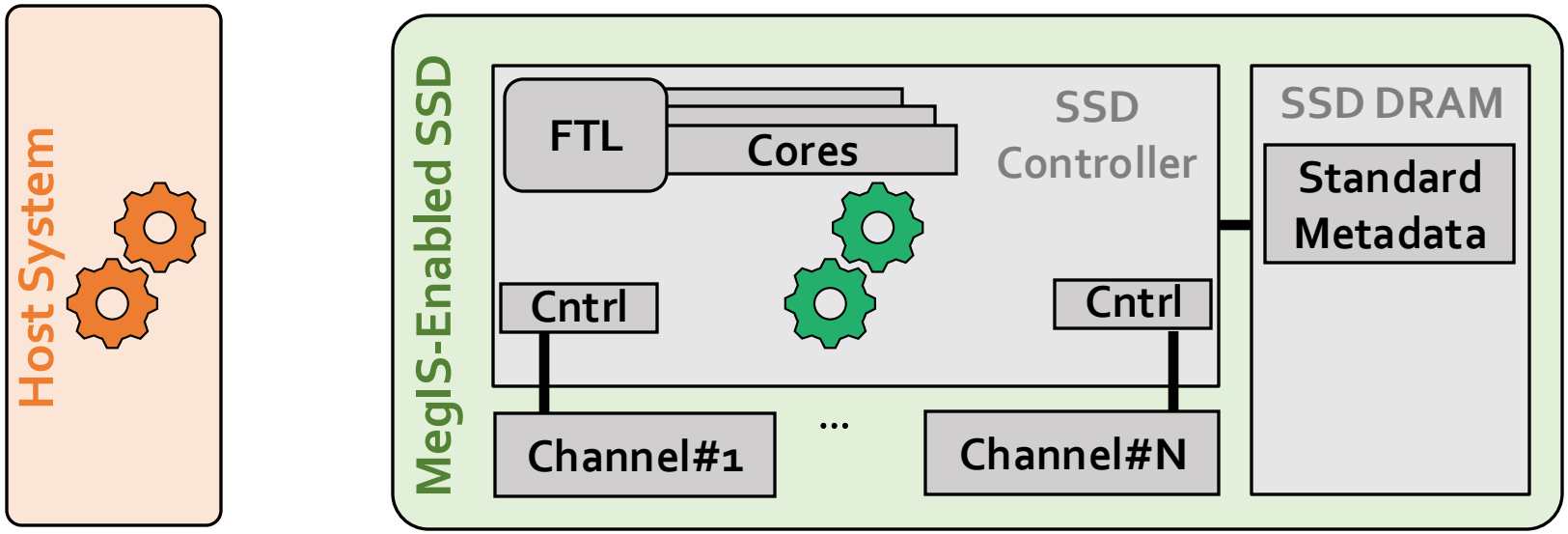
Motivation and Goal

MegIS

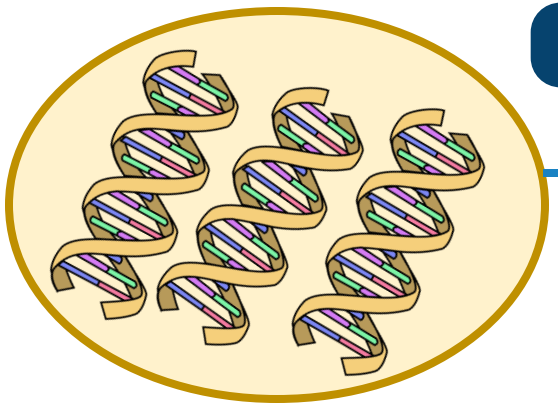
Evaluation

MegIS: Metagenomics In-Storage

- First in-storage system for *end-to-end* metagenomic analysis
- **Idea:** Cooperative in-storage processing for metagenomic analysis
  - Hardware/software co-design between



MegIS's Steps



Metagenomic sample with species that are **not known** in advance



A large database containing information on **many species**

Step 1

Preparation of Input Queries

Query K-mers: GCTCA, CTCAT, TCATG, ...

Step 2

Presence/Absence Identification

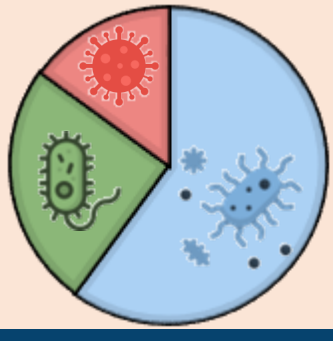
V. cholerae

SARS-CoV-2

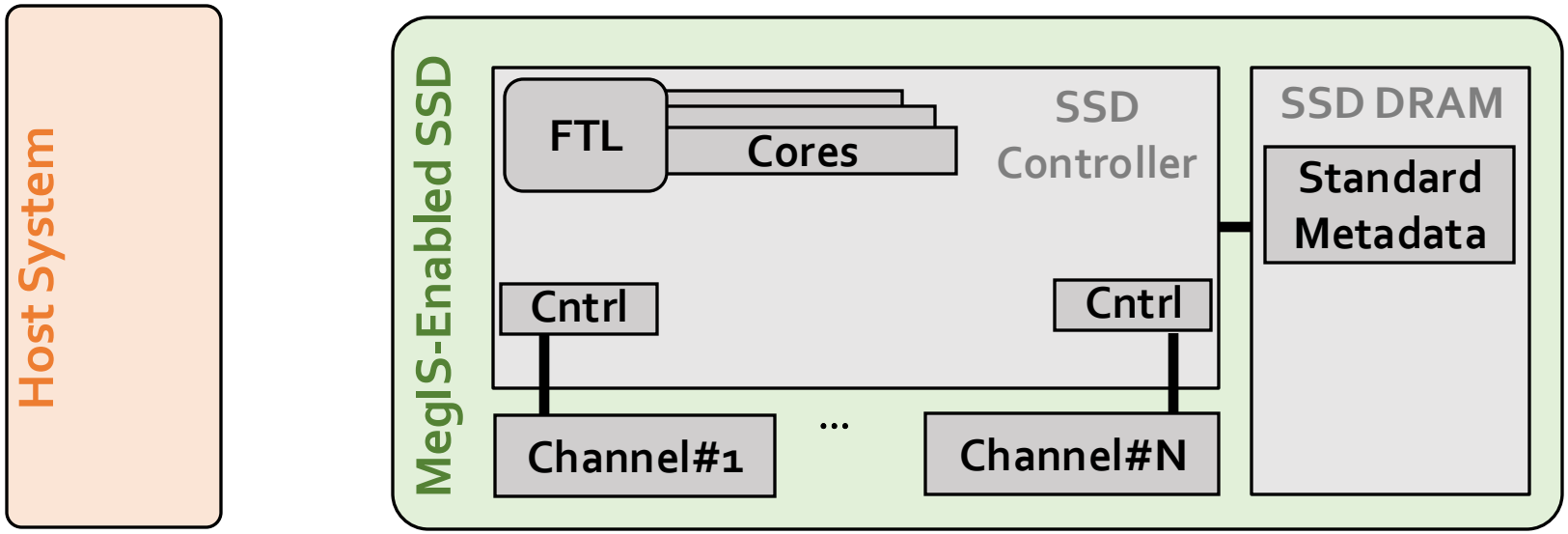
E. coli

Step 3

Abundance Estimation



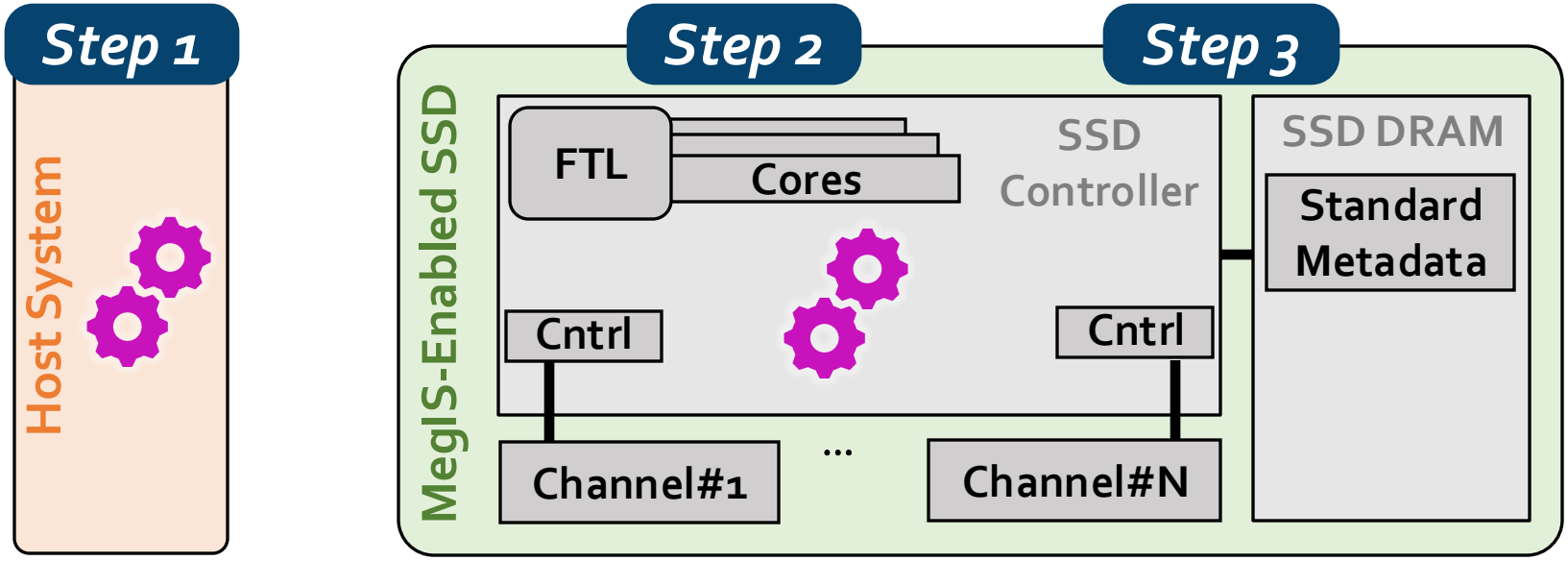
MegIS Hardware-Software Co-Design



MegIS Hardware-Software Co-Design

Task partitioning and mapping

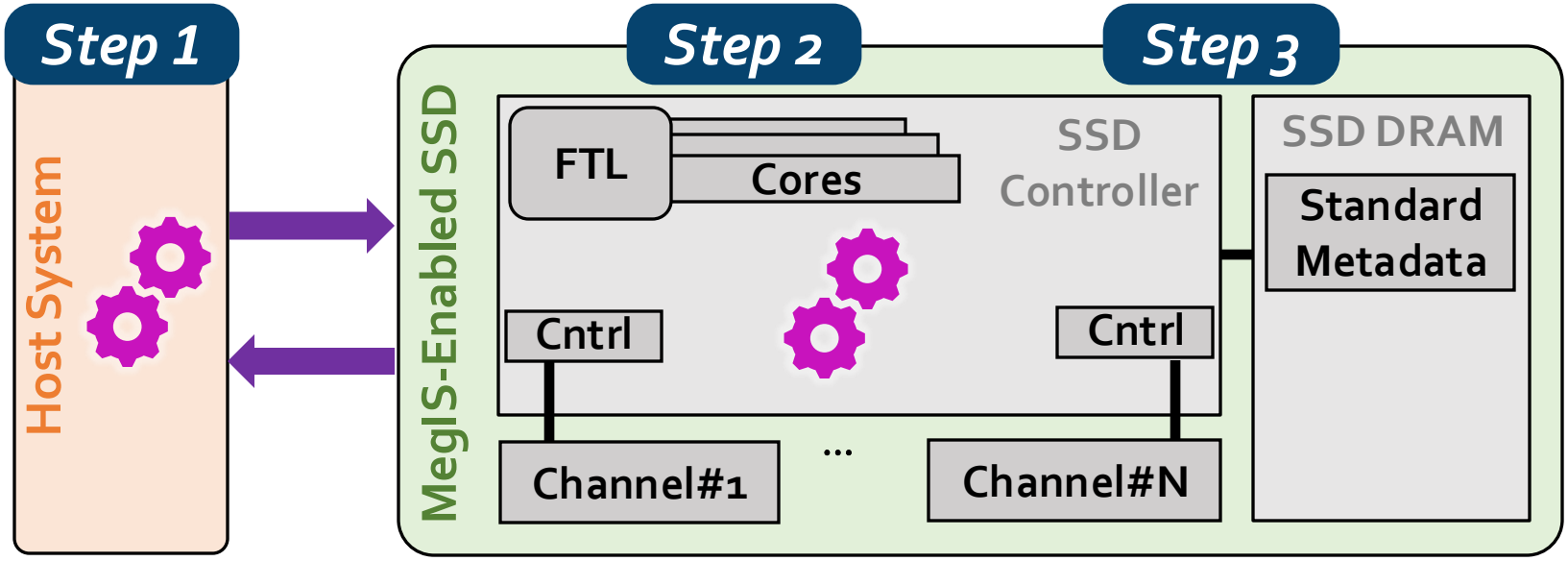
- *Each step executes in its most suitable system*



MegIS Hardware-Software Co-Design

Task partitioning and mapping
- *Each step executes in its most suitable system*

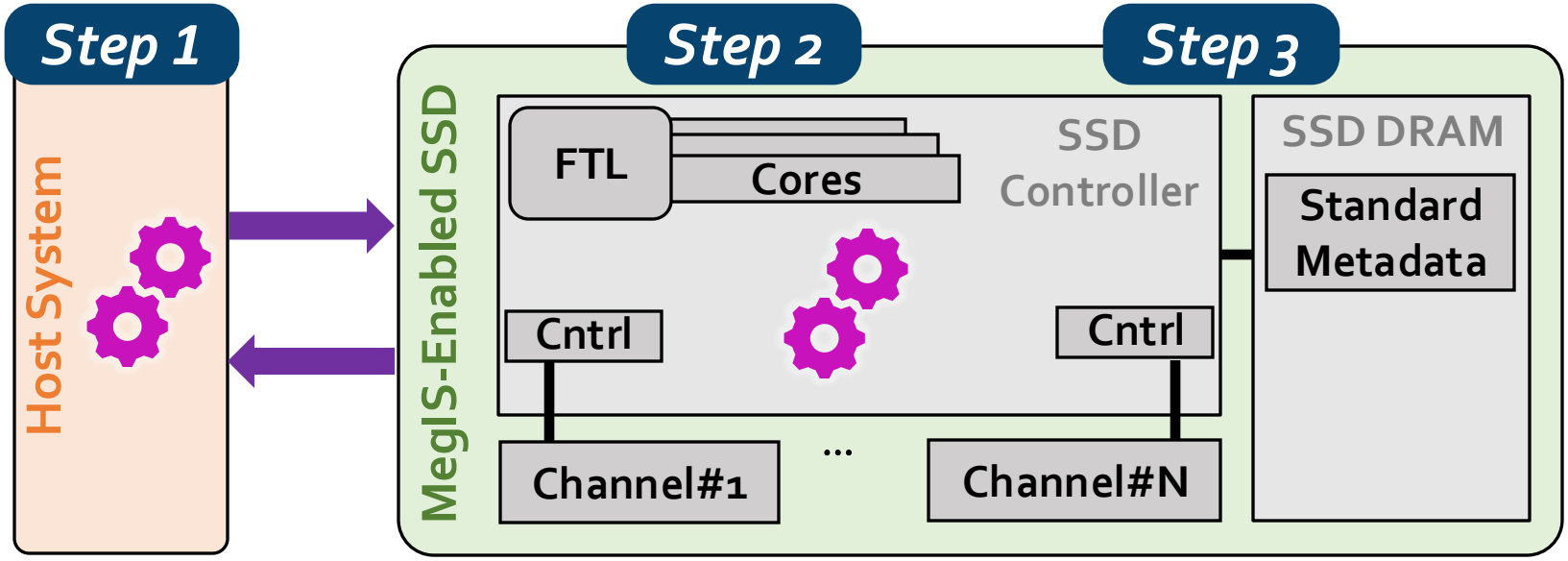
Data/computation flow coordination
- *Reduce communication overhead*
- *Reduce #writes to flash chips*



MegIS Hardware-Software Co-Design

Task partitioning and mapping
- *Each step executes in its most suitable system*

Data/computation flow coordination
- *Reduce communication overhead*
- *Reduce #writes to flash chips*

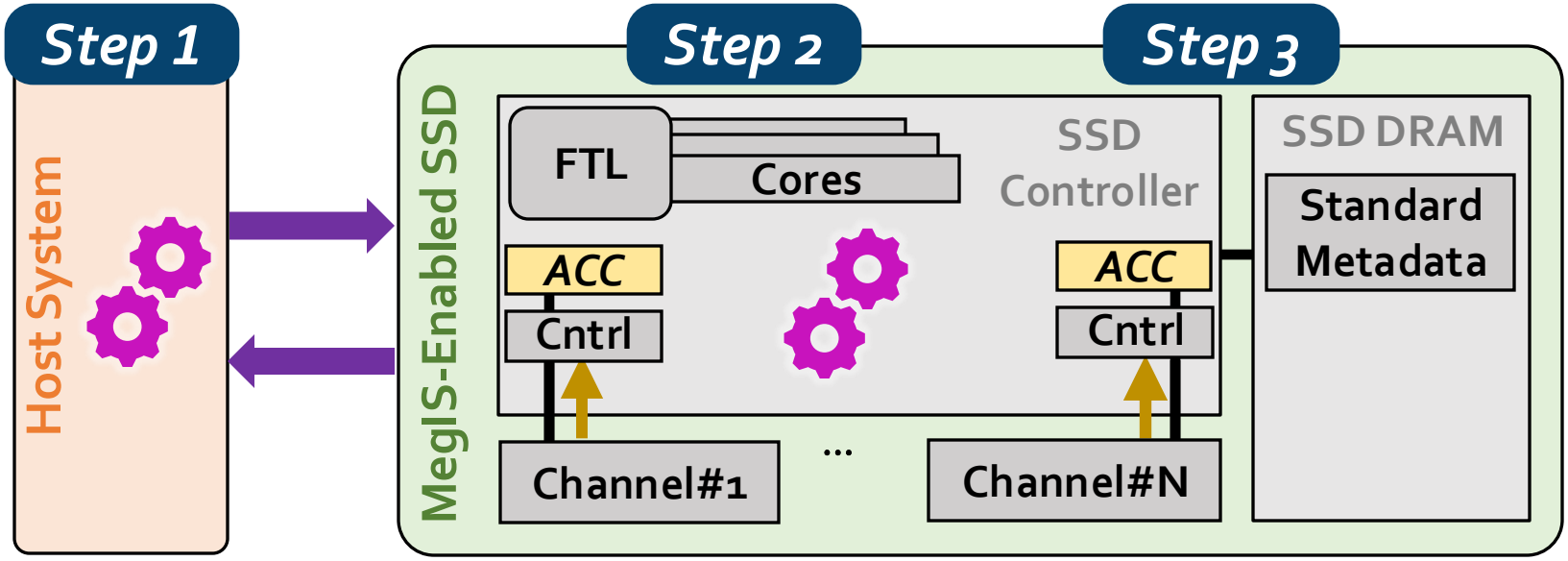


Storage-aware algorithms
- *Enable efficient access patterns to the SSD*

MegIS Hardware-Software Co-Design

Task partitioning and mapping
- *Each step executes in its most suitable system*

Data/computation flow coordination
- *Reduce communication overhead*
- *Reduce #writes to flash chips*



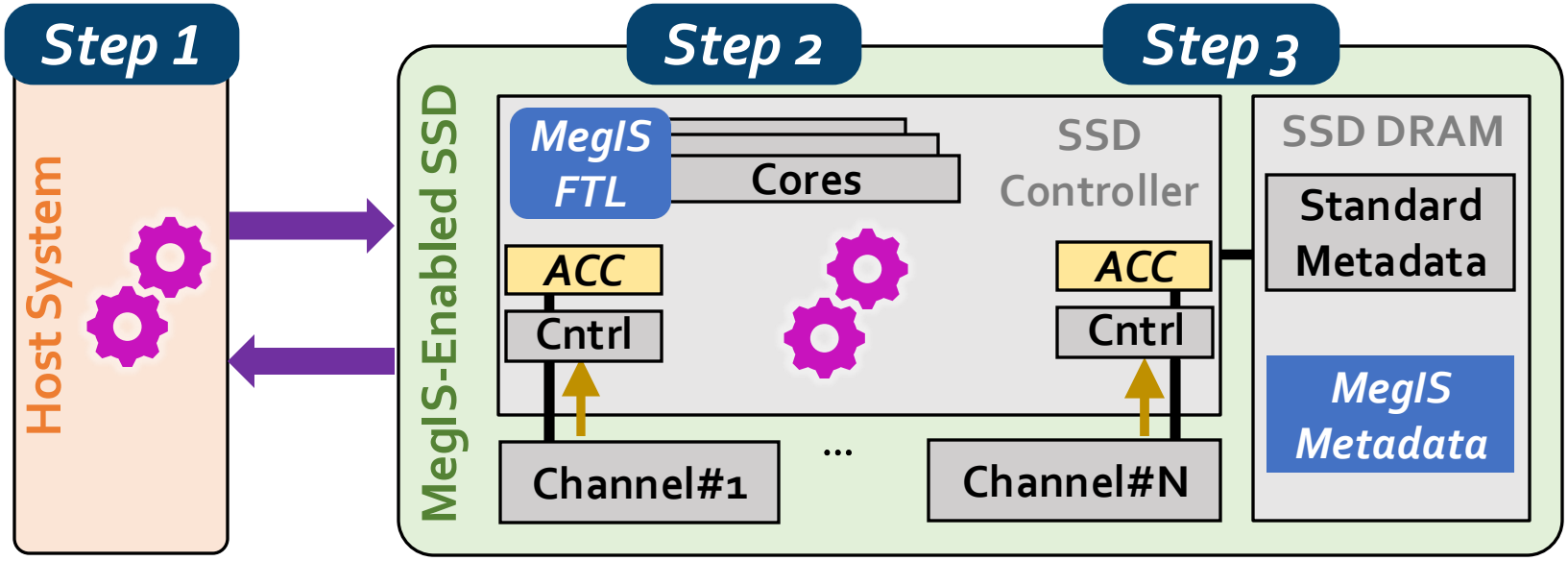
Storage-aware algorithms
- *Enable efficient access patterns to the SSD*

Lightweight in-storage accelerators
- *Minimize SRAM/DRAM buffer spaces needed inside the SSD*

MegIS Hardware-Software Co-Design

Task partitioning and mapping
- *Each step executes in its most suitable system*

Data/computation flow coordination
- *Reduce communication overhead*
- *Reduce #writes to flash chips*

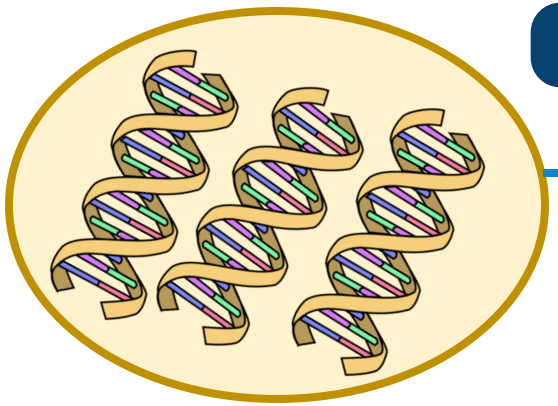


Storage-aware algorithms
- *Enable efficient access patterns to the SSD*

Lightweight in-storage accelerators
- *Minimize SRAM/DRAM buffer spaces needed inside the SSD*

Data mapping scheme and Flash Translation Layer (FTL)
- *Specialize to the characteristics of metagenomic analysis*
- *Leverage the SSD's full internal bandwidth*

Step 1 Overview



Metagenomic sample with species that are **not known** in advance



A large database containing information on **many species**

Step 1

Preparation of Input Queries

Query K-mers: GCTCA, CTCAT, TCATG, ...

Step 2

Presence/Absence Identification

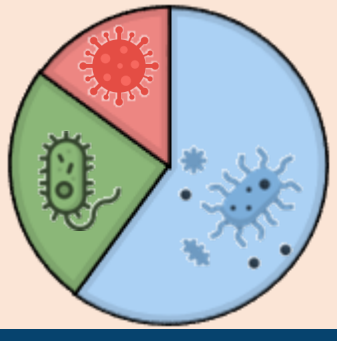
V. cholerae

SARS-CoV-2

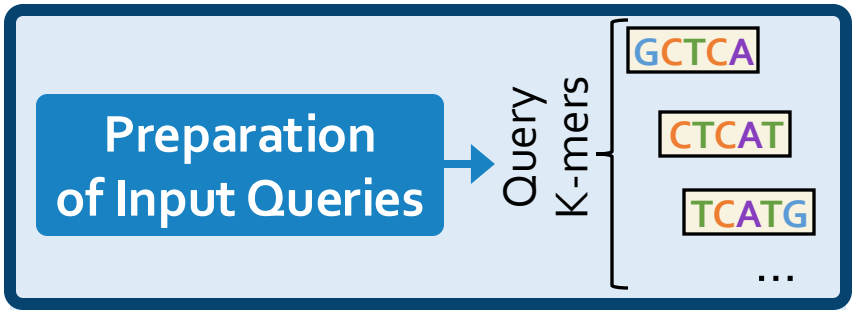
E. coli

Step 3

Abundance Estimation



Step 1 Overview



*MegIS employs **sorted data structures** to avoid expensive random accesses to the SSD*

- **Extract k-mers** from the sample
- **Sort** the k-mers (database is sorted offline)

MegIS executes Step 1 in the host system

- Benefits from **larger DRAM** and **more powerful computation**
- Incurs **fewer writes** to NAND flash chips (than processing this step in the SSD)
- Enables **overlapping** Step 1 with Step 2

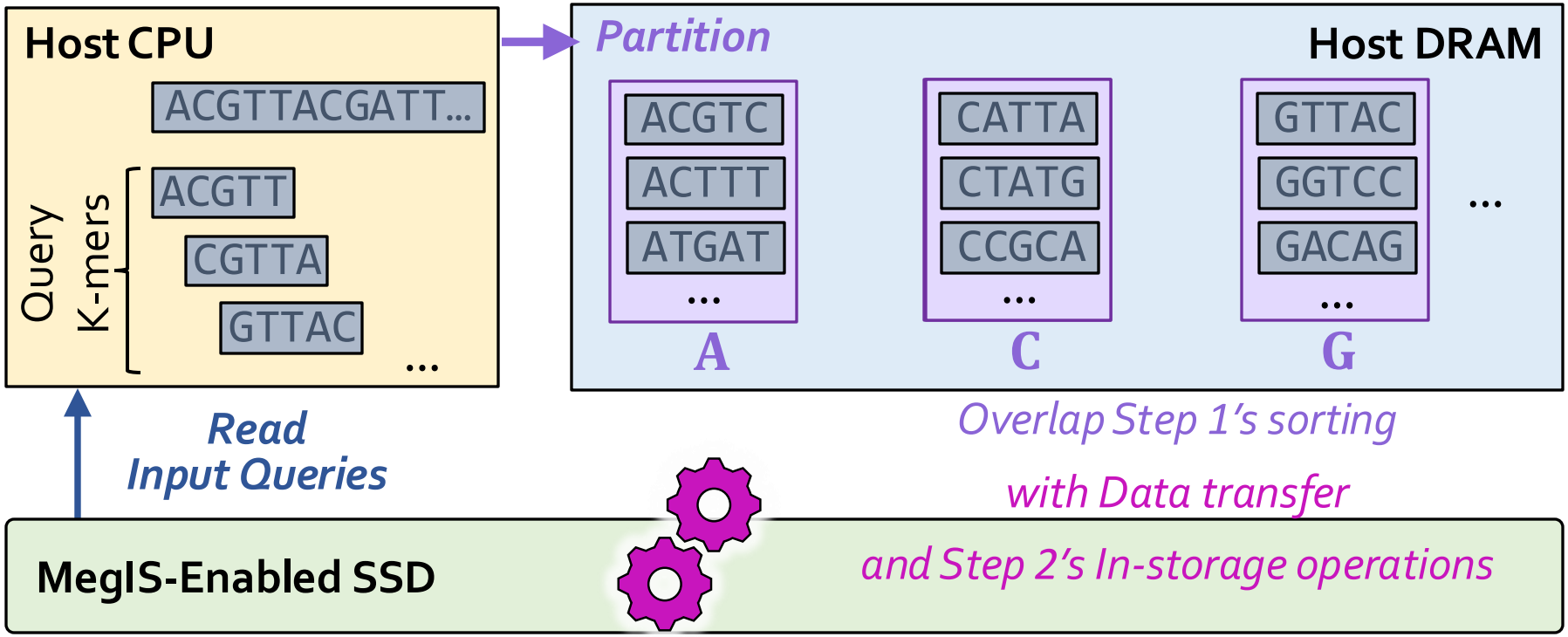
To execute Step 1 efficiently in the host system, MegIS needs to:

- Avoid significant overhead due to **data transfer time** between the steps
- Minimize **performance** and **lifetime** overheads *even* when host DRAM cannot hold all query k-mers

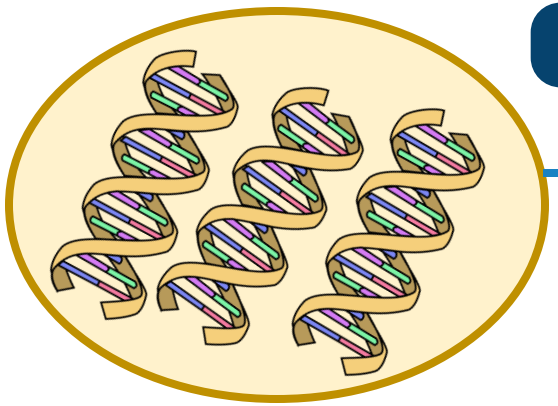
Step 1 Design

Divide k-mers into **independent partitions** by their alphabetical range

✓ **Can overlap operations on different partitions**



Step 2 Overview



Metagenomic sample with species that are **not known** in advance



A large database containing information on **many species**

Step 1

Preparation of Input Queries

Query K-mers: GCTCA, CTCAT, TCATG, ...

Step 2

Presence/Absence Identification

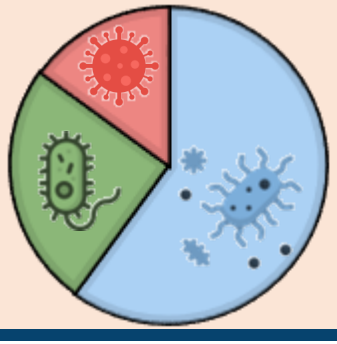
V. cholerae

SARS-CoV-2

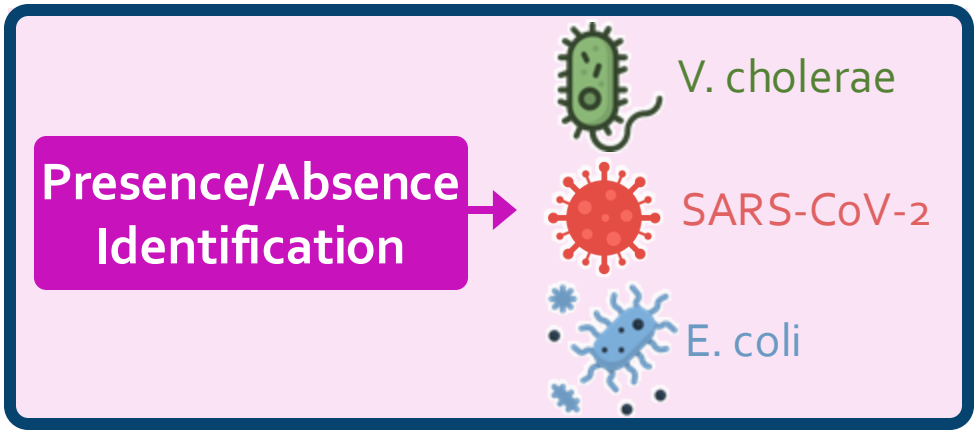
E. coli

Step 3

Abundance Estimation



Step 2 Overview



- **Identify the common k-mers** between the query k-mers and the database k-mers
- **Retrieve the species IDs** of the common k-mers

MegIS executes Step 2 in the SSD

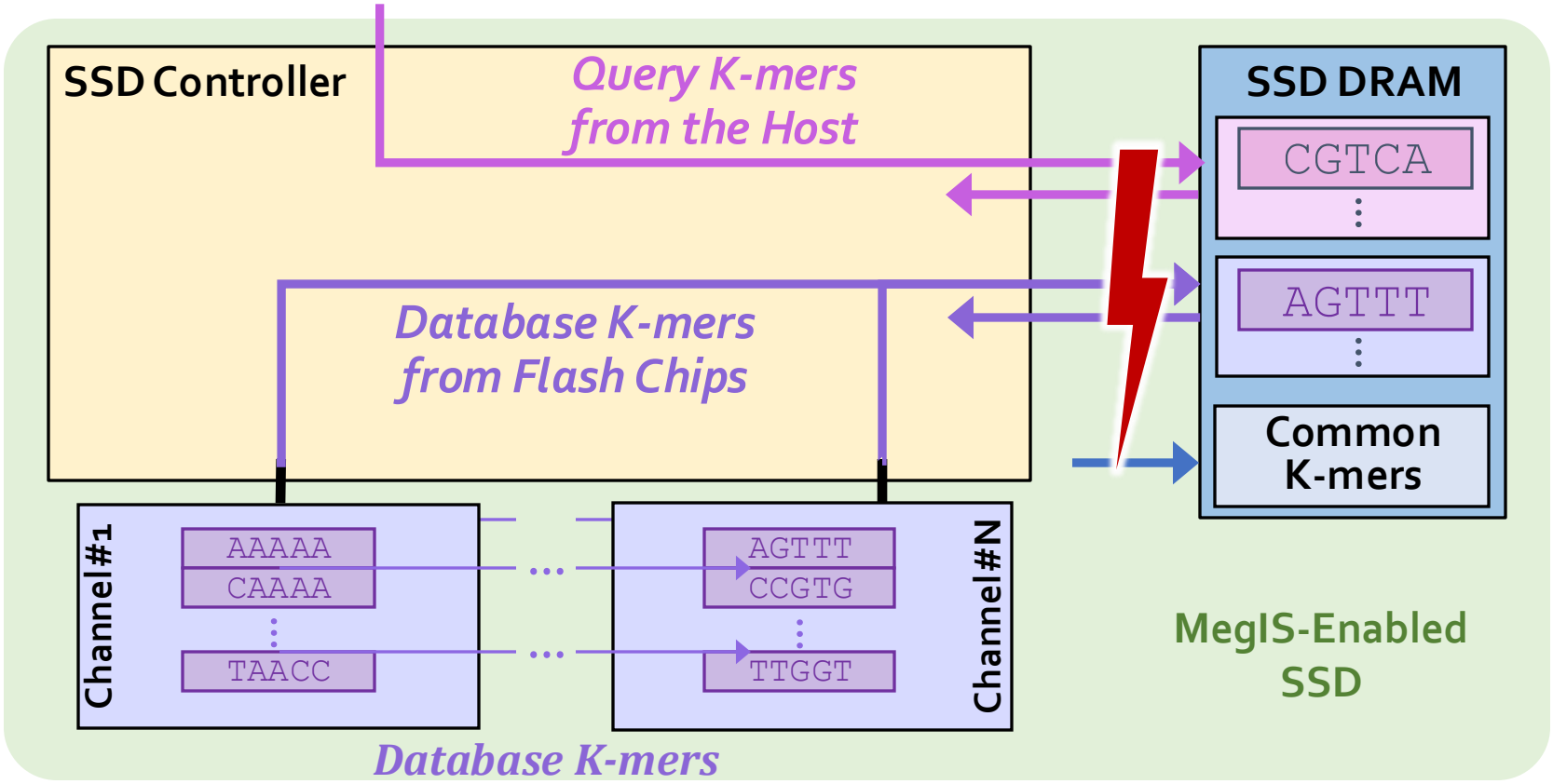
- Accesses **large data** with **low reuse**
- Involves **lightweight computation**

To execute Step 2 efficiently in the SSD, MegIS needs to:

- Leverage **internal bandwidth** efficiently
- Not require **expensive hardware inside the SSD** (e.g., large DRAM bandwidth/capacity and costly logic units)

Step 2 Design: Identifying the Common K-mers

- **Challenge:** Limited internal DRAM bandwidth

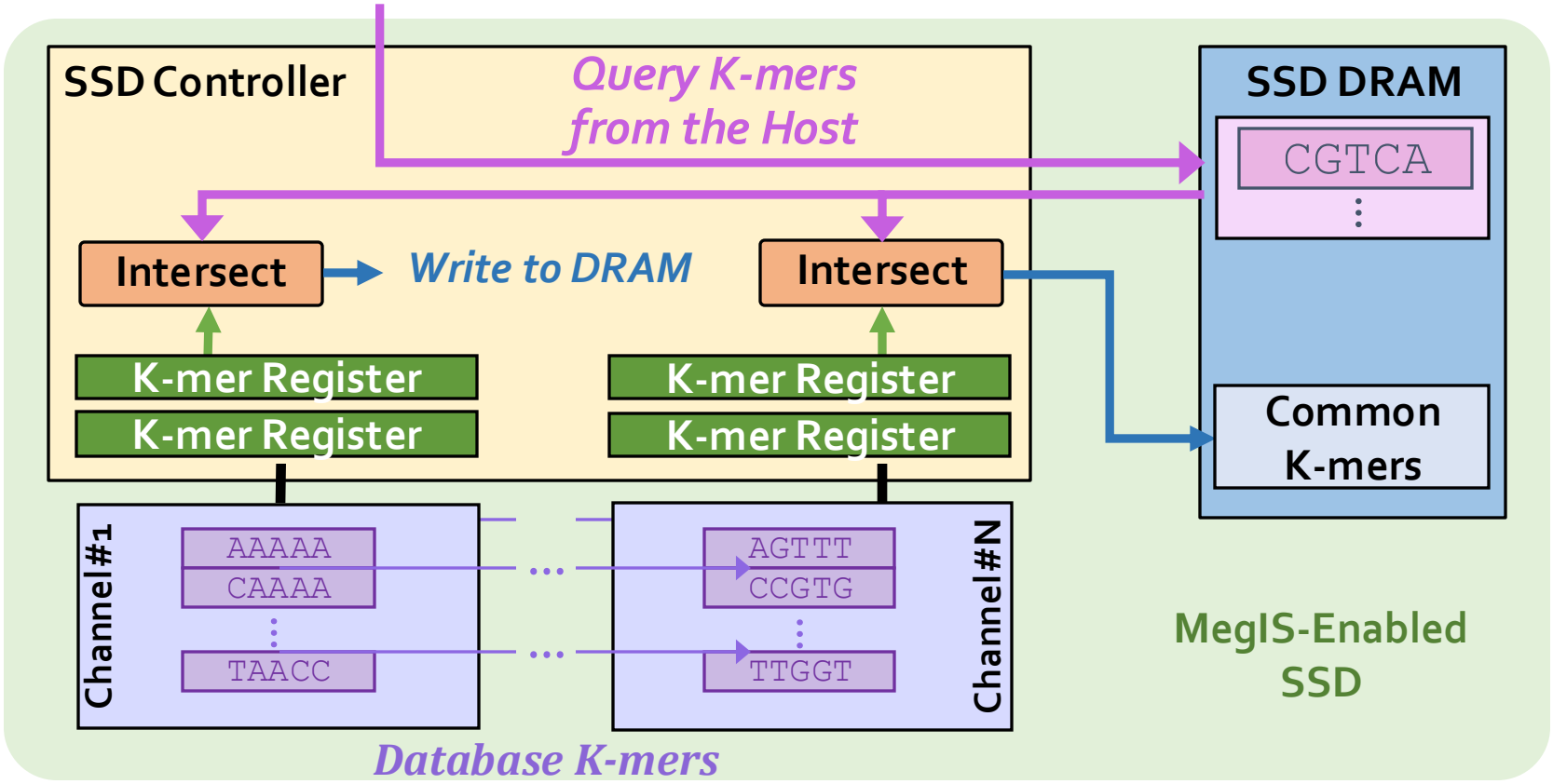


Step 2 Design: Identifying the Common K-mers

- **Challenge:** Limited internal DRAM bandwidth

✓ *Compute directly on the flash data streams* [Zou+, MICRO'22]

✓ *Reduce buffer size based on application features*

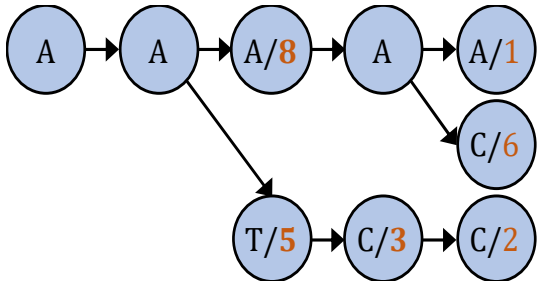


Step 2 Design: Retrieving the Species ID

- MegIS retrieves the species IDs of the **common k-mers** by looking up a **sketch database**

K-mer	
AAAAA	
AAAAC	
AATCC	
…	

Space-Inefficient



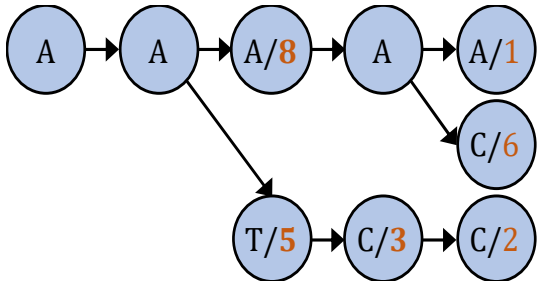
Space-Efficient

✗ ***Slow inside the SSD due to long NAND flash latency***

Step 2 Design: Retrieving the Species ID

- MegIS retrieves the species IDs of the **common k-mers** by looking up a **sketch database**

K-mer	ID
AAAAA	1,5
AAAAC	6
AATCC	2, 9
…	…





K-mer Sketch Streaming is much more suitable for in-storage processing due to its streaming accesses

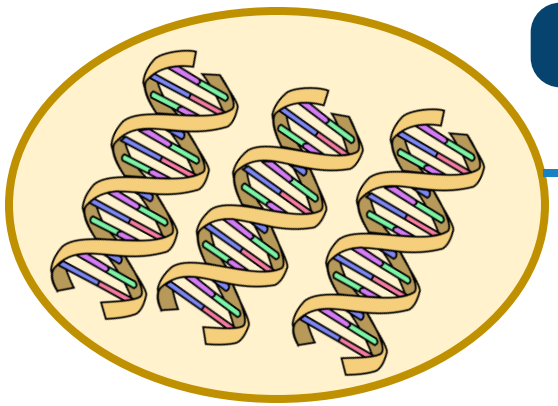
Step 2 Design: Retrieving the Species ID

- MegIS retrieves the species IDs of the **common k-mers** by looking up a **sketch database**

Design details are in the paper

K-mer Sketch Streaming is much more suitable for in-storage processing due to its streaming accesses

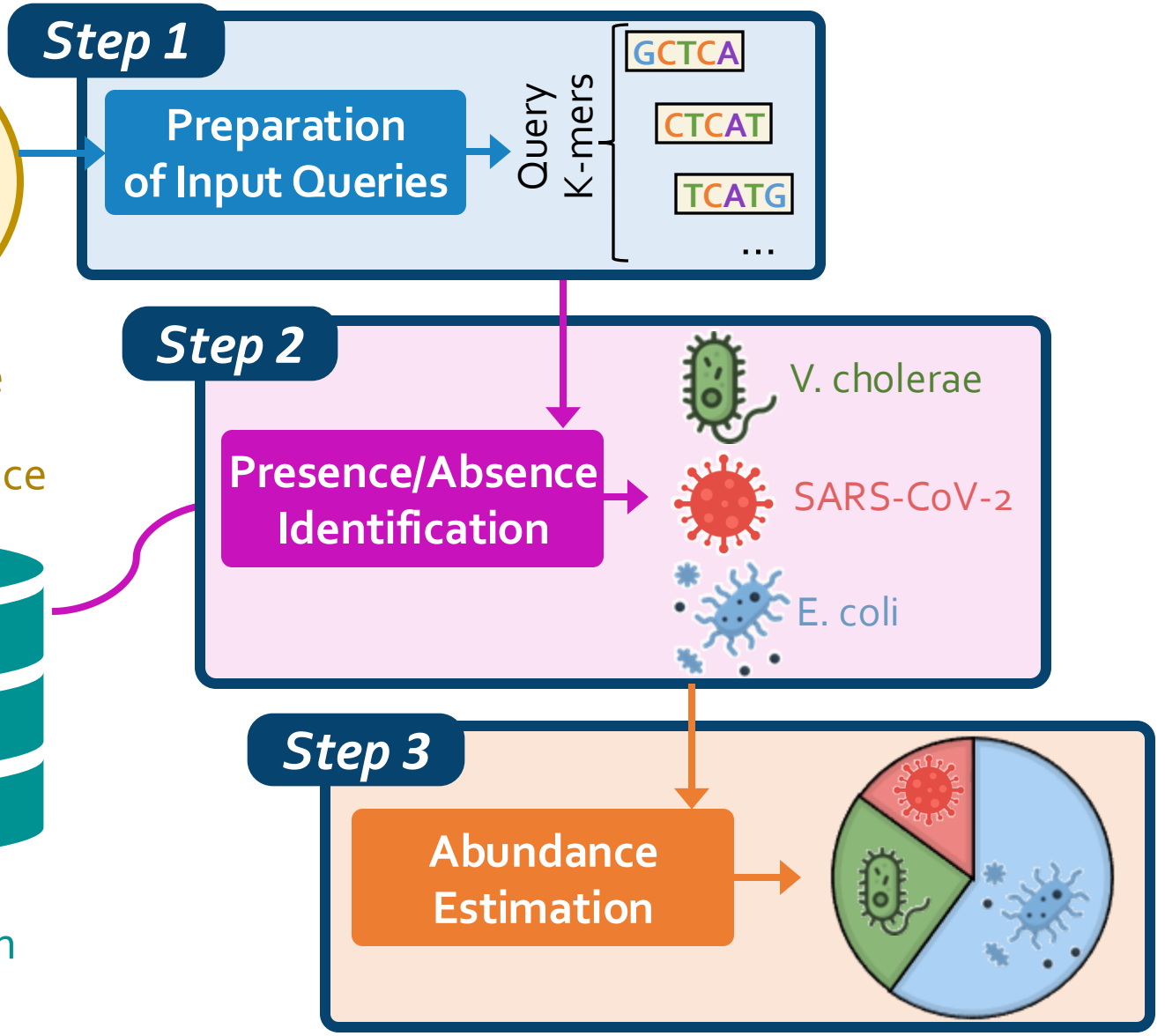
Step 3



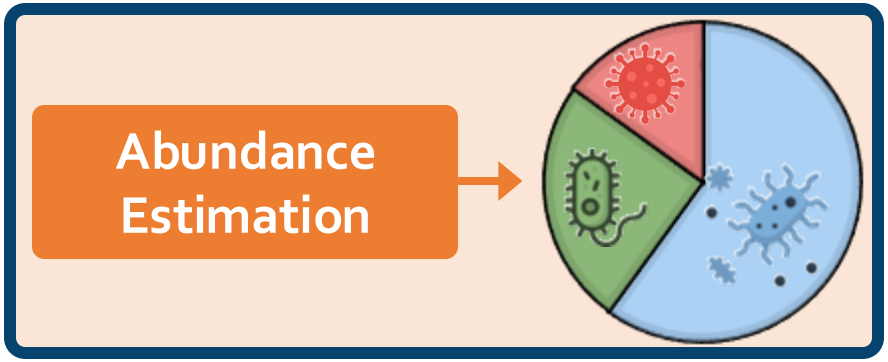
Metagenomic sample with species that are **not known** in advance



A large database containing information on **many species**



Step 3



MegIS performs additional analysis on species identified in the sample to estimate their abundance

MegIS can flexibly integrate with different approaches

1. **Lightweight statistical approaches**: Directly uses the output of Step 2
2. **More accurate and costly read mapping:** MegIS facilitates integration by preparing mapping indexes in the SSD

K-mer	Loc.
ATT	14
CCA	9
GCT	5
...	...

Reference Index Organism A

K-mer	Loc.
AAG	2
CCA	21
TGC	4
...	...

Reference Index Organism B

Merge

K-mer	Loc.
AAG	1002
ATT	14
CCA	9, 1021
GCT	5
TGC	1004
...	...

Unified Reference Index

Step 3 and MegIS FTL are in the paper

Outline

Background

Motivation and Goal

MegIS

Evaluation

Evaluation Methodology Overview (I)

Performance, Energy, and Power Analysis

Hardware Components

- Synthesized Verilog model for the in-storage accelerators
- MQSim *[Tavakkol+, FAST'18]* for SSD's internal operations
- Ramulator *[Kim+, CAL'15]* for SSD's internal DRAM

Software Components

Measure on a real system:

- AMD® EPYC® CPU with 128 physical cores
- 1-TB DRAM

Baseline Comparison Points

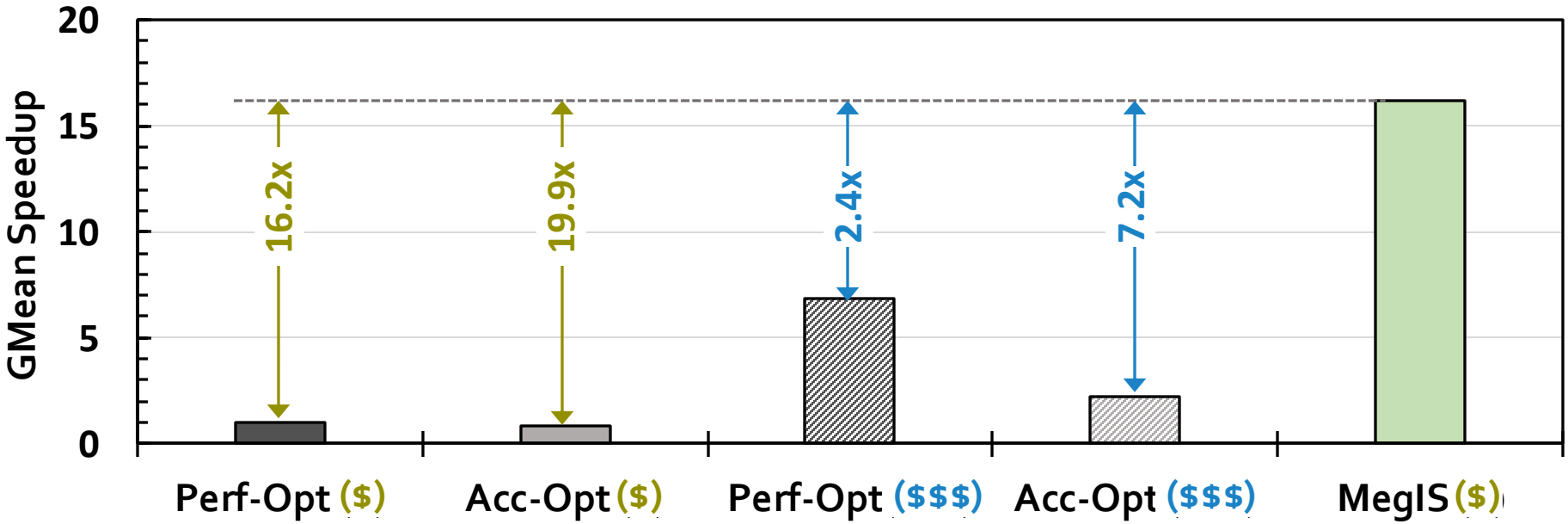
- **Performance-optimized software**, Kraken2 *[Genome Biology'19]*
- **Accuracy-optimized software**, Metalign *[Genome Biology'20]*
- **PIM hardware-accelerated tool** (using processing-in-memory), Sieve *[ISCA'21]*

SSD Configurations

- **SSD-C:** with SATA3 interface (0.5 GB/s sequential read bandwidth)
- **SSD-P:** with PCIe Gen4 interface (7 GB/s sequential read bandwidth)

System Cost-Efficiency

- **Cost-optimized system ($):** With SSD-C and 64-GB DRAM
- **Performance-optimized system ($$$):** With SSD-P and 1-TB DRAM



MegIS outperforms the baselines

***even* when running on a much less costly system**

More in the Paper

MegIS: High-Performance, Energy-Efficient, and Low-Cost Metagenomic Analysis with In-Storage Processing

Nika Mansouri Ghiasi[1] Mohammad Sadrosadati[1] Harun Mustafa[1] Arvid Gollwitzer[1]
Can Firtina[1] Julien Eudine[1] Haiyu Mao[1] Joël Lindegger[1] Meryem Banu Cavlak[1]
Mohammed Alser[1] Jisung Park[2] Onur Mutlu[1]
[1]ETH Zürich [2]POSTECH

https://arxiv.org/abs/2406.19113

https://github.com/CMU-SAFARI/MegIS

Storage-Centric Systems for (Meta)Genomics

Genomics

GenStore *(ASPLOS'22)*

Metagenomics

MegIS *(ISCA'24)*

Emerging Areas in (Meta)Genomics

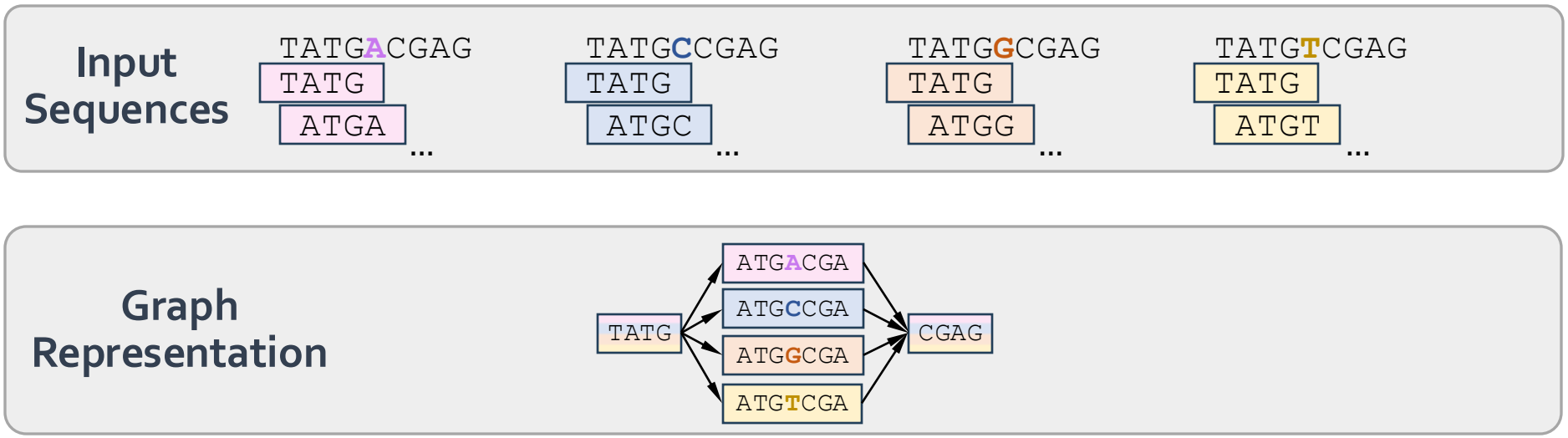
GRAINS
Genome Graph Analysis

MARS *(ICS'25)*
Raw Genomics Signal Analysis

Efficient Analysis on Compressed Genomic Data

SAGe *(to appear in HPCA'26)*

Large-Scale Genome Graphs



More powerful and efficient representations of genomic data

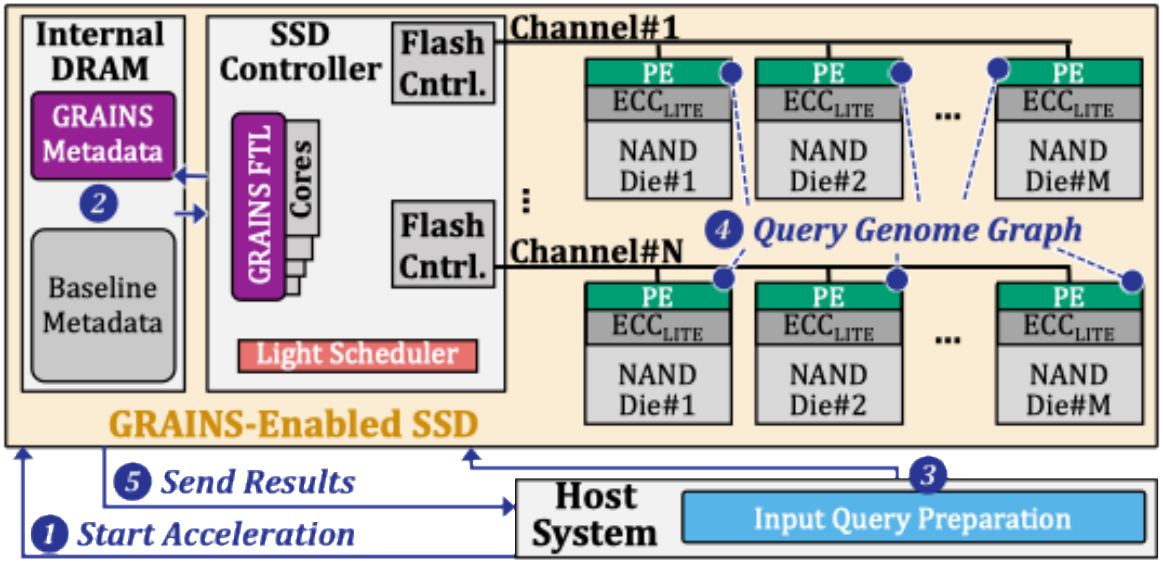
✓ Greater expressive power

✓ More compact representations of massive, population-scale databases

GRAINS

First in-storage processing system for large-scale genome graphs

- More **storage-friendly analysis** via specialized **execution flow** and **scheduling**
- Further improve performance, energy-efficiency, and cost-effectiveness via via **in-storage** and **in-flash processing**



GRAINS provides significant speedup (1.5x – 47.8x) and energy reduction (3.1x-31.6x) at low cost

Storage-Centric Systems for (Meta)Genomics

Genomics

GenStore *(ASPLOS'22)*

Metagenomics

MegIS *(ISCA'24)*

Emerging Areas in (Meta)Genomics

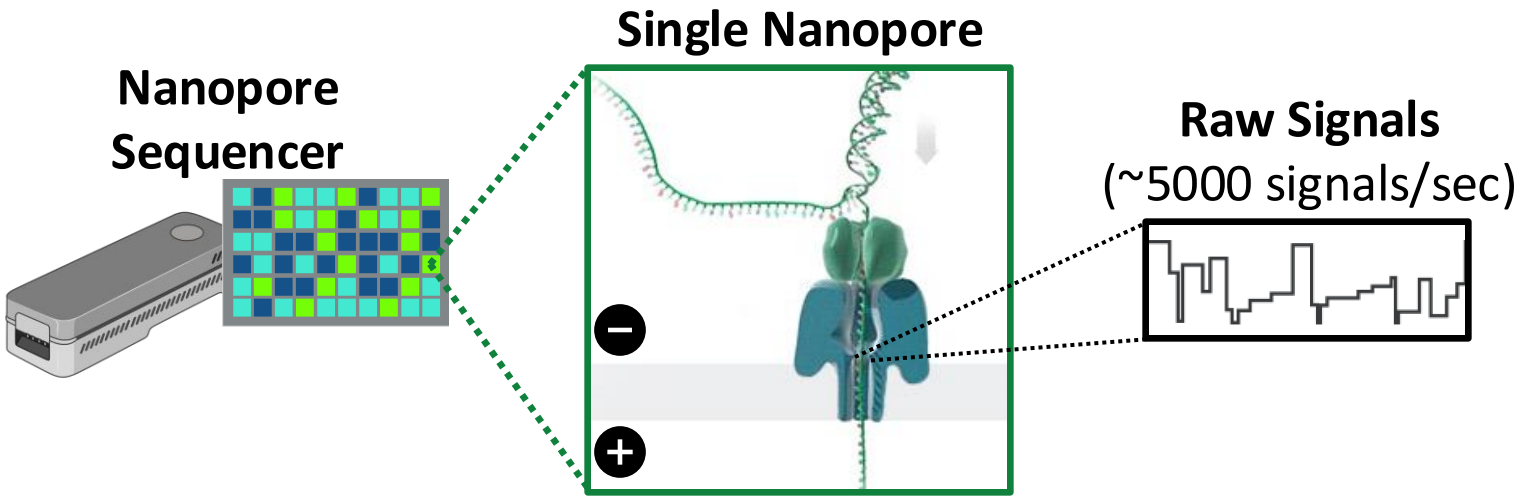
GRAINS
Genome Graph Analysis

MARS *(ICS'25)*
Raw Genomics Signal Analysis

Efficient Analysis on Compressed Genomic Data

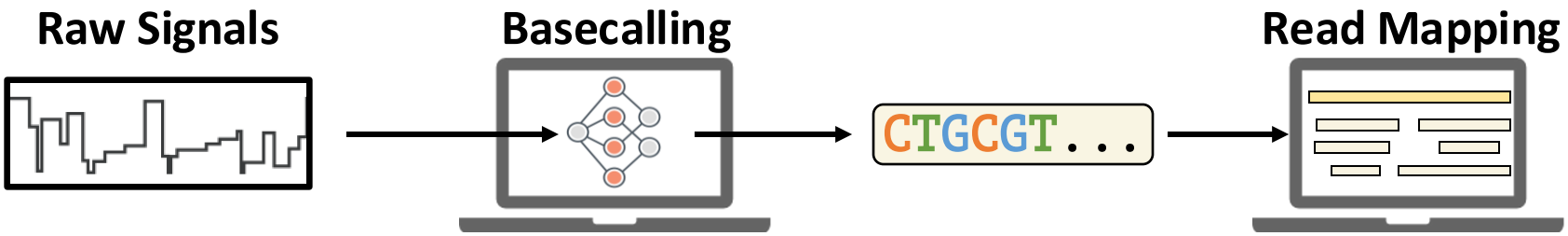
SAGe *(to appear in HPCA'26)*

Raw Signals



Raw Signals: Ionic current measurements generated at a certain **throughput**

Raw Signal Genome Analysis (RSGA)



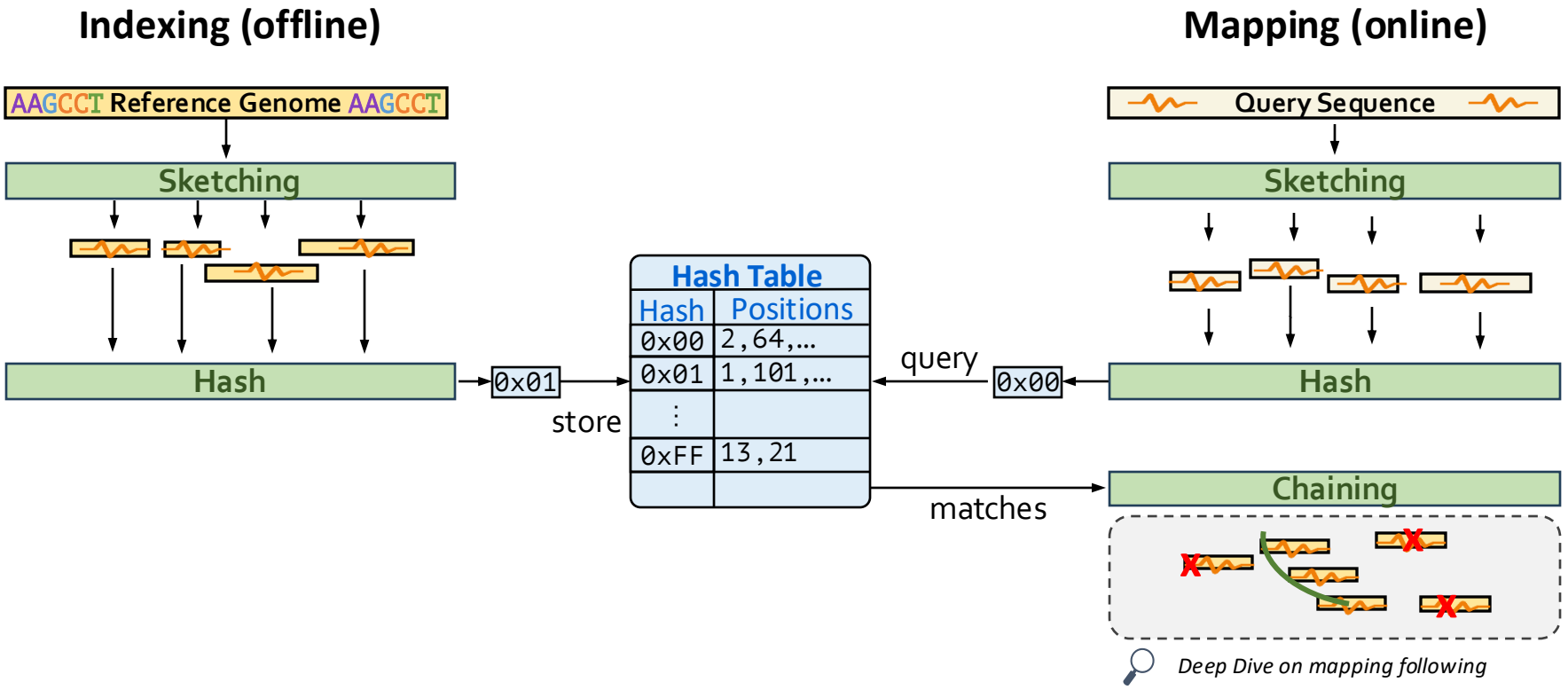
Raw Signals Genome Analysis (RSGA):

A novel approach that operates directly on raw signals

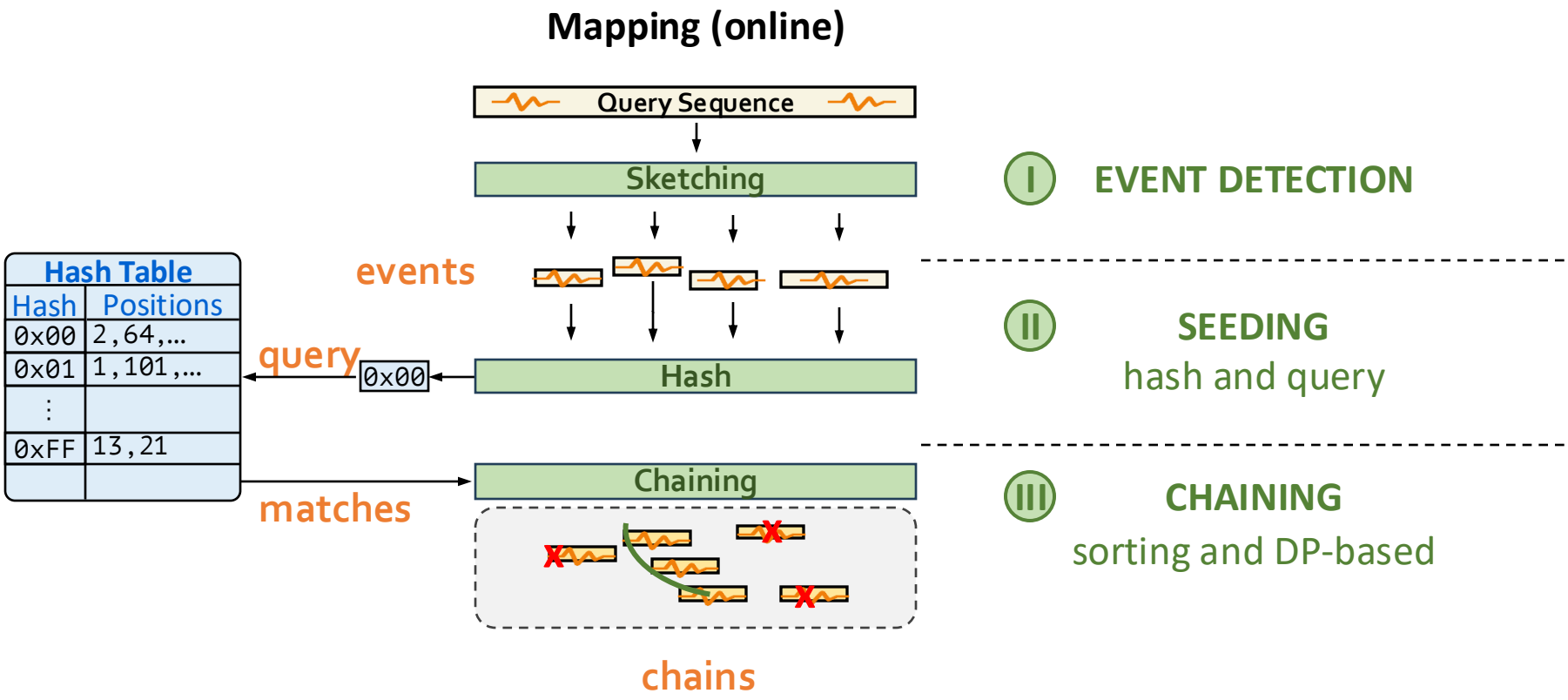
✓ **Raw signals contain richer information than bases**

✓ **Efficient analysis with better scalability and portability**

High Level Overview of RSGA



Mapping in RSGA



Outline

Background

Motivation and Goal

MARS

Evaluation

Conclusion

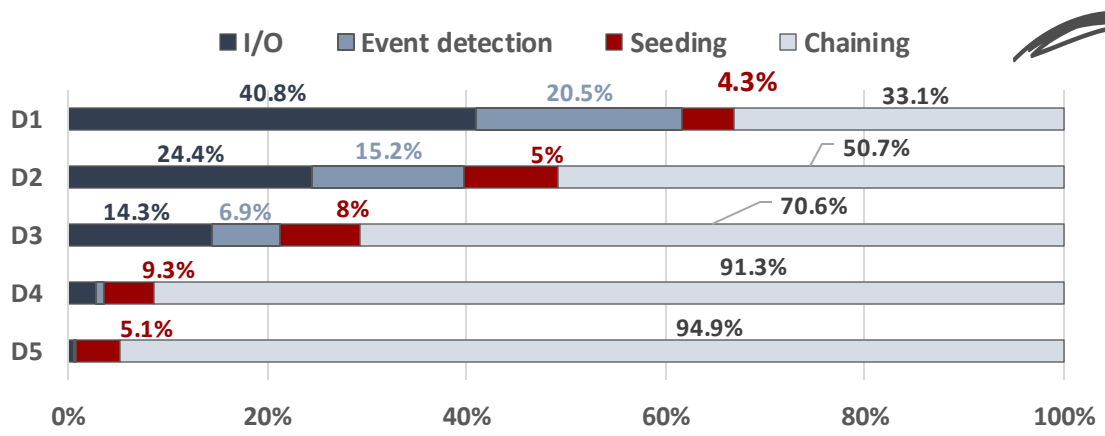
Motivation

Sequencing throughput is growing rapidly
Modern sequencers can generate 100Ks of signal samples per second

RSGA struggles to keep up with increasing real-time requirements
Software pipelines cannot keep up with the pace of raw signal generation

There is a critical need for acceleration

Goal: Identify key performance bottlenecks in the RSGA pipeline
Target the most time- and energy-intensive steps for hardware acceleration

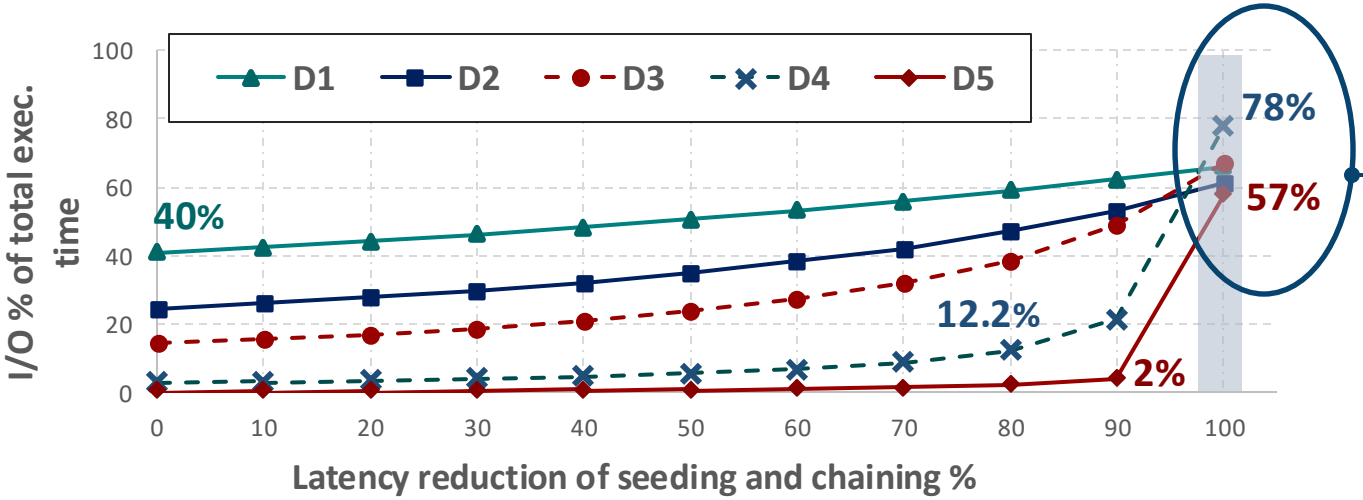


Chaining & **seeding** consistently dominate runtime

Event detection & **I/O overhead** are also non-negligible

Impact of I/O data movement

We model **I/O data movement overhead** when accelerating commonly targeted steps in the pipeline: **seeding** and **chaining**



As compute latency is reduced, **I/O** becomes the **dominant contributor** to total execution time

I/O data movement from SSD becomes dominant overhead as compute steps are accelerated in RSGA

Our Goal



*Design a **high-performance** and **energy-efficient** system for RSGA by addressing both the **I/O data movement** and **computational bottlenecks***

Outline

Background

Motivation and Goal

MARS

Evaluation

Conclusion

MARS: PIM Acceleration of RSGA Inside Storage

- **First In-Storage-Processing system for *RSGA***
- **Key Idea:** In-Storage-Processing system for RSGA that i) **addresses I/O** data movement overhead and ii) exploits SSD parallelization capabilities to **accelerate computationally expensive steps**
- Co-design between **targeted software modifications** and **dedicated hardware components** within the **storage system**



Targeted software modifications to optimize RSGA for in-storage executions

- Tailored to SSD **parallelization capabilities and architecture**



Processing-In-Memory within the SSD to accelerate RSGA steps

- Processing **using** the SSD-internal DRAM
- Processing **near** the SSD-internal DRAM

MARS Genome Analysis Workflow

Software modifications to reduce **computational workload** and **intermediate storage requirements**

- **Filtering Techniques**
  - Discard candidates unlikely to result in valid mapping
  - Includes frequency and vote-based filtering
- **Arithmetic Conversion Techniques**
  - Convert floating-point to fixed-point representations (16 bits)
  - Reduces memory footprint and computational cost

Challenge: Preserve accuracy in the presence of raw signal noise

- Requires careful placement of both techniques in the pipeline
- Involves fine-tuning filter parameters

Impact of SW Modification on Accuracy

We conduct an analysis to show that SW modifications do not have an adverse effect on accuracy

- **Ground Truth**

  Basecalling (Dorado) and **Read Mapping** (minimap2)

- **Datasets:** SARS-CoV-2, E.coli, Yeast, Green Algae, Human

- **Comparison points:**
  - **RawHash2 (RH2)** *[Firtina+, Bioinformatics'24]*: state-of-the-art RSGA algorithm
  - **MS-CPU$_{Float}$**: RSGA + **filtering** techniques
  - **MS_CPU$_{Fixed}$**: RSGA + **filtering** + **arithmetic conversion** techniques

Accuracy Evaluation

	D1 *SARS-CoV-2*			D2 *E.coli*			D3 *Yeast*			D4 *Green Algae*			D5 *Human HG001*		
	Prec.	Recall	F_1	Prec.	Recall	F_1	Prec.	Recall	F_1	Prec.	Recall	F_1	Prec.	Recall	F_1
RH2	0.9868	0.8735	0.9267	0.9573	0.9009	0.9282	0.9862	0.8412	0.9079	0.9691	0.7015	0.8139	0.8949	0.4054	0.5582
MS-CPU$_{Fixed}$	0.9917	0.9694	0.9803	0.9854	0.9574	0.9712	0.9533	0.9643	0.9588	0.9125	0.9166	0.9141	0.8723	0.6318	0.7300
MS-CPU$_{Float}$	0.9939	0.9796	0.9867	0.9893	0.9616	0.9753	0.9551	0.9655	0.9603	0.9254	0.9438	0.9354	0.8763	0.6729	0.7612

Key Observations

Accuracy Evaluation

	D1 *SARS-CoV-2*			D2 *E.coli*			D3 *Yeast*			D4 *Green Algae*			D5 *Human HG001*		
	Prec.	Recall	F_1	Prec.	Recall	F_1	Prec.	Recall	F_1	Prec.	Recall	F_1	Prec.	Recall	F_1
RH2	0.9868	0.8735	0.9267	0.9573	0.9009	0.9282	0.9862	0.8412	0.9079	0.9691	0.7015	0.8139	0.8949	0.4054	0.5582
MS-CPU$_{Fixed}$	0.9917	0.9694	0.9803	0.9854	0.9574	0.9712	0.9533	0.9643	0.9588	0.9125	0.9166	0.9141	0.8723	0.6318	0.7300
MS-CPU$_{Float}$	0.9939	0.9796	0.9867	0.9893	0.9616	0.9753	0.9551	0.9655	0.9603	0.9254	0.9438	0.9354	0.8763	0.6729	0.7612

Key Observations

- MS-CPU **improves recall and F1-score** over RH2

  *Our filtering techniques are **effective in eliminating ambiguous and redundant** candidate matches & allow pipeline to **focus on more informative regions** that are more likely to represent the correct match*

Accuracy Evaluation

	D1 *SARS-CoV-2*			D2 *E.coli*			D3 *Yeast*			D4 *Green Algae*			D5 *Human HG001*		
	Prec.	Recall	F_1	Prec.	Recall	F_1	Prec.	Recall	F_1	Prec.	Recall	F_1	Prec.	Recall	F_1
RH2	0.9868	0.8735	0.9267	0.9573	0.9009	0.9282	0.9862	0.8412	0.9079	0.9691	0.7015	0.8139	0.8949	0.4054	0.5582
MS-CPU$_{Fixed}$	0.9917	0.9694	0.9803	0.9854	0.9574	0.9712	0.9533	0.9643	0.9588	0.9125	0.9166	0.9141	0.8723	0.6318	0.7300
MS-CPU$_{Float}$	0.9939	0.9796	0.9867	0.9893	0.9616	0.9753	0.9551	0.9655	0.9603	0.9254	0.9438	0.9354	0.8763	0.6729	0.7612

Key Observations

- MS-CPU **improves recall and F1-score** over RH2

  *Our filtering techniques are **effective in eliminating ambiguous and redundant** candidate matches & allow pipeline to **focus on more informative regions** that are more likely to represent the correct match*

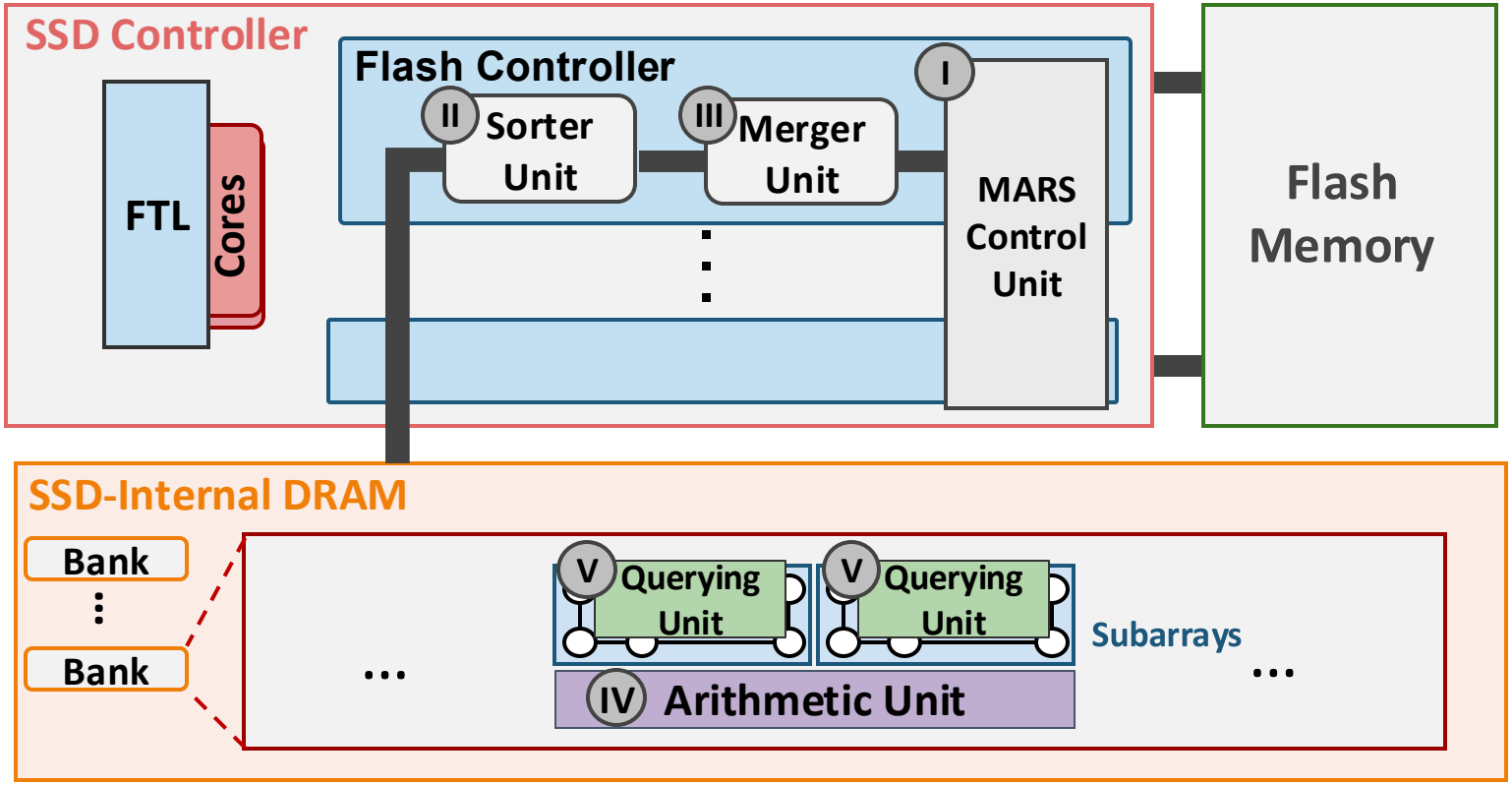
- Using **fixed-point arithmetic** in MS-CPU only leads to minimal accuracy loss

  *We apply **earlier normalization & quantization** to ensure fixed-point arithmetic has no significant impact on our results*

MARS Architecture & System - Overview

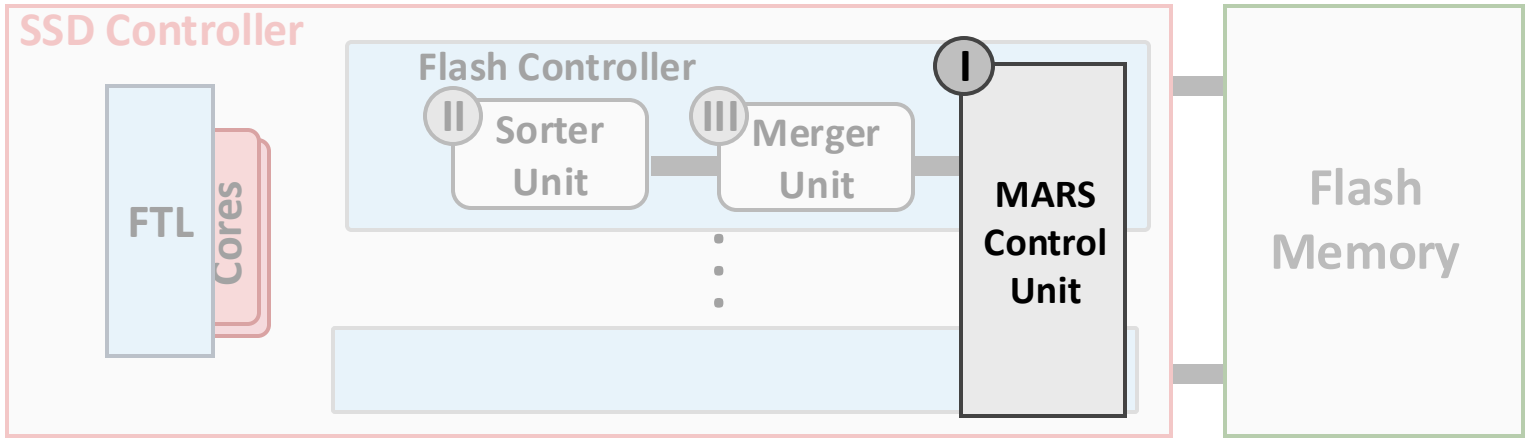
In-Storage-Processing system for *RSGA*

- **Conventional Mode:** SSD performs standard SSD storage operations
- **Accelerator Mode:** SSD executes the full RSGA pipeline



MARS Architecture & System (I/IV)

SSD Controller Components



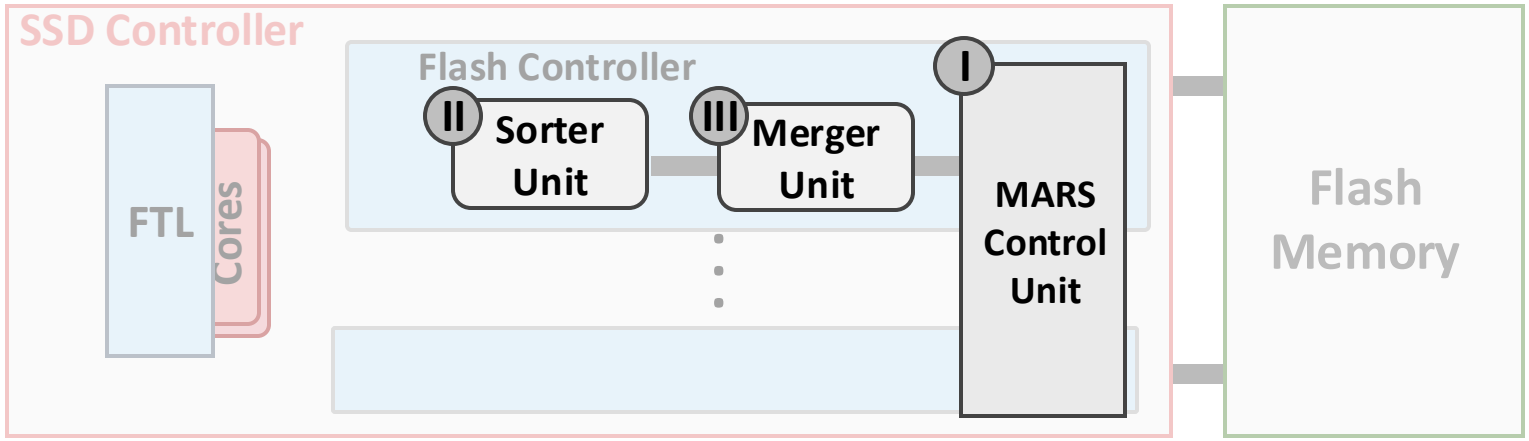
I

MARS Control Unit

Finite State Machine that orchestrates pipeline execution, control and data flow

MARS Architecture & System (II/IV)

SSD Controller Components



I — MARS Control Unit

Finite State Machine that orchestrates pipeline execution, control and data flow

II — Sorter Unit

Performs parallel sorting of sequences using dedicated logic

III — Merger Unit

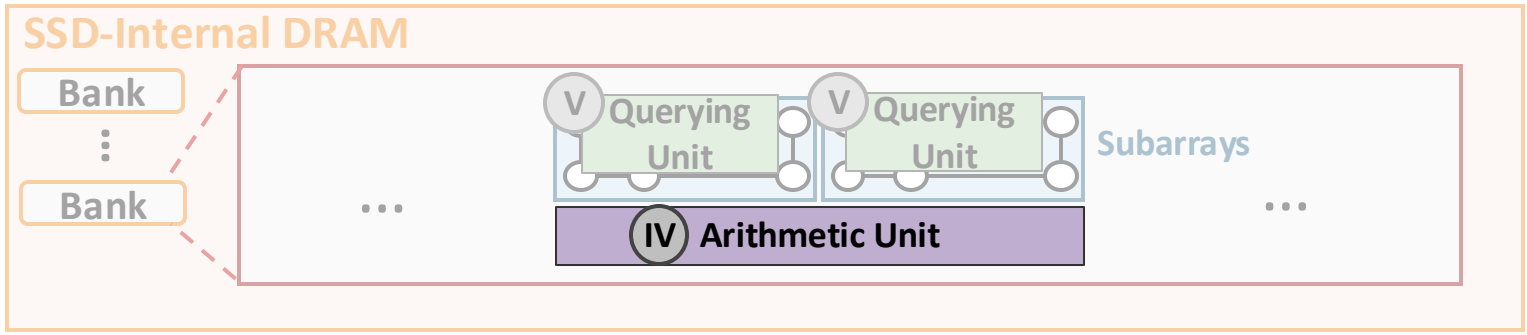
Combines sorted subsequences into longer sorted ones

Paradigm: Implements **Processing Near DRAM**

1 Sorter-Merger pair per flash controller

MARS Architecture & System (III/IV)

SSD-Internal DRAM Components



IV

Arithmetic Unit

Performs arithmetic and logical operations required for RSGA

Paradigm: Implements Processing Near DRAM

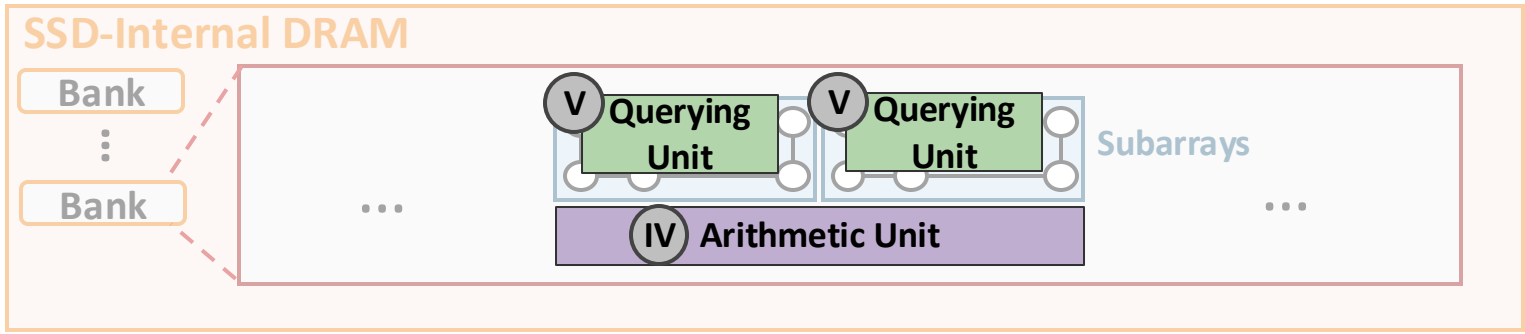
Design: Based on the FULCRUM[1] architecture for leveraging DRAM subarray proximity for low-latency parallel arithmetic operations

1 per two subarrays

1: [Lenjani, HPCA'20];

MARS Architecture & System (IV/IV)

SSD-Internal DRAM Components



IV

Arithmetic Unit

Performs arithmetic and logical operations required for RSGA

Paradigm: Implements Processing Near DRAM

Design: Based on the FULCRUM[1] architecture for leveraging DRAM subarray proximity for low-latency parallel arithmetic operations

1 per two subarrays

V

Querying Unit

Performs efficient, in-memory hash table lookups for seeding

Paradigm: Implements Processing Using DRAM

Design: Based on the PLUTO[2] architecture leveraging DRAM row activation for massively parallel hash-table lookup operations

1 per subarray

1: [Lenjani, HPCA'20]; 2: [Ferreira+, MICRO'22]

Mapping RSGA Workflow to MARS Architecture

1. Break down RSGA steps into **fine-grained tasks**
   - Arithmetic, querying and sorting operations

Mapping RSGA Workflow to MARS Architecture

1. Break down RSGA steps into **fine-grained tasks**
   - Arithmetic, querying and sorting operations

2. Implement MARS as a **pipeline of these tasks**
   - Each task mapped to a dedicated compute unit
     - e.g., multiplications mapped to arithmetic unit

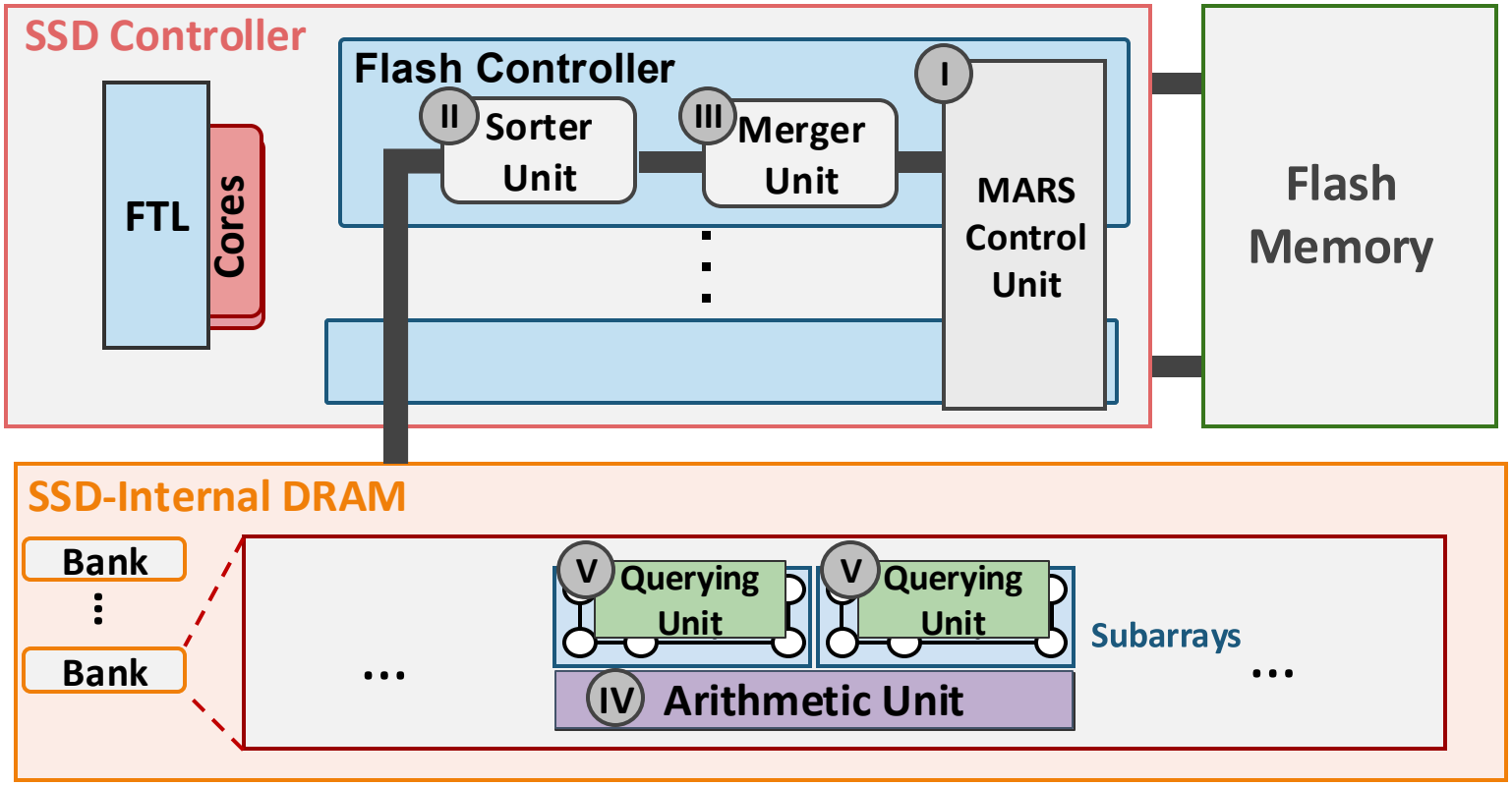
Mapping RSGA Workflow to MARS Architecture

1. Break down RSGA steps into **fine-grained tasks**
   - Arithmetic, querying and sorting operations

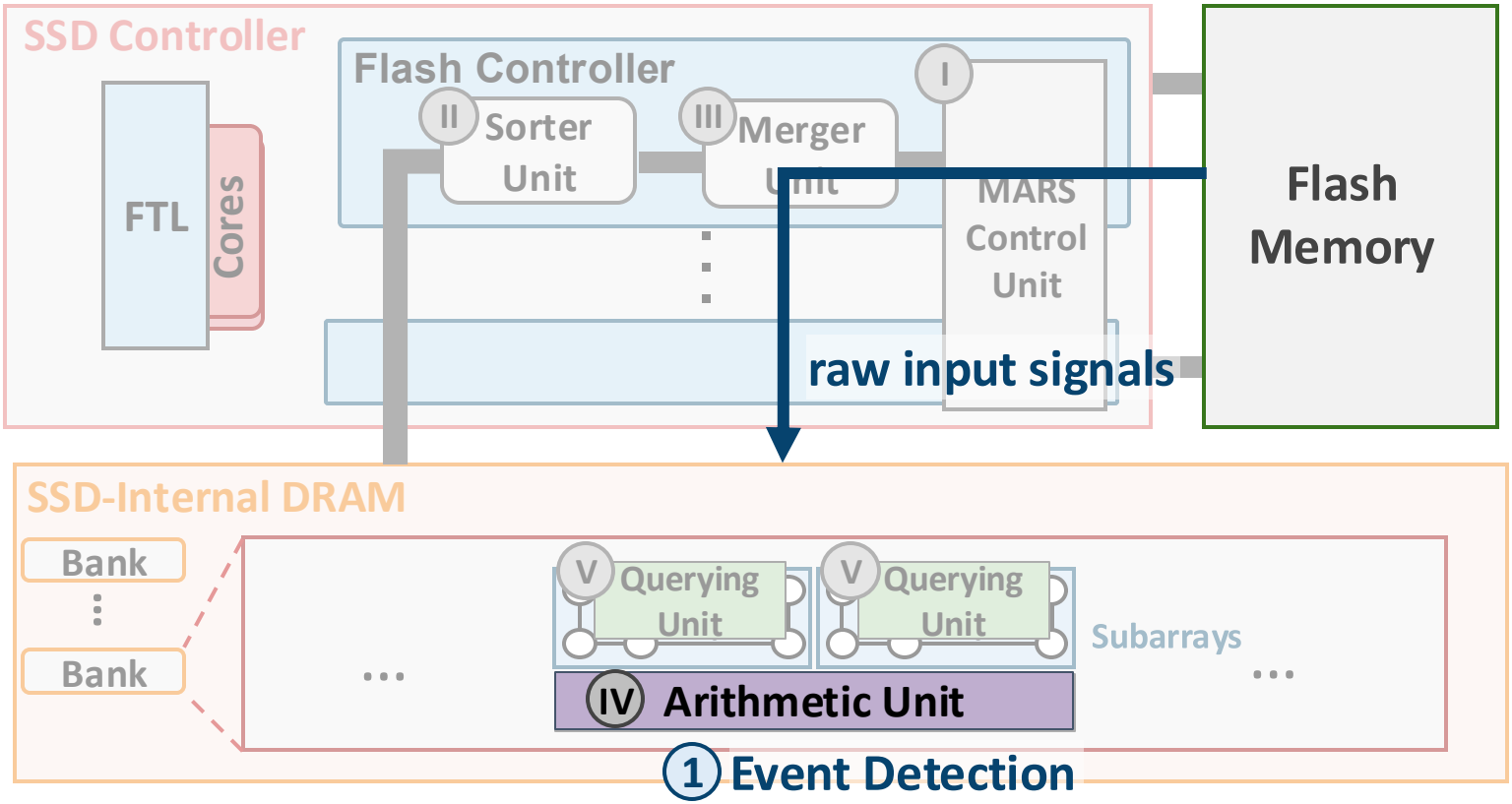
2. Implement MARS as a **pipeline of these tasks**
   - Each task mapped to a dedicated compute unit
     - e.g., multiplications mapped to arithmetic unit

3. MARS Control Unit encodes pipeline into a **Finite State Machine**
   - Defines a fixed execution sequence
   - Dynamically triggers tasks when input data is ready

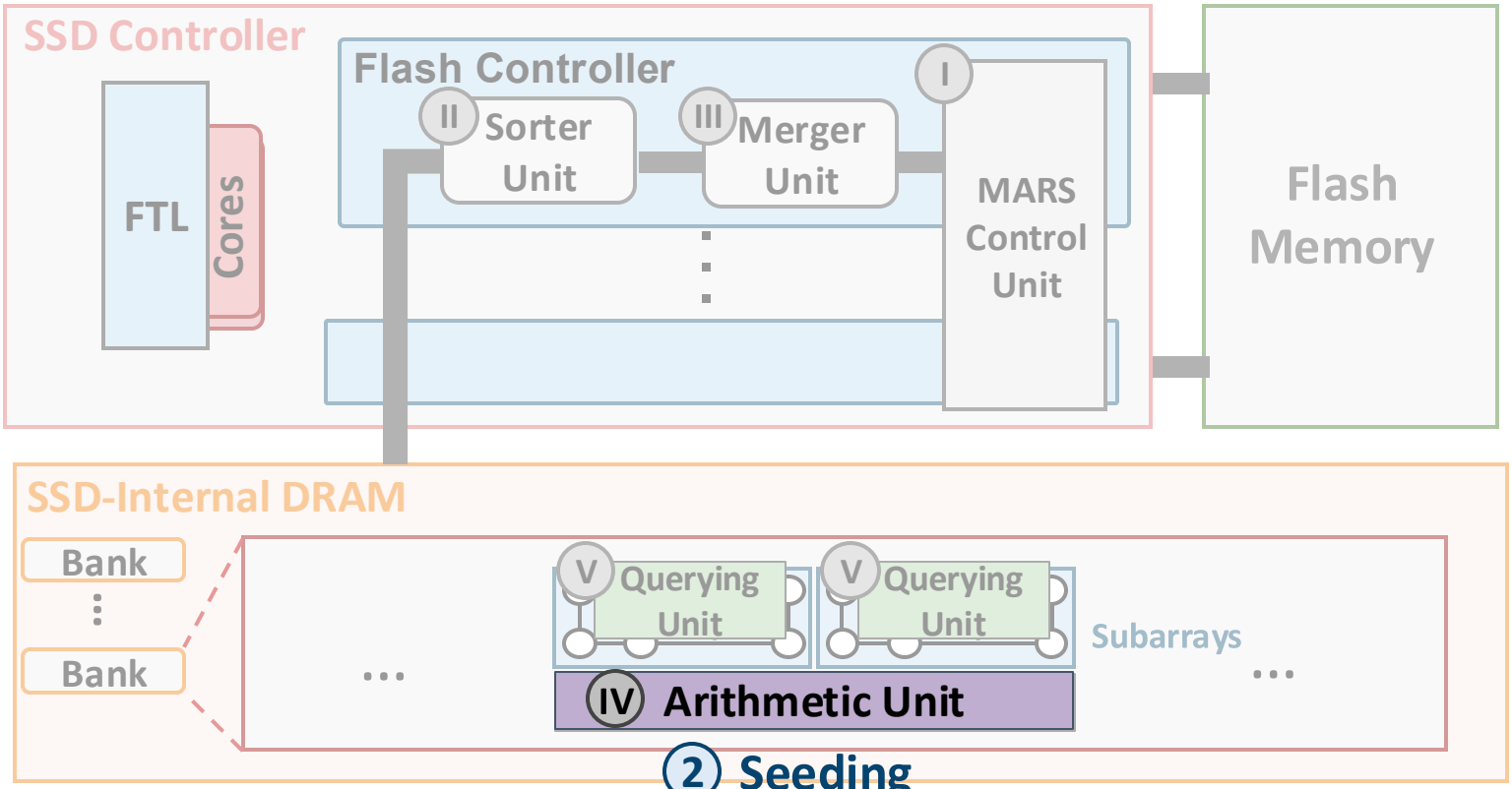
Control and Data Flow



Control and Data Flow (I/VI)



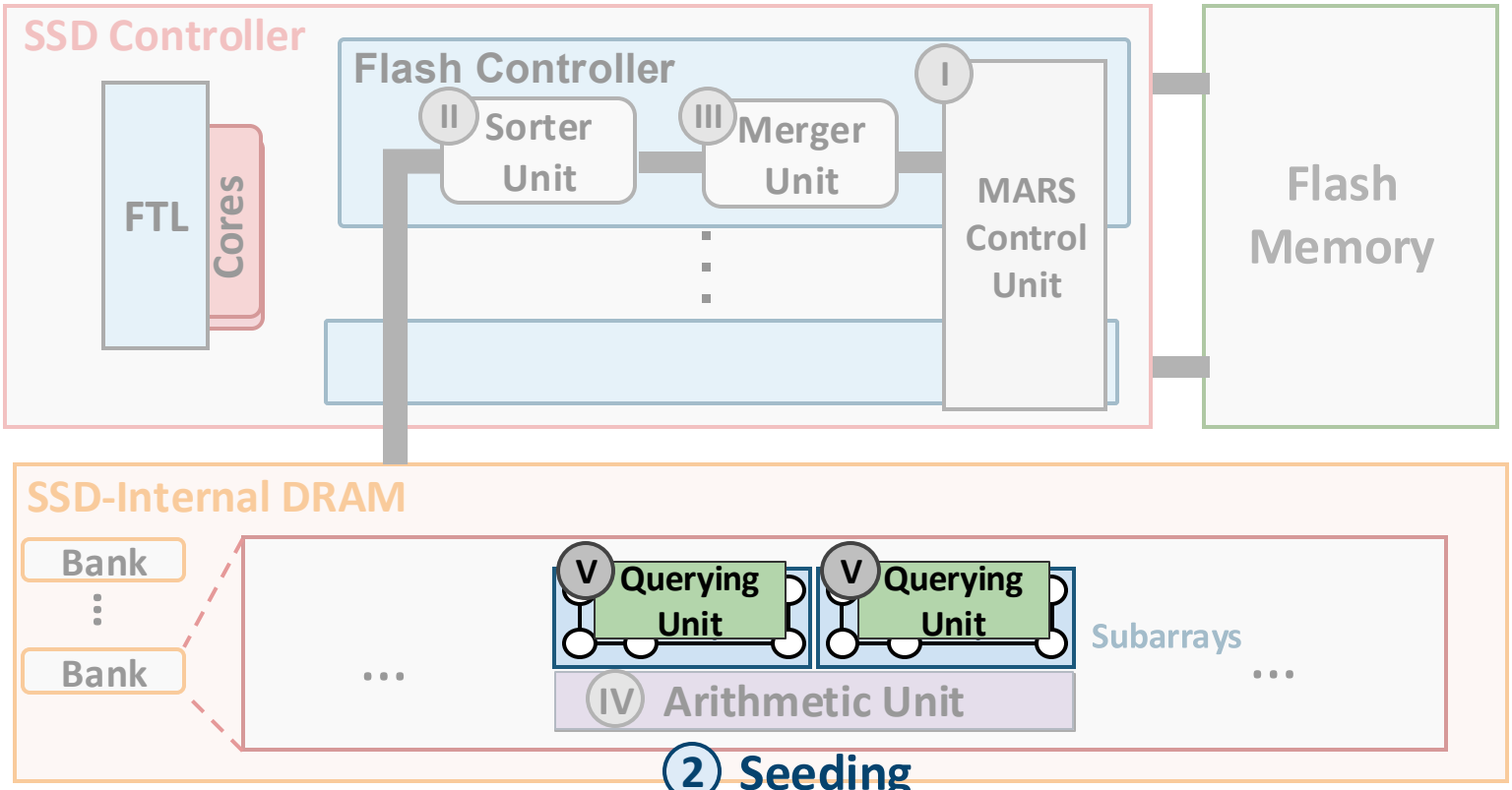
Control and Data Flow (II/VI)



2 Seeding

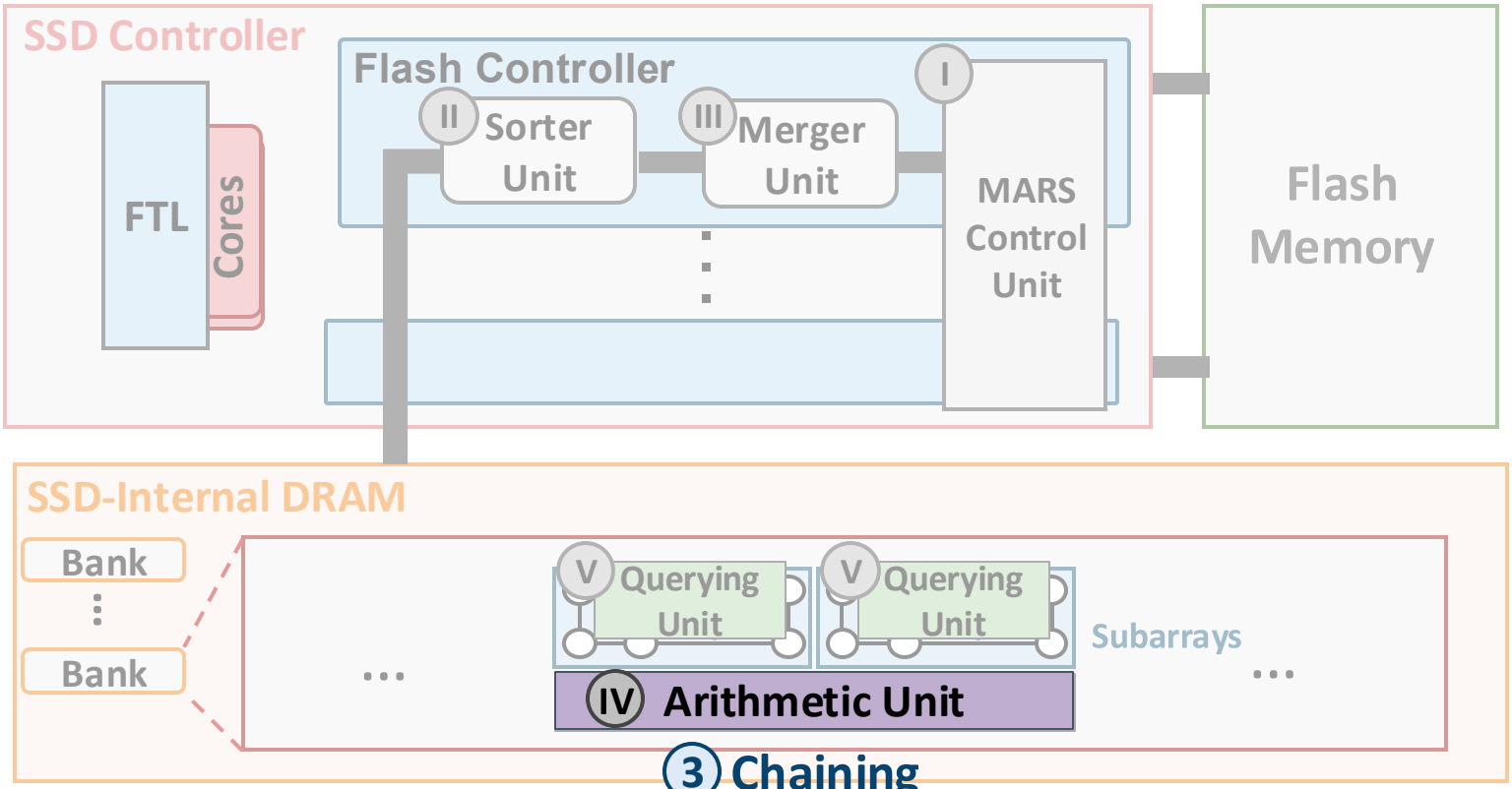
(Hash-value generation & filtering)

Control and Data Flow (III/VI)

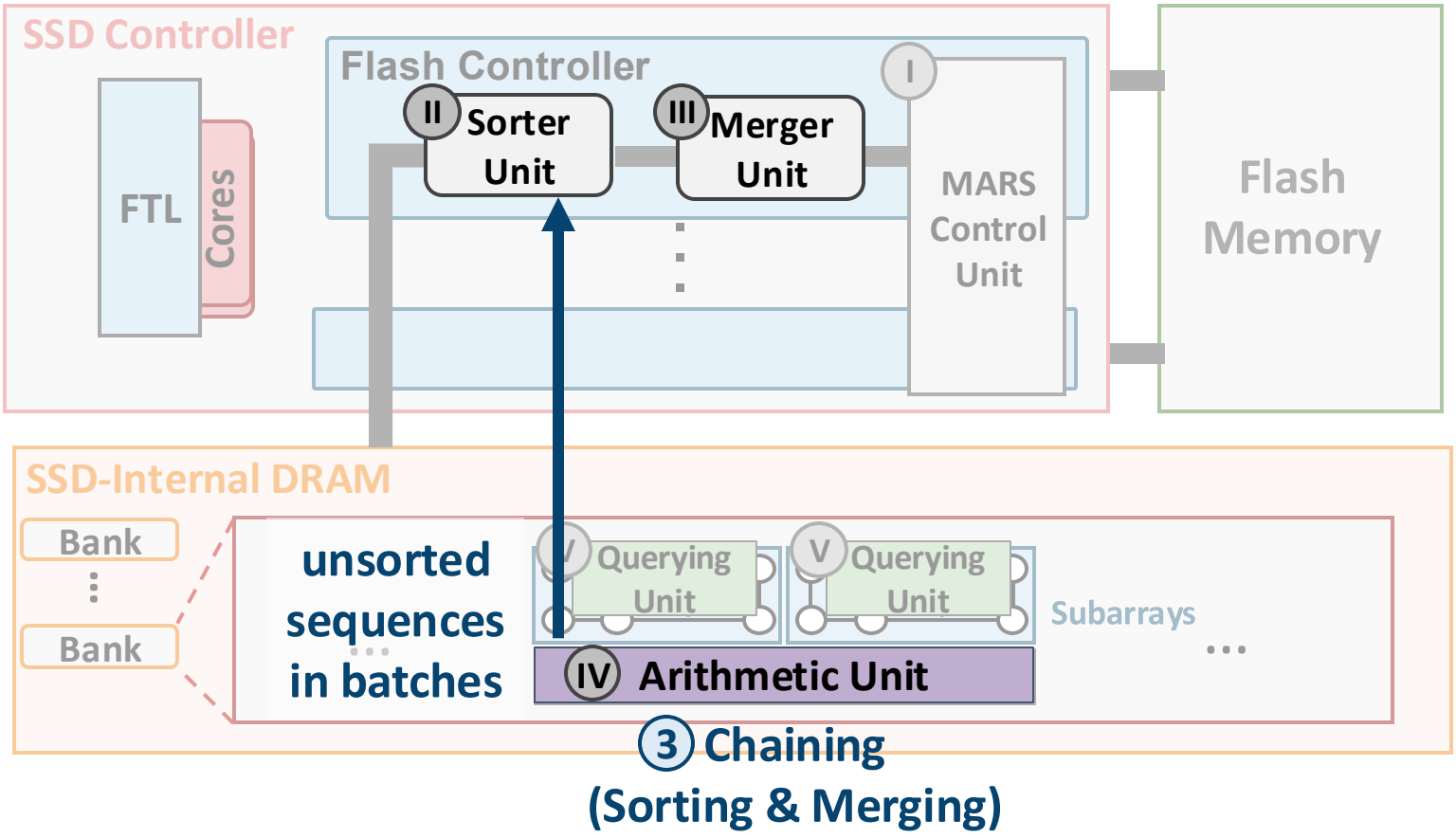


② Seeding
(Hash-table querying)

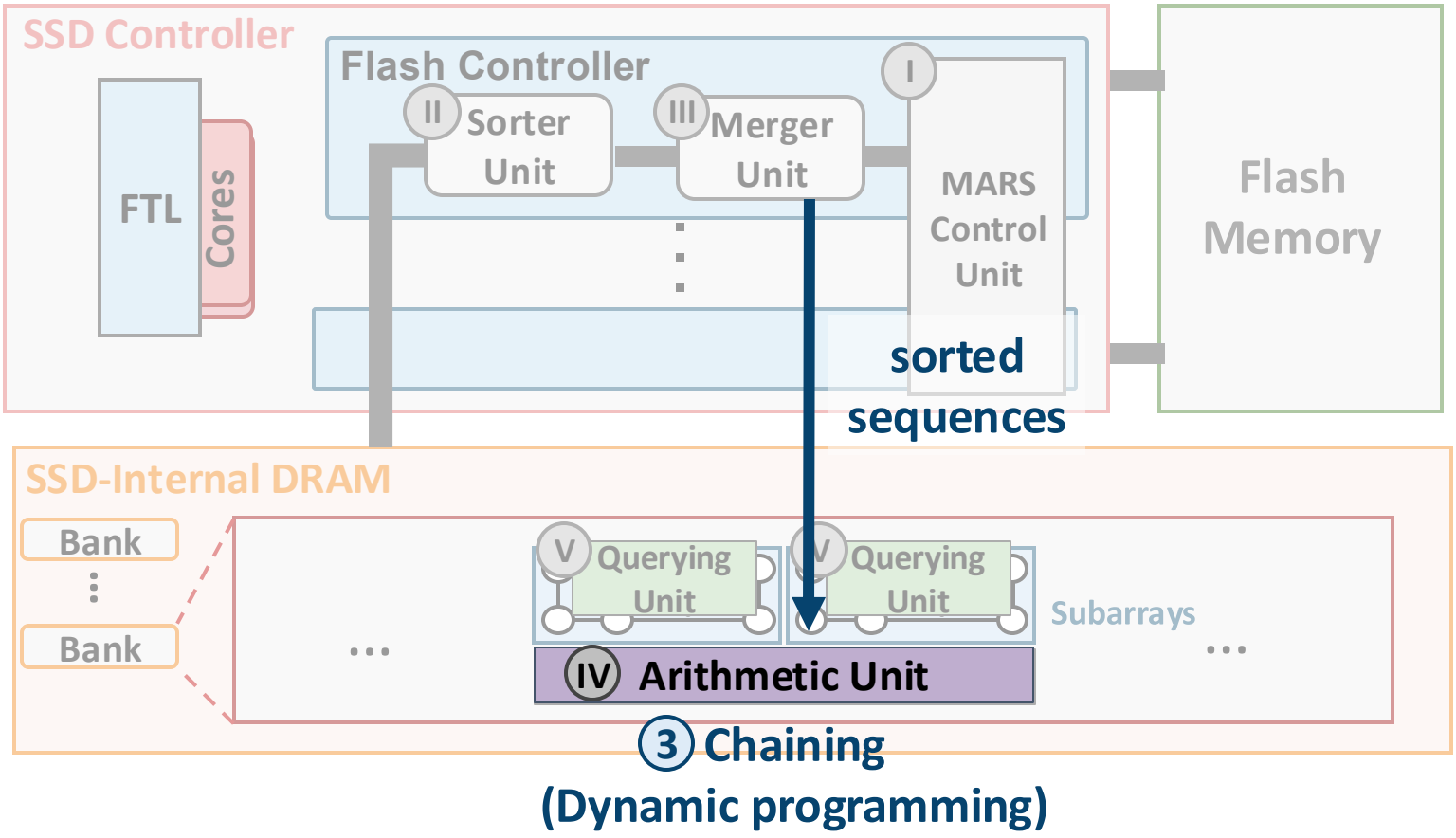
Control and Data Flow (IV/VI)



Control and Data Flow (V/VI)



Control and Data Flow (VI/VI)



Computational Units & System Integration

- Arithmetic Unit
  - Processing-Near-Memory
  - Add Logic near the subarrays
  - Column-Selection Latches

More details on the Mechanism and System Integration

can be found in the paper

and Gated Sense Amplifiers

- Sorter and Merger
  - Inside the SSD controller
  - Processing-Near-Memory
  - Bitonic Sorter and Merger

Outline

Background

Motivation and Goal

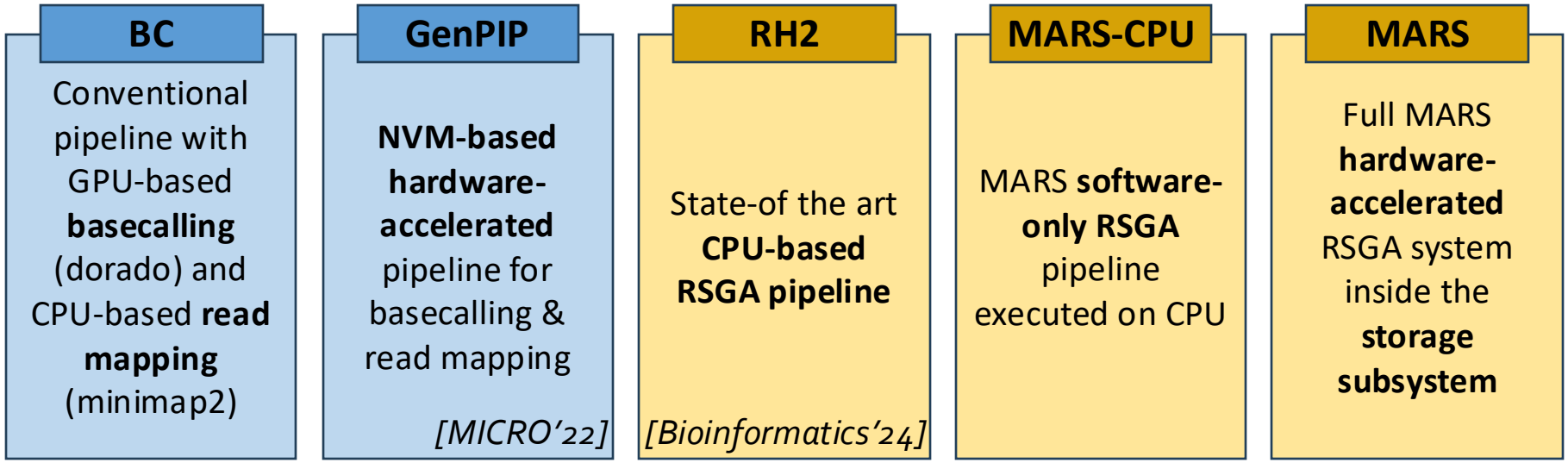
MARS

Evaluation

Conclusion

Evaluation Methodology (I/II)

We compare MARS against four systems for our performance and energy analysis



Evaluation Methodology (II/II)

We use the following hardware setup and simulate elements of our system setup

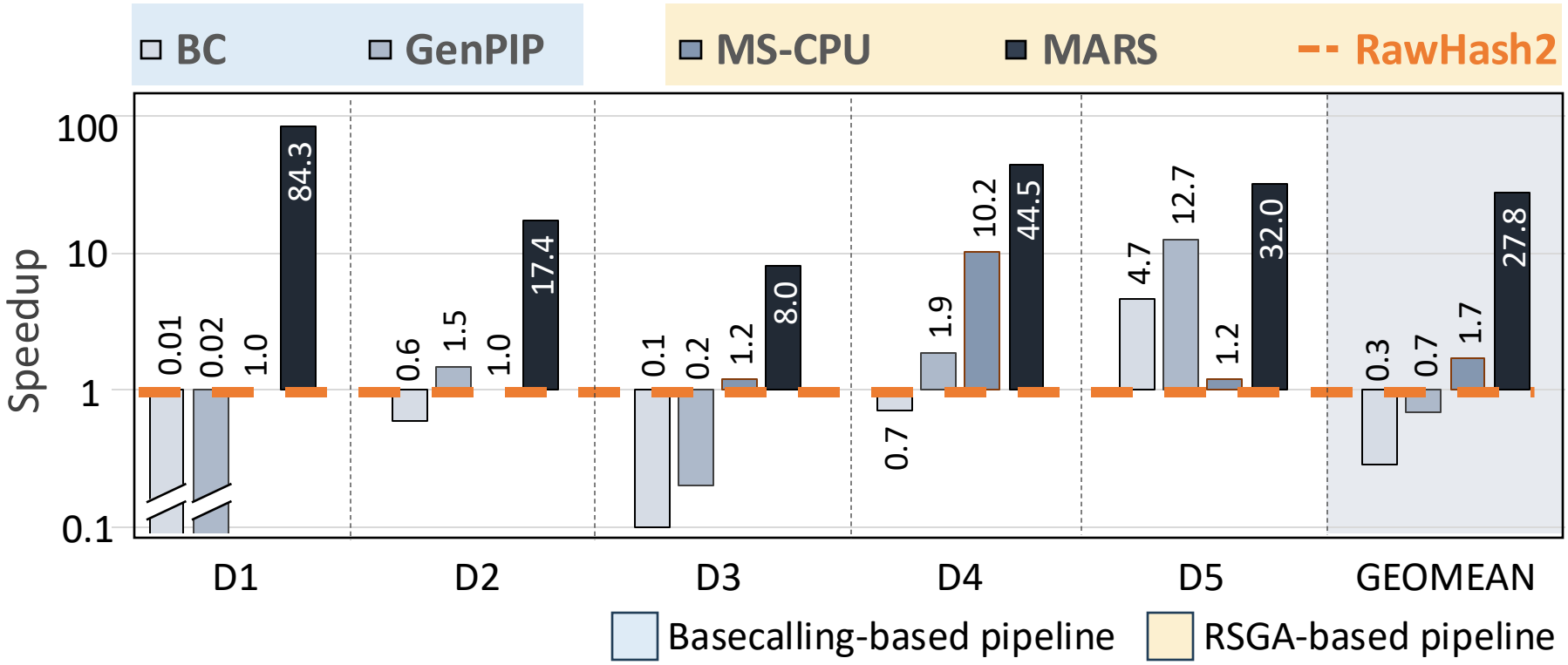
Hardware Components

- **CPU-based baseline**
  - state-of-the-art server with two AMD® EPYC® CPUs with 64 physical cores
  - 1-TB DDR4 DRAM
  - NVIDIA RTX A6000 GPU *(for the BC baseline)*
- **SSD configurations**
  - 8 channels with each 8 flash chips, PCIe 4.0 interface
  - 4GB LPDDR4 DRAM with 16 banks and 512 subarrays

Simulators Used

- Synopsis Design Compiler - for accelerators
- MQSim *[Tavakkol+, FAST'18]* - for SSD-internal operations
- CACTI7 *[Kim+, CAL'15]* - for SSD-internal LPDDR4 DRAM

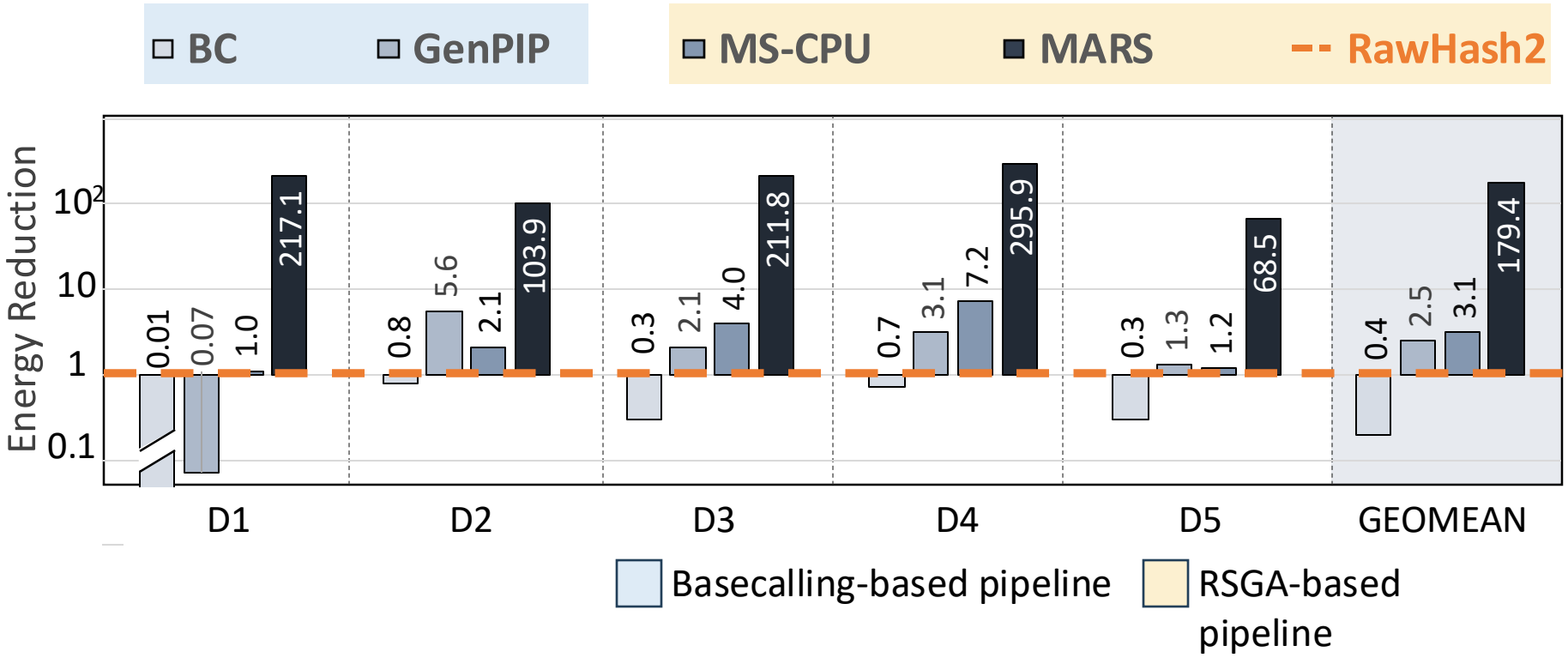
Performance Evaluation



Key Observations

- MARS outperforms all other baselines across all datasets
- MARS outperforms GenPIP adopting PIM outside storage
  We fundamentally solve the I/O data movement overhead problem
- MARS-CPU provide speedup of 1.7x over RH2
  Our SW modifications are effective in reducing the computational load

Energy Evaluation



Key Observation

- MARS reduces energy consumption over all other baselines across all datasets
  - *Improvement stems from*
  - *(1) reduced execution time*
  - *(2) fundamentally addressing the I/O data movement overhead*

Outline

Background

Motivation and Goal

MARS

Evaluation

Conclusion

Conclusion

MARS

The *first* **in-storage processing** system for *end-to-end* raw signal genome analysis enables through **SW improvements** and a **HW system** inside the storage system



High accuracy

matches **basecalling-based pipelines**

improves F1 scores over **state-of-the-art RSGA pipeline**



Improves performance

93× over **state-of-the-art software-based basecalling read mapping** pipeline

40× over **state-of-the-art hardware-accelerated basecalling read mapping** pipeline

28× over **state-of-the-art software-based RSGA read mapping** pipeline



Reduces energy consumption

427× over **state-of-the-art software-based read mapping** pipeline

72× over **state-of-the-art hardware-accelerated read mapping** pipeline

180× over **state-of-the-art software-based RSGA read mapping** pipeline

Raw Signal Genome Analysis (RSGA)

MARS: Processing-In-*Memory* *Acceleration* of *Raw Signal Genome Analysis Inside the Storage Subsystem*

Melina Soysal[†] Konstantina Koliogeorgi[†] Can Firtina[†] Nika Mansouri Ghiasi[†]
Rakesh Nadig[†] Haiyu Mao[⋆] Geraldo F. Oliveira[†]
Yu Liang[†] Klea Zambaku[†] Mohammad Sadrosadati[†] Onur Mutlu[†]

[†] *ETH Zürich* [⋆] *King's College London*

https://arxiv.org/pdf/2506.10931

Further Readings on RSGA (I)

- Can Firtina, Nika Mansouri Ghiasi, Joel Lindegger, Gagandeep Singh, Meryem Banu Cavlak, Haiyu Mao, and Onur Mutlu,
  "RawHash: Enabling Fast and Accurate Real-Time Analysis of Raw Nanopore Signals for Large Genomes"
  *Proceedings of the 31st Annual Conference on Intelligent Systems for Molecular Biology (**ISMB**) and the 22nd European Conference on Computational Biology (**ECCB**)*, Jul 2023
  [Bioinformatics Journal version]
  [Slides (pptx) (pdf)]
  [Talk Video at ISMB/ECCB 2023] (19 minutes)
  [RawHash Source Code]

Bioinformatics, 2023, **39**, i297–i307
https://doi.org/10.1093/bioinformatics/btad272
ISMB/ECCB 2023

OXFORD

RawHash: enabling fast and accurate real-time analysis of raw nanopore signals for large genomes

Can Firtina [1,*], Nika Mansouri Ghiasi [1], Joel Lindegger [1], Gagandeep Singh [1], Meryem Banu Cavlak [1], Haiyu Mao [1], Onur Mutlu [1,*]

[1]Department of Information Technology and Electrical Engineering, ETH Zurich, 8092 Zurich, Switzerland

*Corresponding author. Department of Information Technology and Electrical Engineering, ETH Zurich, Gloriastrasse 35, 8092 Zurich, Switzerland.
E-mail: firtinac@ethz.ch (C.F.), omutlu@ethz.ch (O.M.)

Further Readings on RSGA (II)

- Can Firtina, Melina Soysal, Joël Lindegger, and Onur Mutlu,
  "RawHash2: Mapping Raw Nanopore Signals Using Hash-Based Seeding and Adaptive Quantization"
  Bioinformatics, 30 July 2024.
  [Online link at Bioinformatics Journal]
  [arXiv version]
  [RawHash Talk Video] (19 minutes)
  [RawHash2 Source Code]

Bioinformatics, 2024, **40(8)**, btae478
https://doi.org/10.1093/bioinformatics/btae478
Advance Access Publication Date: 30 July 2024
Applications Note

OXFORD

Sequence analysis

RawHash2: mapping raw nanopore signals using hash-based seeding and adaptive quantization

Can Firtina [1,*], Melina Soysal [1], Joël Lindegger [1], Onur Mutlu [1,*]

[1]Department of Information Technology and Electrical Engineering, ETH Zurich, Zurich 8092, Switzerland

*Corresponding authors. Department of Information Technology and Electrical Engineering, ETH Zurich, Zurich 8092, Switzerland.
E-mail: firtinac@ethz.ch (C.F.); omutlu@ethz.ch (O.M.)

Further Readings on RSGA (III)

- Joel Lindegger, Can Firtina, Nika Mansouri Ghiasi, Mohammad Sadrosadati, Mohammed Alser, and Onur Mutlu,
  "RawAlign: Accurate, Fast, and Scalable Raw Nanopore Signal Mapping via Combining Seeding and Alignment"
  IEEE Access, December 2024.
  [arXiv version]
  [RawAlign Source Code]

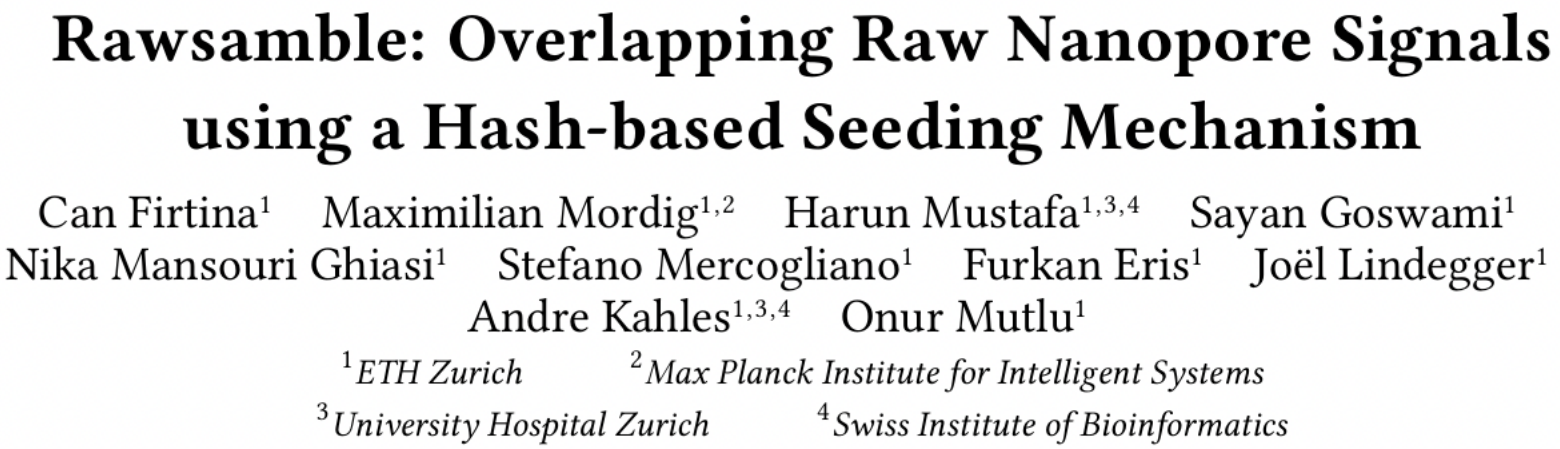
RawAlign: Accurate, Fast, and Scalable Raw Nanopore Signal Mapping via Combining Seeding and Alignment

Joël Lindegger[§] Can Firtina[§] Nika Mansouri Ghiasi[§]
Mohammad Sadrosadati[§] Mohammed Alser[§] Onur Mutlu[§]

[§]*ETH Zürich*

Further Readings on RSGA (IV)

- Can Firtina, Maximilian Mordig, Harun Mustafa, Sayan Goswami, Nika Mansouri Ghiasi, Stefano Mercogliano, Furkan Eris, Joël Lindegger, Andre Kahles, and Onur Mutlu, **"Rawsamble: Overlapping and Assembling Raw Nanopore Signals using a Hash-based Seeding Mechanism"**
[arXiv preprint]
*Presented at the 32nd Annual Conference on Intelligent Systems for Molecular Biology (**ISMB**)*, Montreal, QC, Canada, July 2024.
[Slides (pptx) (pdf)]
[CSHL Biological Data Science 2024 Talk Slides (pptx) (pdf)]
[CSHL Biological Data Science 2024 Talk Video (19 minutes) (16 minutes)]
[Rawsamble Source Code]



Rawsamble: Overlapping Raw Nanopore Signals using a Hash-based Seeding Mechanism

Can Firtina[1] Maximilian Mordig[1,2] Harun Mustafa[1,3,4] Sayan Goswami[1]
Nika Mansouri Ghiasi[1] Stefano Mercogliano[1] Furkan Eris[1] Joël Lindegger[1]
Andre Kahles[1,3,4] Onur Mutlu[1]

[1] *ETH Zurich* [2] *Max Planck Institute for Intelligent Systems*
[3] *University Hospital Zurich* [4] *Swiss Institute of Bioinformatics*

Storage-Centric Systems for (Meta)Genomics

Genomics

GenStore *(ASPLOS'22)*

Metagenomics

MegIS *(ISCA'24)*

Emerging Areas in (Meta)Genomics

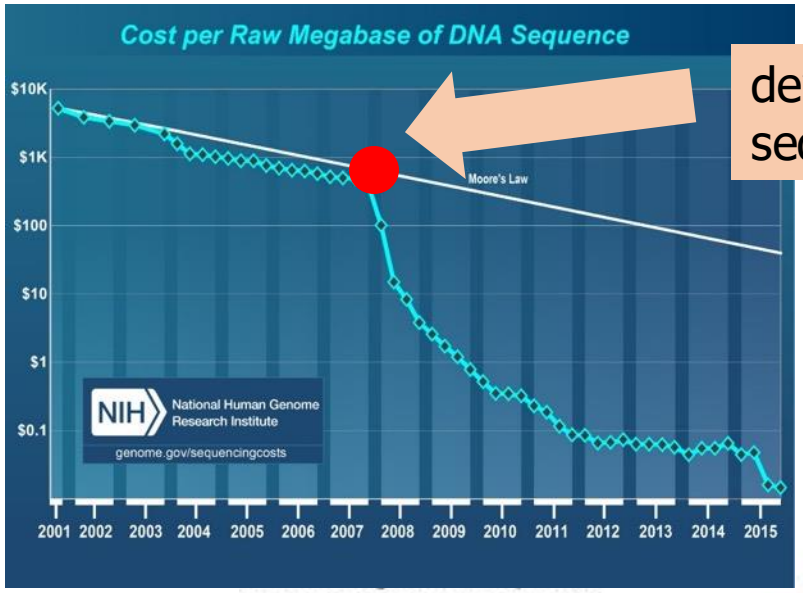
GRAINS
Genome Graph Analysis

MARS *(ICS'25)*
Raw Genomics Signal Analysis

Efficient Analysis on Compressed Genomic Data

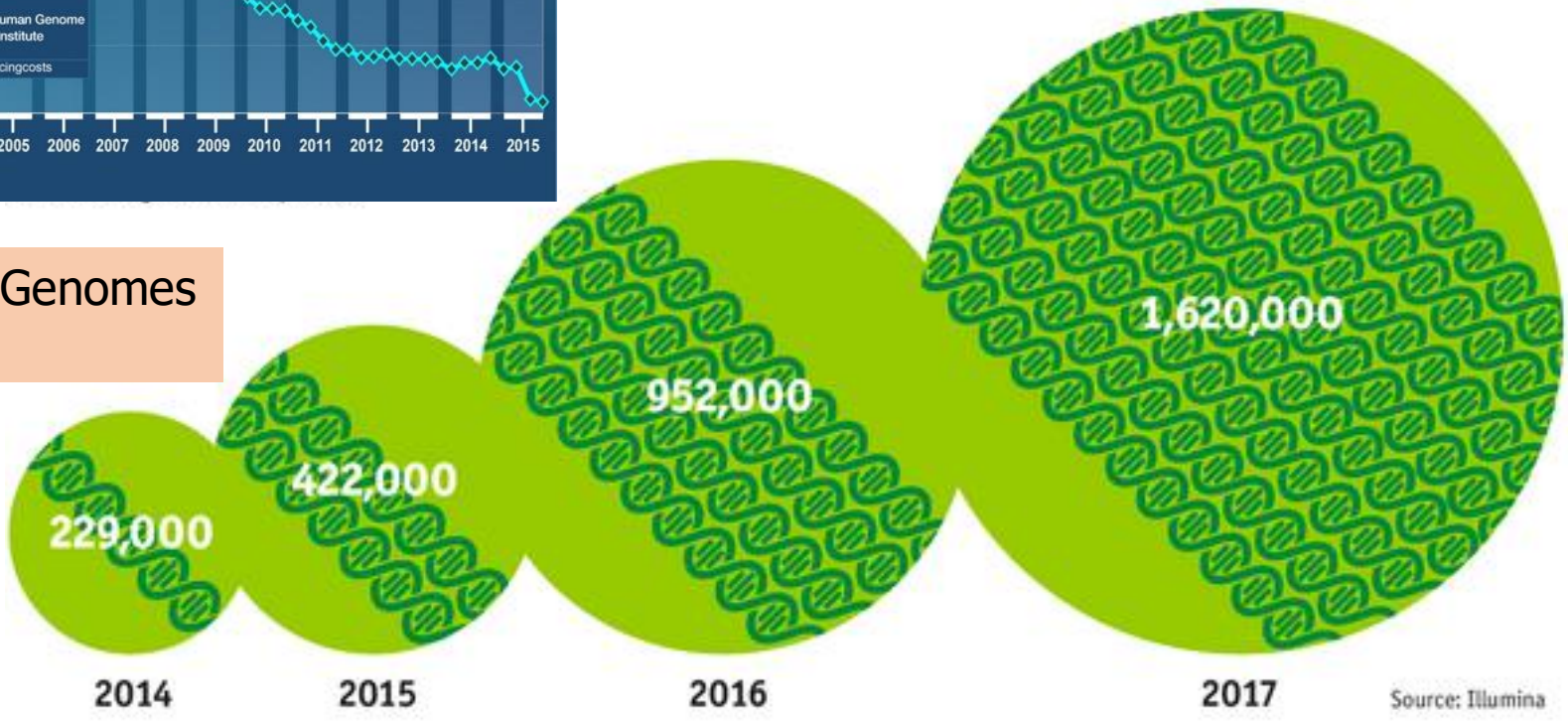
SAGe *(to appear in HPCA'26)*

Genomic Data is Growing at a Fast Pace



development of high-throughput sequencing (HTS) technologies

Number of Genomes Sequenced

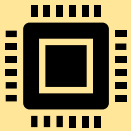


The Economist

SAFARI

http://www.economist.com/news/21631808-so-much-genetic-data-so-many-uses-genes-unzipped

Genomic Data is Growing at a Fast Pace



Accelerate genome sequence analysis



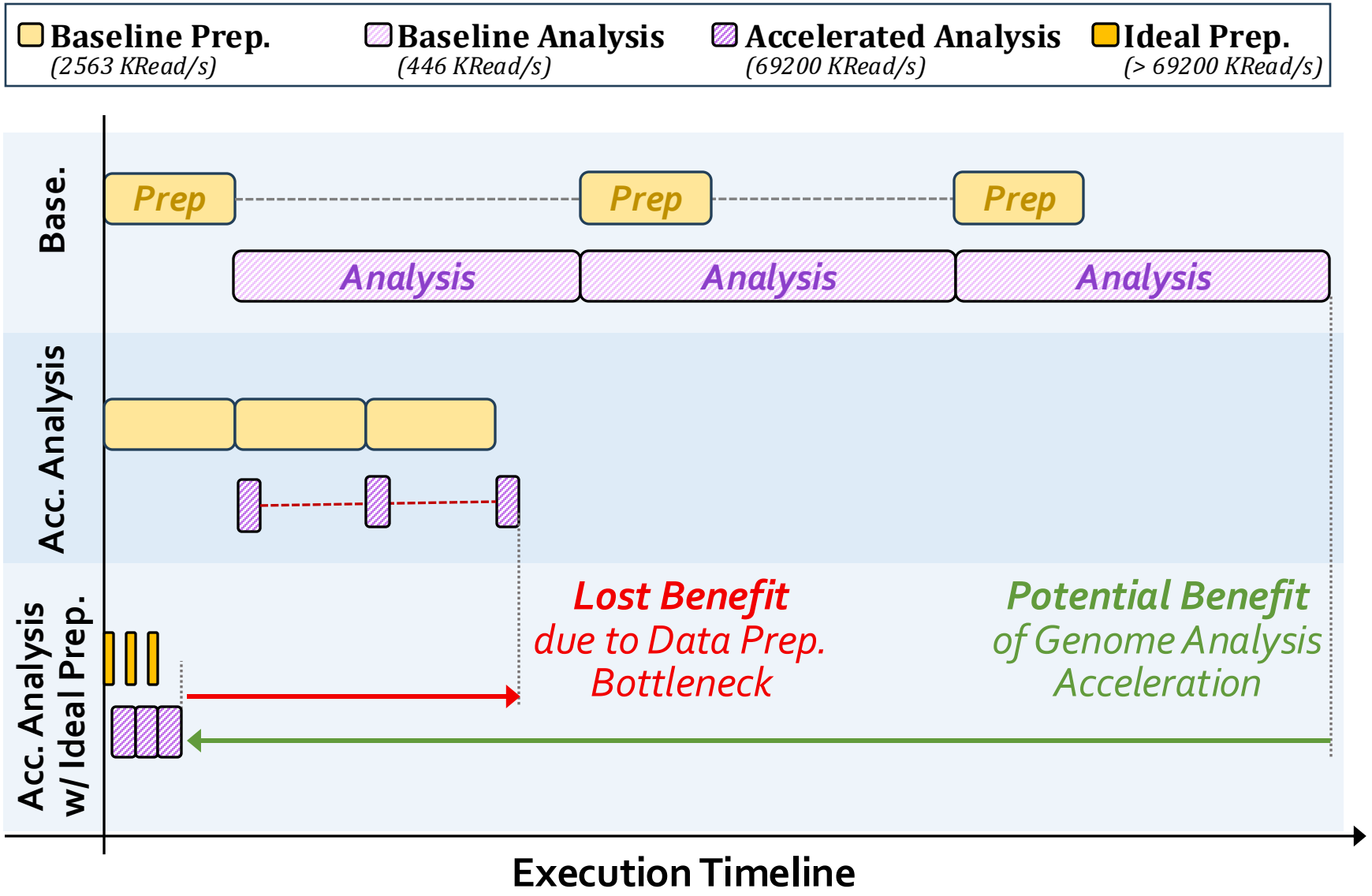
Store genome sequence data in highly-compressed formats

Data Preparation Bottleneck

Where genomic sequence data is stored in compressed form and needs to be **decompressed** and **formatted** first before an accelerator can operate on it

http://www.economist.com/news/21631808-so-much-genetic-data-so-many-uses-genes-unzipped

Effect of the Data Preparation Bottleneck



SAGe

- An algorithm-architecture co-design for **highly-compressed storage** and **high-performance access** of large-scale genomic sequence data
  - Carefully exploits **properties of genomic data** in synergistically co-designing algorithms and hardware

✓ **High compression ratios**

Comparable to state-of-the-art genomic-specific compressors

✓ **Fast decompression using only lightweight hardware and efficient streaming accesses**

Can integrate with genomics systems even in resource-constrained environments

- *Portable systems*
- *Near-data processing systems (e.g., in storage or memory)*

SAGe improves the average end-to-end performance (3.0×–32.1×) and energy efficiency (13.0×–34.0×) of genome sequence analysis accelerators

More in the Paper

SAGe: A Lightweight Algorithm-Architecture Co-Design for Mitigating the Data Preparation Bottleneck in Large-Scale Genome Sequence Analysis

Nika Mansouri Ghiasi[1] Talu Güloglu[1] Harun Mustafa[1] Can Firtina[1]
Konstantina Koliogeorgi[1] Konstantinos Kanellopoulos[1] Haiyu Mao[2] Rakesh Nadig[1]
Mohammad Sadrosadati[1] Jisung Park[3] Onur Mutlu[1]
[1]ETH Zürich [2]King's College London [3]POSTECH

https://arxiv.org/pdf/2504.03732

Conclusion

Genomics

GenStore *(ASPLOS'22)*

Metagenomics

MegIS *(ISCA'24)*

Storage-Centric Systems for (Meta)Genomics

Emergi

GRAINS

Genome Graph Analysis

enomics

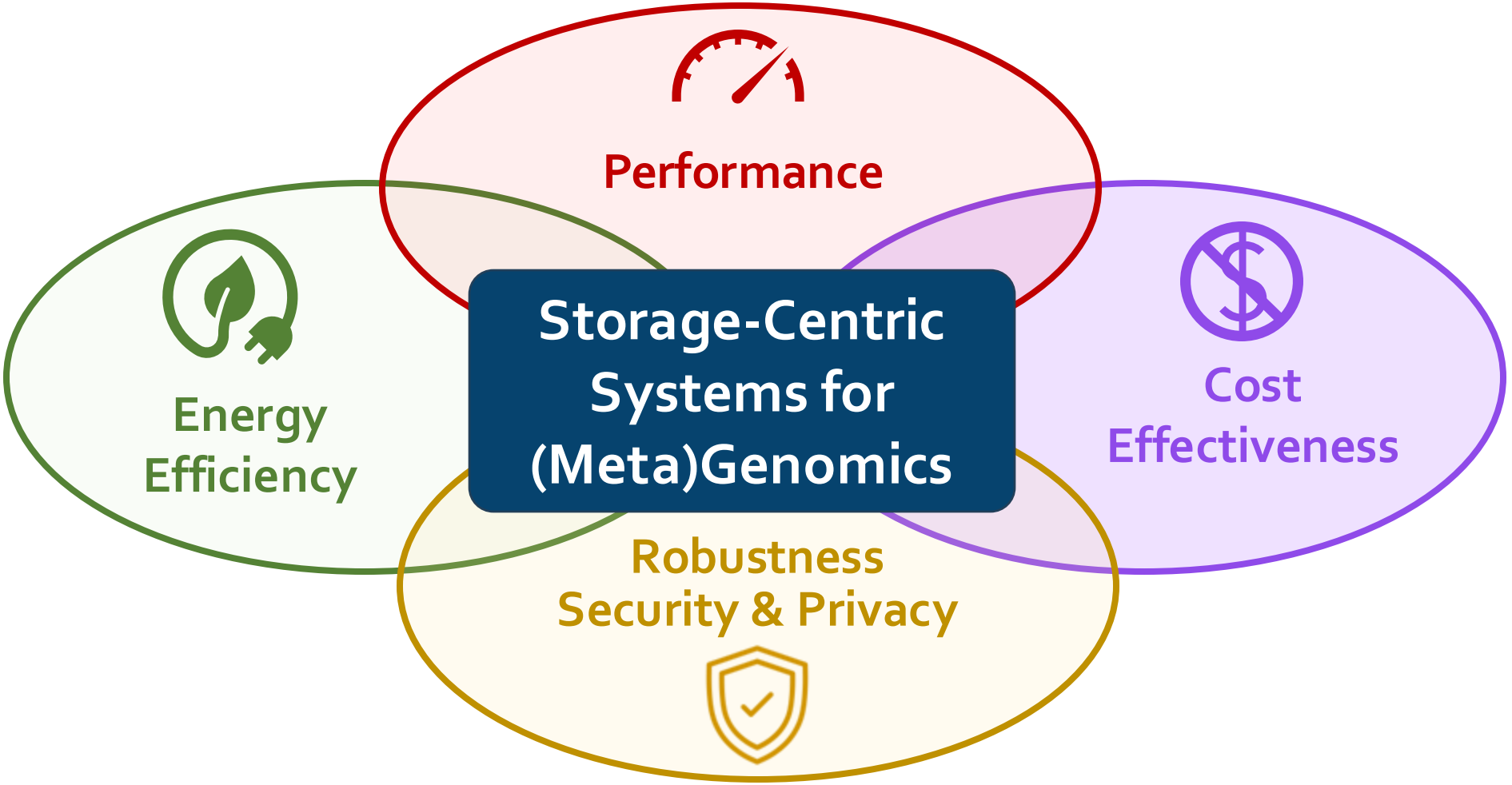
ARS *(ICS'25)*

Raw Genomics Signal Analysis

Efficient Analysis on Compressed Genomic Data

SAGe *(to appear in HPCA'26)*

Conclusion



Make accurate (meta)genomic analysis more accessible for wider adoption

Architecture for Health

First Workshop on Architecture for Health



Arch4Health

A half-day workshop @ MICRO 2025, Seoul, October 18th (Saturday)

Exploring the key computational challenges in health-related applications and the vital role of computer architecture in overcoming them

The workshop will cover 10 talks, spanning critical topics such as

- Genomics
- Proteomics
- Wearable systems
- Brain-computer interfaces
- Agent-based simulations

Full Agenda:

Livestream:

Opportunities



Recent biotechnological advances enable high-throughput, low-cost, and accurate biological data generation

- Genome sequencing
- Continuous wearable sensing
- Medical imagining
- …



This wealth of data enables unique opportunities for advancing healthcare

- Precision medicine
- Bedside personalized care
- Discovering early warning signs of communicable diseases
- Continuous physiological tracking
- Enhanced diagnostic capabilities
- …

Challenges

Efficiently analyzing large-scale biological data poses significant challenges for conventional computing systems



Performance & Scalability



Energy Efficiency



Privacy & Security



Cost Effectiveness

Goals of Arch4Health



Exploring the key computational challenges in health-related applications

&

The vital role of computer architecture in overcoming them to advance healthcare

Example Areas



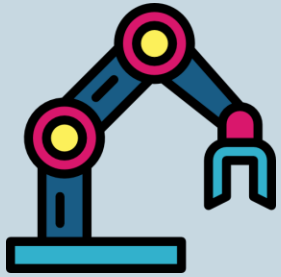
Computational Biology
- Genomics
- Metagenomics
- Gene Editing
- Drug Discovery
- Proteomics
- Other Areas in Precision Medicine



Neuroscience
- Brain-Machine Interfaces
- Prosthetics



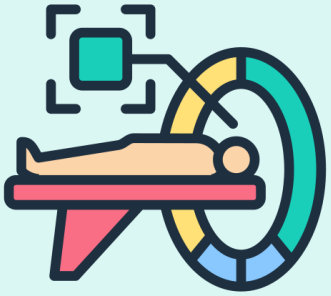
Wearable Systems & Bio-Sensors



Medical Robotics



Mental Health



Medical Imaging
- Brain Scans
- Radiology
- Single-Cell Analysis



Agent-Based Simulation

& More…

Agenda of Arch4Health @ MICRO 2025

Time	Speaker	Domain	Title
1:00–1:05	Nika Mansouri Ghiasi Dr. Konstantina Koliogeorgi Prof. Onur Mutlu	—	Welcome
1:05–1:25	Prof. Hunjun Lee *(Hanyang University)*	**Brain-Computer Interfaces**	Pathfinding Future BCI System with Full-Stack Design Space Exploration
1:25–1:45	Prof. Lizy John *(UT Austin)*	**Wearables**	Energy-Efficient Neural Networks for Medical Wearables
1:45–2:05	Prof. Sang-Woo Jun *(UC Irvine)*	**Genomics**	Addressing the Data Movement Overhead of Genomic Accelerators
2:05–2:25	Dr. Hasindu Gamaarachchi *(UNSW)*	**Real-Time Genomics**	Algorithms & Architectures for Long Read Sequence Analysis
2:25–2:45	Elton Shih *(Cornell University)*	**Real-Time Genomics**	Towards an Edge Algorithm–Hardware Co-Design Framework for Adaptive Sampling
2:45–3:05	Seunghee Han *(KAIST)*	**Genomics & Proteomics**	From Genomics to Proteomics: Architectural Solutions for Memory-Intensive Health Applications
3:05–3:30 — Coffee Break			
3:30–3:50	Nika Mansouri Ghiasi *(ETH Zurich)*	**(Meta)Genomics**	Storage-Centric Systems for Genomics and Metagenomics
3:50–4:10	Dr. Konstantina Koliogeorgi *(ETH Zurich)*	**Genomics**	MARS: Processing-In-Memory Acceleration of Raw Signal Genome Analysis Inside the Storage Subsystem
4:10–4:30	Dr. Noelia Oliete Escuin *(BSC)*	**Genomics**	A Tightly-Coupled Hardware Accelerator for Fast Pattern Exact-Matching in Genome Analysis
4:30–4:50	Dr. Lukas Breitwieser *(CERN)*	**Agent-Based Simulation**	Current State and Future Directions in Scalable Agent-Based Simulation with TeraAgent

More Information

2nd Arch4Health Workshop will be @ HPCA 2026!





More information:
https://events.safari.ethz.ch/hpca26-arch4health/

Computer Architecture

Lecture 25: Accelerating Biological Sequence Analysis via New Architectures and Algorithms

Nika Mansouri Ghiasi
Prof. Onur Mutlu
ETH Zürich

Fall 2025

19 December 2025